TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO DỰ ÁN
HỌC PHẦN HỌC SÂU (DEEP LEARNING)

Đề tài:

PHÁT HIỆN VÀ ĐỊNH VỊ BẤT THƯỜNG TRÊN SẢN PHẨM CÔNG NGHIỆP BẰNG HỌC KHÔNG GIÁM SÁT

Tên nhóm: **NHÓM 13**

Sinh viên thực hiện: **Trần Văn Dưỡng**
Đặng Hoàng An

Giáo viên hướng dẫn: **TS. Ngô Hữu Huy**

THÁI NGUYÊN, NĂM 2026

LỜI CẢM ƠN

Nhóm 13 xin gửi lời cảm ơn chân thành đến TS. Ngô Hữu Huy, giảng viên học phần Học sâu. Thầy đã định hướng cho nhóm từ lúc chọn đề tài và góp ý về cách đặt bài toán, cách so sánh các phương pháp và cách trình bày báo cáo. Những góp ý đó giúp nhóm hiểu rằng một kết quả chỉ có giá trị khi được đánh giá đúng cách, chứ không chỉ cần một con số cao.

Nhóm xin cảm ơn Khoa Công nghệ Thông tin, Trường Đại học Công nghệ Thông tin và Truyền thông, đã xây dựng học phần và tạo điều kiện để sinh viên được tự tay thực hiện một dự án học sâu hoàn chỉnh, từ dữ liệu, mô hình đến đánh giá và ứng dụng minh họa.

Nhóm cũng cảm ơn MVTec Software GmbH và nhóm tác giả bộ dữ liệu VisA đã công bố miễn phí các bộ dữ liệu MVTec AD, MVTec AD 2 và VisA cho mục đích nghiên cứu. Nếu không có các bộ dữ liệu này, nhóm khó có thể đánh giá ba phương pháp trên ảnh công nghiệp thật. Nhóm cũng cảm ơn các tác giả của PaDiM và PatchCore, vì bài báo của họ mô tả phương pháp đủ rõ để nhóm tự cài đặt lại.

Cuối cùng, hai thành viên cảm ơn nhau vì đã cùng chia việc, theo kịp tiến độ và hỗ trợ nhau trong những lúc gặp lỗi khi chạy thực nghiệm.

Do thời gian và kiến thức còn hạn chế, báo cáo khó tránh khỏi thiếu sót. Nhóm rất mong nhận được góp ý của thầy để hoàn thiện hơn.

MỤC LỤC

LỜI CẢM ƠN . . . . . **i**

DANH MỤC HÌNH . . . . . **v**

DANH MỤC BẢNG . . . . . **vi**

PHÂN CÔNG NHIỆM VỤ . . . . . **vii**

MỞ ĐẦU **1**

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT **2**

1.1 Trí tuệ nhân tạo, học máy và học sâu 2

1.1.1 Trí tuệ nhân tạo 2

1.1.2 Học máy 2

1.1.3 Học sâu 3

1.1.4 Quan hệ giữa ba khái niệm 4

1.2 Mạng nơ-ron và quá trình huấn luyện 4

1.2.1 Cấu trúc mạng nơ-ron 4

1.2.2 Hàm kích hoạt 6

1.2.3 Hàm mất mát 6

1.2.4 Quá trình huấn luyện và lan truyền ngược 7

1.2.5 Khởi tạo trọng số và tốc độ học 7

1.2.6 Thuật toán tối ưu 8

1.2.7 Quá khớp và cách hạn chế 8

1.2.8 Autoencoder và các biến thể 9

1.3 Mạng nơ-ron tích chập 9

1.3.1 Phép tích chập 9

1.3.2 Gộp, tích chập chuyển vị và chuẩn hóa theo lô 10

1.3.3 Trường tiếp nhận và thứ bậc đặc trưng 11

1.3.4 Các kiến trúc mạng tích chập tiêu biểu 11

1.4 Học chuyển giao: dùng lại đặc trưng của mạng học trên ImageNet 12

1.4.1 ImageNet và mạng học trước 12

1.4.2 Tinh chỉnh và trích đặc trưng cố định 13

1.4.3 Khoảng cách miền 13

1.5 Thị giác máy tính và xử lý ảnh số 14

1.5.1 Các nhiệm vụ cơ bản 14

1.5.2 Ảnh số và thay đổi kích thước ảnh 14

1.5.3 Lọc Gauss 15

1.5.4 Ứng dụng trong kiểm tra chất lượng 15

1.6 Bài toán phát hiện bất thường không giám sát 16

1.6.1 Kiểm tra lỗi sản phẩm công nghiệp và các khó khăn 16

1.6.2 Phát hiện và định vị bất thường . . . 16
1.6.3 Vì sao chỉ học từ ảnh tốt . . . 17
1.6.4 Hai loại bất thường . . . 17
1.6.5 Cơ sở thống kê của điểm bất thường . . . 17
1.6.6 Ngưỡng quyết định và đánh đổi giữa báo nhầm và bỏ sót . . . 18
1.6.7 Các bộ dữ liệu chuẩn . . . 19
1.6.8 Các hướng tiếp cận và vị trí của ba phương pháp . . . 19
1.6.9 Một số phương pháp tiêu biểu theo từng hướng . . . 20

CHƯƠNG 2. PHƯƠNG PHÁP PHÁT HIỆN BẤT THƯỜNG . . . 22
2.1 Phát biểu bài toán . . . 22
2.2 Tổng quan giải pháp . . . 22
2.3 Dữ liệu thực nghiệm và cách tiền xử lý ảnh . . . 24
2.3.1 Dữ liệu ảnh và các bước tiền xử lý . . . 24
2.3.2 Ba bộ dữ liệu và chín loại sản phẩm . . . 24
2.3.3 Tiền xử lý ảnh và chia tập fit, validation, test . . . 25
2.4 Lựa chọn phương pháp và giả thuyết . . . 26
2.5 Autoencoder . . . 27
2.5.1 Nguyên lý . . . 27
2.5.2 Kiến trúc . . . 27
2.5.3 Huấn luyện . . . 28
2.5.4 Bản đồ và điểm bất thường . . . 28
2.5.5 Ưu điểm và hạn chế . . . 28
2.6 PaDiM . . . 29
2.6.1 Nguyên lý . . . 29
2.6.2 Trích xuất đặc trưng . . . 29
2.6.3 Ước lượng phân phối và tính điểm . . . 30
2.6.4 Triển khai và bộ nhớ . . . 30
2.6.5 Ưu điểm và hạn chế . . . 30
2.7 PatchCore . . . 30
2.7.1 Nguyên lý . . . 30
2.7.2 Đặc trưng patch có ngữ cảnh lân cận . . . 31
2.7.3 Bộ nhớ và chọn coreset . . . 31
2.7.4 Tính điểm . . . 32
2.7.5 Ưu điểm và hạn chế . . . 32

CHƯƠNG 3. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ . . . 33
3.1 Chỉ số đánh giá . . . 33
3.1.1 Các chỉ số phân loại thông thường . . . 33
3.1.2 Đường cong ROC và AUROC . . . 33
3.1.3 AUPRO . . . 34
3.1.4 Đặt ngưỡng quyết định . . . 34
3.2 Tiêu chí so sánh và cấu hình thực nghiệm . . . 34
3.3 Công cụ, môi trường và cách chạy thực nghiệm . . . 35
3.3.1 Công cụ và thư viện . . . 35
3.3.2 Môi trường và thiết lập chung . . . 35

3.3.3 Tổ chức mã nguồn . . . 35
3.3.4 Quy trình chạy một loại sản phẩm . . . 36
3.3.5 Cách chạy trên Colab . . . 37
3.3.6 Các vấn đề kỹ thuật đã xử lý . . . 37
3.3.7 Đo thời gian và vẽ hình minh họa . . . 37
3.4 Kết quả trên 9 loại sản phẩm . . . 37
3.4.1 Kết quả tổng hợp . . . 37
3.4.2 Kết quả theo từng loại . . . 38
3.4.3 Kết quả tại ngưỡng . . . 39
3.4.4 Phân tích định tính . . . 40
3.5 Các thí nghiệm chuyên sâu . . . 42
3.5.1 Thiết lập các thí nghiệm . . . 42
3.5.2 Thí nghiệm 1: dữ liệu huấn luyện lẫn ảnh lỗi . . . 44
3.5.3 Thí nghiệm 2: chỉ có vài ảnh tốt . . . 45
3.5.4 Thí nghiệm 3: dùng mô hình cho loại sản phẩm khác . . . 49
3.5.5 Thí nghiệm 4: thay đổi điều kiện chụp . . . 52
3.5.6 Thí nghiệm 5: phân tích theo kiểu lỗi và các ca thất bại . . . 57
3.5.7 Thảo luận tổng hợp . . . 59
3.6 Chi phí tính toán . . . 61
3.7 So sánh với kết quả đã công bố . . . 62
3.8 Ứng dụng minh họa . . . 63
3.9 Hạn chế . . . 63

KẾT LUẬN . . . 64

TÀI LIỆU THAM KHẢO . . . 65

PHỤ LỤC. MÃ NGUỒN VÀ CÁCH TÁI LẬP . . . 69

DANH MỤC HÌNH

Hình 1.1 Phân loại thuật toán học máy theo việc dữ liệu có nhãn hay không 3
Hình 1.2 Học sâu: mạng nơ-ron nhiều lớp tự học đặc trưng từ dữ liệu 3
Hình 1.3 Quan hệ giữa trí tuệ nhân tạo, học máy và học sâu 5
Hình 1.4 Cấu trúc mạng nơ-ron nhiều lớp 5
Hình 1.5 Các tầng của Wide-ResNet-50-2 và tầng đặc trưng được sử dụng 12
Hình 1.6 Phát hiện đối tượng, một nhiệm vụ của thị giác máy tính 14

Hình 2.1 Thiết lập bài toán phát hiện và định vị bất thường 22
Hình 2.2 So sánh cơ chế của ba phương pháp 26
Hình 2.3 Autoencoder và cách tính bản đồ bất thường 27
Hình 2.4 Quy trình PaDiM 29
Hình 2.5 Quy trình PatchCore 31

Hình 3.1 Đường cong ROC và AUROC 34
Hình 3.2 Grid, lỗi cong lưới 41
Hình 3.3 Screw, lỗi ren bên thân 41
Hình 3.4 Transistor, lỗi lệch vị trí (cả linh kiện bị đặt lệch khỏi chỗ) 41
Hình 3.5 Pcb3, chân linh kiện bị cong 42
Hình 3.6 Vial, ảnh lỗi chụp ở điều kiện quá sáng 42
Hình 3.7 Sheet_metal, ba vết xước mảnh, ảnh chụp ở điều kiện quá sáng 43
Hình 3.8 Image-AUROC và AUPRO theo tỷ lệ ảnh lỗi trong tập huấn luyện 45
Hình 3.9 Image-AUROC và AUPRO theo số ảnh huấn luyện 46
Hình 3.10 Ảnh screw tốt ở nhiều góc xoay khác nhau, kèm góc ước lượng 48
Hình 3.11 Điểm bất thường theo khoảng cách góc tới ảnh huấn luyện gần nhất . . . 48
Hình 3.12 Image-AUROC trước và sau khi căn chỉnh góc trên screw 49
Hình 3.13 Ma trận image-AUROC nguồn và đích trên VisA 50
Hình 3.14 Ma trận image-AUROC nguồn và đích trên grid, screw, transistor 51
Hình 3.15 Image-AUROC khi thích nghi từ pcb1 sang loại khác 52
Hình 3.16 Image-AUROC, AUPRO và FPR theo cường độ từng loại thay đổi trên transistor 53
Hình 3.17 Image-AUROC, AUPRO và FPR theo cường độ từng loại thay đổi trên screw 54
Hình 3.18 Image-AUROC và AUPRO của ba phương pháp trên vial theo từng điều kiện chụp 55
Hình 3.19 Cùng một cảnh vial ở ba điều kiện chụp 56
Hình 3.20 Phân phối điểm trên vial theo điều kiện ánh sáng 56
Hình 3.21 Image-AUROC và AUPRO trên sheet_metal theo từng điều kiện chụp . . 56
Hình 3.22 Ví dụ từng kiểu lỗi của screw 58
Hình 3.23 Ví dụ từng kiểu lỗi của transistor với bản đồ của ba phương pháp 59
Hình 3.24 Ảnh screw tốt có điểm PaDiM cao nhất 60
Hình 3.25 Các ca thất bại của Autoencoder trên grid 61

DANH MỤC BẢNG

Bảng 1.1 Một số mốc phát triển của học sâu liên quan đến đề tài 4
Bảng 1.2 Các hàm kích hoạt thường gặp và nơi dùng trong đề tài 6
Bảng 1.3 Kích thước, số tham số và trường tiếp nhận các lớp tích chập của bộ mã hóa Autoencoder 10
Bảng 1.4 Các tầng của Wide-ResNet-50-2 với ảnh vào 256×256 12
Bảng 1.5 Các hướng tiếp cận phát hiện bất thường không giám sát trên ảnh . . . 20

Bảng 2.1 Các lựa chọn cấu hình và lý do 23
Bảng 2.2 Chín loại sản phẩm sử dụng 25
Bảng 2.3 Kiến trúc Autoencoder 28

Bảng 3.1 Tiêu chí so sánh và cách đo 35
Bảng 3.2 Cấu hình đã chạy của ba phương pháp 35
Bảng 3.3 Công cụ và thư viện sử dụng trong đề tài 36
Bảng 3.4 Môi trường chạy thực nghiệm 36
Bảng 3.5 Các thiết lập dùng chung cho mọi lần chạy 36
Bảng 3.6 Kết quả trung bình theo bộ dữ liệu (cao nhất in đậm) 38
Bảng 3.7 Image-AUROC và pixel-AUROC theo từng loại 38
Bảng 3.8 AUPRO theo từng loại (cao nhất in đậm) 39
Bảng 3.9 FPR và TPR tại ngưỡng đặt trên ảnh tốt tách riêng 40
Bảng 3.10 PatchCore trên vial theo điều kiện chụp 40
Bảng 3.11 Image-AUROC của mốc huấn luyện trên toàn bộ `train/good` 44
Bảng 3.12 Image-AUROC theo tỷ lệ ảnh lỗi trong tập huấn luyện (trung bình 5 seed) 44
Bảng 3.13 Image-AUROC và AUPRO theo số ảnh huấn luyện k 46
Bảng 3.14 Image-AUROC trên screw trước và sau khi căn chỉnh góc 48
Bảng 3.15 Độ sáng trung bình (thang 0–255) của các tập ảnh screw 49
Bảng 3.16 Image-AUROC trung bình trên và ngoài đường chéo của ma trận nguồn và đích 50
Bảng 3.17 Image-AUROC khi thích nghi từ nguồn pcb1 sang các loại đích 51
Bảng 3.18 Thay đổi AUPRO và FPR tại ngưỡng cố định ở mức thay đổi mạnh nhất 53
Bảng 3.19 Kết quả trên vial theo điều kiện chụp 55
Bảng 3.20 Độ nhạy của ba phương pháp với thay đổi điều kiện chụp 57
Bảng 3.21 Image-AUROC theo từng kiểu lỗi của screw 57
Bảng 3.22 Image-AUROC theo từng kiểu lỗi của transistor và grid 58
Bảng 3.23 Điểm mạnh và điểm yếu của ba phương pháp theo từng tình huống . . . 62
Bảng 3.24 Thời gian suy luận và dung lượng mô hình 62

PHÂN CÔNG NHIỆM VỤ

Mã nguồn được chia thành các module, mỗi module có một người phụ trách chính để hai thành viên làm song song trên nhánh riêng mà không sửa đè lên nhau. Phần dùng chung (cấu hình, giao thức đánh giá) được thay đổi qua yêu cầu hợp nhất (pull request) để cả hai cùng xem trước khi đưa vào nhánh chính. Module dữ liệu và module đánh giá là nền cho mọi phương pháp nên được làm trước, trong hai tuần đầu.

Bảng phân công theo công việc

Thành viên	Nhiệm vụ
Đặng Hoàng An	Xây dựng và huấn luyện Autoencoder; chạy PatchCore trên 5 loại của MVTec AD và MVTec AD 2 (grid, screw, transistor, vial, sheet_metal); demo; tài liệu hướng dẫn của kho mã nguồn.
Trần Văn Dưỡng	Module đọc dữ liệu MVTec AD, MVTec AD 2 và VisA; module đánh giá (AUROC, AUPRO, ngưỡng, hình minh họa, ghi kết quả); PaDiM; chạy PatchCore trên 4 loại của VisA (pcb1–pcb4); năm thí nghiệm chuyên sâu.
Cả nhóm	Cấu hình và giao thức đánh giá chung; chạy đủ 9 loại trên Colab; tổng hợp kết quả; viết báo cáo.

Bảng phân công theo tiến độ

Thời gian	Đặng Hoàng An	Trần Văn Dưỡng	Cả nhóm
Tuần 1 19/10–25/10	Mô hình Autoencoder (hạn 22/10)	Module đọc dữ liệu (hạn 22/10)	Chọn đề tài, chia module, dựng khung mã nguồn
Tuần 2 26/10–01/11	Huấn luyện và đánh giá Autoencoder	Module đánh giá (hạn 26/10); PaDiM (hạn 29/10)	Chạy thử trên Colab; viết Chương 1
Tuần 3 02/11–08/11	Chạy Autoencoder; PatchCore trên 5 loại MVTec AD, MVTec AD 2	Chạy PaDiM; PatchCore trên 4 loại VisA; năm thí nghiệm chuyên sâu	Chạy đủ 9 loại trên Colab; viết Chương 2
Tuần 4 09/11–15/11	Ứng dụng minh họa (hạn 15/11)	Vẽ hình minh họa, kiểm tra số liệu	Viết Chương 3, hoàn thiện báo cáo

MỞ ĐẦU

Kiểm tra chất lượng là khâu bắt buộc trong sản xuất công nghiệp: sản phẩm có vết xước, nứt, nhiễm bẩn hay thiếu chi tiết phải bị loại trước khi đến tay người dùng. Kiểm tra bằng mắt người chậm, phụ thuộc kinh nghiệm và dễ bỏ sót khi sản lượng lớn hoặc lỗi nhỏ. Thị giác máy tính có thể tự động hóa khâu này, nhưng gặp một khó khăn về dữ liệu: ảnh sản phẩm lỗi hiếm, đa dạng và khó lường trước, còn việc khoanh vùng lỗi trên từng ảnh để làm nhãn tốn nhiều công sức, trong khi mô hình phân loại có giám sát chỉ nhận ra những kiểu lỗi đã thấy. Ngược lại, ảnh sản phẩm tốt có sẵn với số lượng lớn. Vì vậy, hướng học không giám sát, tức chỉ học từ ảnh tốt rồi coi mọi sai khác đáng kể là bất thường, phù hợp hơn với điều kiện thực tế [1].

Đề tài xây dựng và đánh giá ba phương pháp phát hiện và định vị bất thường không giám sát trên ảnh sản phẩm công nghiệp, nhằm trả lời ba câu hỏi: phương pháp nào phát hiện và khoanh vùng lỗi chính xác nhất; phương pháp nào phù hợp với loại sản phẩm nào (bề mặt hay vật thể, vị trí cố định hay xoay, ánh sáng ổn định hay thay đổi); và khi chốt một ngưỡng để dùng thật thì mỗi phương pháp báo nhầm và bỏ sót bao nhiêu.

Ba phương pháp được chọn đại diện cho hai hướng tiếp cận. Autoencoder học tái tạo ảnh từ đầu và dùng sai số tái tạo làm tín hiệu bất thường. PaDiM [2] và PatchCore [3] dùng đặc trưng của mạng tích chập đã học trước trên ImageNet, lần lượt mô hình hóa đặc trưng bình thường bằng phân phối Gauss theo vị trí và bằng một bộ nhớ các vector đặc trưng. Cả ba do nhóm tự cài đặt. Dữ liệu là 9 loại sản phẩm từ ba bộ dữ liệu công khai: grid, screw, transistor của MVTec AD [1]; vial, sheet_metal của MVTec AD 2 [4], có ảnh test chụp ở nhiều điều kiện ánh sáng; và bốn loại bảng mạch pcb1–pcb4 của VisA [5]. Đề tài dừng ở mức đánh giá trên dữ liệu ảnh có sẵn và một ứng dụng minh họa, chưa triển khai trên dây chuyền.

Về cách làm, nhóm tìm hiểu cơ chế của từng phương pháp qua bài báo gốc, cài đặt chúng theo cùng một giao thức, xây dựng mô hình cho cả 9 loại và đánh giá bằng image-AUROC cho nhiệm vụ phát hiện ảnh lỗi, pixel-AUROC và AUPRO cho nhiệm vụ định vị vùng lỗi, cùng tỷ lệ báo nhầm và tỷ lệ bắt lỗi tại ngưỡng đặt trên ảnh tốt tách riêng. Kết quả cho thấy PatchCore tốt nhất ở cả ba bộ dữ liệu, với image-AUROC trung bình 0,937 và bắt được 75% ảnh lỗi khi chỉ báo nhầm 6% ảnh tốt. PaDiM (0,828) và Autoencoder (0,820) gần bằng nhau: PaDiM hơn trên MVTec AD, Autoencoder hơn trên VisA. Cả ba phương pháp giảm mạnh trên MVTec AD 2: vial vì ánh sáng lúc kiểm thử khác lúc huấn luyện, sheet_metal vì vết xước quá mảnh so với độ phân giải của đặc trưng. Ngoài ra, năm thí nghiệm chuyên sâu kiểm tra ba phương pháp khi tập huấn luyện lẫn ảnh lỗi, khi chỉ có vài ảnh tốt, khi đổi loại sản phẩm, khi điều kiện chụp thay đổi, và phân tích các ca thất bại theo từng kiểu lỗi.

Chương 1 trình bày cơ sở lý thuyết: học sâu, mạng tích chập, đặc trưng học trước và bài toán phát hiện bất thường. Chương 2 trình bày dữ liệu, cách tiền xử lý và cách triển khai ba phương pháp. Chương 3 trình bày chỉ số đánh giá, cách chạy thực nghiệm, kết quả trên 9 loại, năm thí nghiệm chuyên sâu và ứng dụng minh họa.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

Chương này đặt đề tài vào bức tranh chung của trí tuệ nhân tạo và học máy, rồi trình bày các nền tảng mà ba phương pháp dựa vào: huấn luyện mạng nơ-ron (Autoencoder), mạng tích chập (cả ba phương pháp), việc dùng lại đặc trưng của mạng học trước (PaDiM, PatchCore), xử lý ảnh số và bài toán phát hiện bất thường cùng cơ sở thống kê của nó.

1.1. Trí tuệ nhân tạo, học máy và học sâu

1.1.1. Trí tuệ nhân tạo

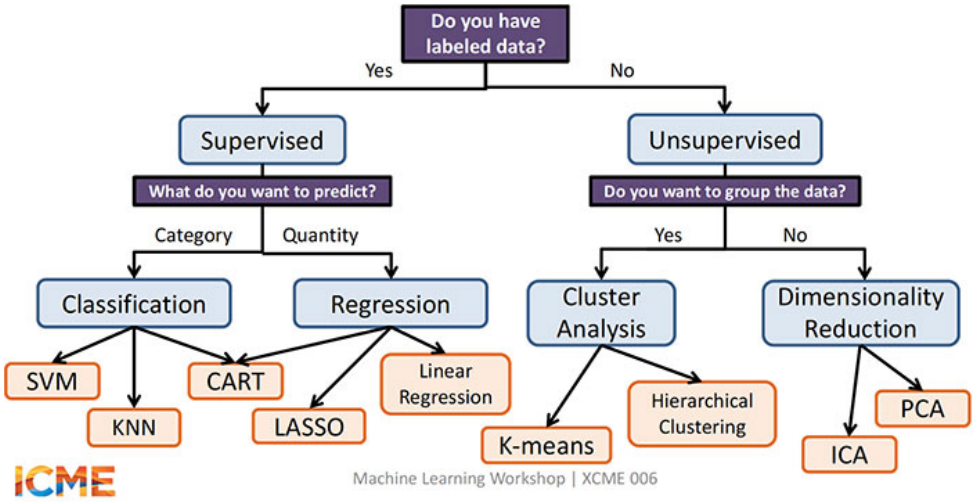
Trí tuệ nhân tạo (Artificial Intelligence, AI) là lĩnh vực xây dựng các hệ thống máy tính thực hiện được những việc vốn cần trí tuệ con người, như nhận biết, suy luận, học từ kinh nghiệm và ra quyết định. AI thường được chia làm ba cấp độ. AI hẹp (narrow AI) giải quyết một nhiệm vụ cụ thể, như nhận dạng khuôn mặt hay gợi ý sản phẩm. AI tổng quát (general AI) học và làm được mọi nhiệm vụ trí tuệ như con người. AI siêu việt (super AI) vượt con người ở mọi lĩnh vực. Hai cấp độ sau hiện vẫn ở mức lý thuyết; mọi hệ thống đang dùng trong thực tế, kể cả hệ thống kiểm tra lỗi sản phẩm của đề tài, đều là AI hẹp.

1.1.2. Học máy

Học máy (Machine Learning, ML) là một nhánh của AI, trong đó hệ thống tự rút ra quy luật từ dữ liệu thay vì được lập trình sẵn từng quy tắc. Mô hình được xây dựng trên dữ liệu huấn luyện, rồi được dùng để dự đoán hoặc ra quyết định cho dữ liệu mới mà nó chưa thấy. Vì vậy, điều quan trọng của một mô hình học máy không phải là thuộc lòng dữ liệu huấn luyện mà là tổng quát hóa được sang dữ liệu mới [6].

Theo cách dùng dữ liệu, học máy chia làm ba nhóm (Hình 1.1). Trong học có giám sát, dữ liệu đã được gán nhãn và mô hình học cách dự đoán nhãn, ví dụ phân loại ảnh sản phẩm thành "tốt" hoặc "lỗi" khi có sẵn nhiều ảnh lỗi đã gán nhãn. Trong học không giám sát, dữ liệu không có nhãn và mô hình tìm cấu trúc ẩn trong dữ liệu, như nhóm các mẫu giống nhau, giảm số chiều hoặc nhận ra mẫu nào khác thường. Trong học tăng cường, hệ thống học qua tương tác với môi trường và nhận thưởng hoặc phạt cho từng hành động; nhóm này không liên quan đến đề tài.

Dù thuộc nhóm nào, một bài toán học máy thường đi qua cùng một quy trình: thu thập dữ liệu, tiền xử lý và chia tập, chọn mô hình, huấn luyện, đánh giá trên dữ liệu chưa dùng khi huấn luyện, rồi triển khai và theo dõi. Báo cáo đi theo đúng trình tự này: dữ liệu và tiền xử lý ở Mục 2.3, mô hình và cách huấn luyện ở Mục 2.5 đến 2.7, đánh giá ở Chương 3, còn triển khai dừng ở mức một ứng dụng minh họa (Mục 3.8). Điểm khác của bài toán phát hiện bất thường nằm ở bước chia tập: dữ liệu huấn luyện chỉ gồm ảnh tốt, còn ảnh lỗi chỉ xuất hiện ở bước đánh giá.

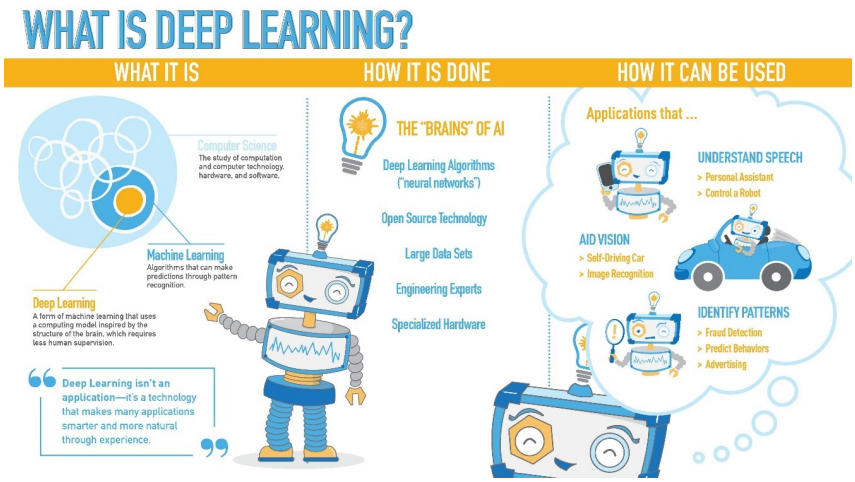


Hình 1.1. Phân loại thuật toán học máy theo việc dữ liệu có nhãn hay không

Phát hiện bất thường trong đề tài thuộc học không giám sát theo nghĩa hẹp: mô hình chỉ được thấy ảnh tốt, không thấy ảnh lỗi hay nhãn vùng lỗi nào khi học. Một số tài liệu gọi thiết lập này là học một lớp, vì dữ liệu huấn luyện chỉ thuộc một lớp là "bình thường". Mục 1.6 trình bày chi tiết bài toán này.

1.1.3. Học sâu

Học sâu (Deep Learning, DL) là một nhánh của học máy, dùng mạng nơ-ron nhiều lớp để học cách biểu diễn dữ liệu. Điểm khác biệt chính so với học máy truyền thống là đặc trưng không do con người thiết kế mà được mạng tự học từ dữ liệu, theo thứ bậc: với ảnh, các lớp đầu học cạnh và kết cấu, các lớp sau ghép chúng thành bộ phận và vật thể [7]. Học sâu phát triển mạnh nhờ ba yếu tố: lượng dữ liệu lớn, sức tính của GPU, và các thư viện như PyTorch giúp xây dựng và huấn luyện mạng dễ dàng hơn (Hình 1.2).



Hình 1.2. Học sâu: mạng nơ-ron nhiều lớp tự học đặc trưng từ dữ liệu

Học sâu không phải ý tưởng mới; các khái niệm nền tảng đã có từ nhiều thập kỷ trước,

nhưng chỉ trở nên thực tế khi có đủ dữ liệu và sức tính. Bảng 1.1 tóm tắt các mốc liên quan trực tiếp đến những kỹ thuật được dùng trong đề tài.

Bảng 1.1. Một số mốc phát triển của học sâu liên quan đến đề tài

Năm	Mốc	Liên quan đến đề tài
1958	Perceptron [8]	Mô hình nơ-ron nhân tạo, công thức (1.1)
1986	Lan truyền ngược [9]	Cách huấn luyện Autoencoder
1998	LeNet [10]	Lớp tích chập, nền của cả ba phương pháp
2009	ImageNet [11]	Dữ liệu huấn luyện của backbone dùng cho PaDiM, PatchCore
2012	AlexNet [12]	Huấn luyện mạng sâu trên GPU, ReLU, dropout
2015	Chuẩn hóa theo lô [13], Adam [14]	Dùng khi huấn luyện Autoencoder
2016	ResNet [15], Wide ResNet [16]	Backbone Wide-ResNet-50-2
2019	MVTec AD [1]	Bộ dữ liệu chuẩn cho phát hiện bất thường công nghiệp
2021–2022	PaDiM [2], PatchCore [3]	Hai trong ba phương pháp của đề tài

Mỗi kiến trúc học sâu tiêu biểu hợp với một loại dữ liệu. Mạng nơ-ron tích chập (CNN) xử lý ảnh và giữ được cấu trúc không gian; đây là nền của cả ba phương pháp trong đề tài (Mục 1.3). Autoencoder nén rồi dựng lại dữ liệu, được dùng để giảm chiều hoặc phát hiện bất thường, và là một trong ba phương pháp của đề tài. Mạng hồi quy (RNN, LSTM) xử lý dữ liệu chuỗi như văn bản, tiếng nói. Transformer dùng cơ chế chú ý để mô hình hóa quan hệ giữa các phần của dữ liệu, phổ biến trong xử lý ngôn ngữ và ngày càng dùng nhiều cho ảnh. Hai kiến trúc sau không dùng trong đề tài.

1.1.4. Quan hệ giữa ba khái niệm

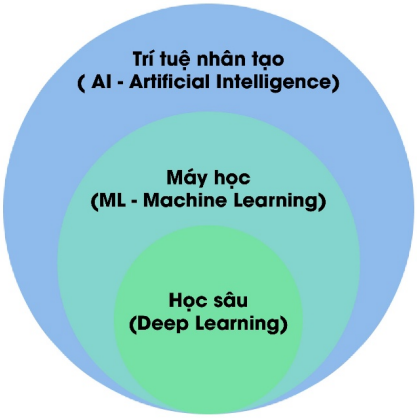
Ba khái niệm lồng nhau: học sâu là một phần của học máy, học máy là một phần của trí tuệ nhân tạo (Hình 1.3). Cả ba phương pháp của đề tài đều thuộc học sâu: Autoencoder là một mạng nơ-ron tự huấn luyện, còn PaDiM và PatchCore dựa trên đặc trưng của một mạng nơ-ron sâu đã huấn luyện sẵn.

1.2. Mạng nơ-ron và quá trình huấn luyện

1.2.1. Cấu trúc mạng nơ-ron

Một nơ-ron nhân tạo nhận vector đầu vào x, tính tổ hợp tuyến tính với trọng số w và độ lệch b, rồi đưa qua hàm kích hoạt phi tuyến φ:

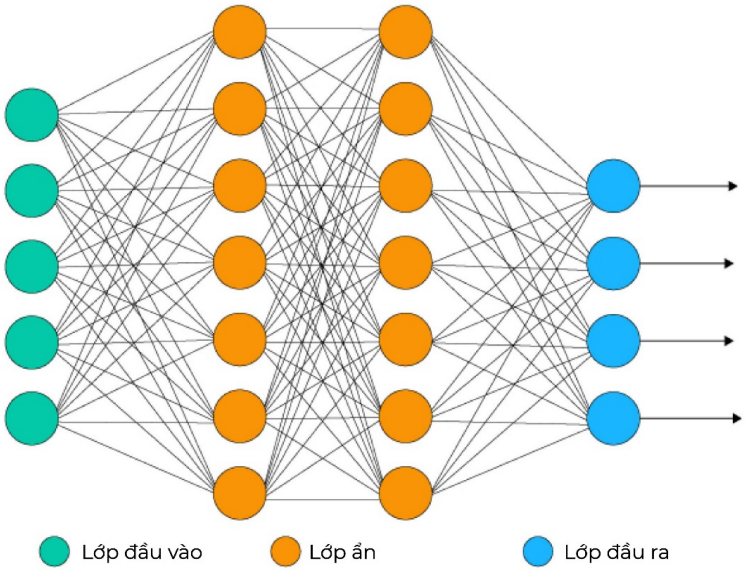
$$y = \varphi\big(w^{\top} x + b\big) \tag{1.1}$$



Hình 1.3. Quan hệ giữa trí tuệ nhân tạo, học máy và học sâu

Nhiều nơ-ron xếp thành lớp, nhiều lớp nối tiếp thành mạng; đầu ra của lớp trước là đầu vào của lớp sau (Hình 1.4) [17]. Viết gọn cho cả lớp thứ l, với ma trận trọng số $W^{(l)}$ và vector độ lệch $b^{(l)}$:

$$a^{(l)} = W^{(l)} h^{(l-1)} + b^{(l)}, \qquad h^{(l)} = \varphi\big(a^{(l)}\big), \qquad h^{(0)} = x. \tag{1.2}$$



Hình 1.4. Cấu trúc mạng nơ-ron nhiều lớp

Mạng gồm lớp vào nhận dữ liệu thô, các lớp ẩn biến đổi dữ liệu thành đặc trưng ngày càng trừu tượng, và lớp ra cho kết quả cuối cùng. Mạng có nhiều lớp ẩn được gọi là mạng sâu; càng nhiều lớp, mạng càng biểu diễn được quan hệ phức tạp, nhưng cũng càng khó huấn luyện và càng cần nhiều dữ liệu.

Một lớp kết nối đầy đủ từ $n_{\text{vào}}$ sang n_{ra} nơ-ron có $n_{\text{vào}} n_{\text{ra}} + n_{\text{ra}}$ tham số. Con số này tăng rất nhanh: lớp nén $16384 \to 256$ ở cổ chai Autoencoder của đề tài (Mục 2.5) có $16384 \cdot 256 + 256 \approx 4{,}19$ triệu tham số, và hai lớp kết nối đầy đủ ở cổ chai chiếm khoảng 70% tham số của cả mạng. Đây là lý do các lớp xử lý ảnh dùng phép tích chập thay vì kết nối đầy đủ (Mục 1.3).

1.2.2. Hàm kích hoạt

Hàm kích hoạt phi tuyến là điều kiện để mạng nhiều lớp biểu diễn được quan hệ phi tuyến: nếu bỏ φ trong (1.2), tích các ma trận $W^{(L)} \cdots W^{(1)}$ vẫn chỉ là một phép biến đổi tuyến tính, nên thêm lớp không làm mạng mạnh hơn. Bốn hàm thường gặp được tóm tắt ở Bảng 1.2.

Bảng 1.2. Các hàm kích hoạt thường gặp và nơi dùng trong đề tài

Hàm	Công thức	Miền giá trị	Dùng trong đề tài
Sigmoid	$\sigma(t) = 1/(1+e^{-t})$	$(0, 1)$	Lớp ra của Autoencoder
Tanh	$\tanh(t) = (e^{t} - e^{-t})/(e^{t} + e^{-t})$	$(-1, 1)$	Không dùng
ReLU	$\max(0, t)$	$[0, \infty)$	Wide-ResNet-50-2; bộ giải mã Autoencoder
LeakyReLU	$\max(t, \alpha t)$, $0 < \alpha < 1$	$(-\infty, \infty)$	Bộ mã hóa Autoencoder, $\alpha = 0{,}2$

Sigmoid và tanh bão hòa khi $|t|$ lớn: đạo hàm của sigmoid lớn nhất chỉ bằng 0,25 và gần 0 ở hai đầu. Khi lan truyền ngược qua nhiều lớp, gradient bị nhân với các đạo hàm nhỏ này liên tiếp và gần như biến mất ở các lớp đầu, khiến mạng sâu học rất chậm. ReLU [18] có đạo hàm bằng 1 với mọi $t > 0$ nên tránh được hiện tượng này và tính rất nhanh; nó là hàm kích hoạt của hầu hết mạng tích chập hiện đại, trong đó có Wide-ResNet-50-2. Nhược điểm của ReLU là nơ-ron có $t < 0$ với mọi đầu vào sẽ có gradient bằng 0 và ngừng học. LeakyReLU giữ một độ dốc nhỏ α ở nửa âm để gradient luôn khác 0; bộ mã hóa của Autoencoder dùng hàm này. Lớp ra của Autoencoder dùng sigmoid vì giá trị điểm ảnh tái tạo cần nằm trong $[0, 1]$, cùng thang với ảnh đầu vào.

1.2.3. Hàm mất mát

Hàm mất mát đo sai lệch giữa đầu ra của mạng và mục tiêu, và là đại lượng mà quá trình huấn luyện làm nhỏ. Với bài toán dựng lại ảnh, mục tiêu chính là ảnh đầu vào x, và hàm thường dùng nhất là sai số bình phương trung bình (MSE) trên n giá trị điểm ảnh:

$$L_{\text{MSE}}(x, \hat{x}) = \frac{1}{n} \sum_{i=1}^{n} (x_i - \hat{x}_i)^2. \tag{1.3}$$

Với bài toán phân loại hai lớp, mạng cho xác suất p để mẫu thuộc lớp dương, và hàm thường dùng là entropy chéo $L = -[\, y \log p + (1-y) \log(1-p) \,]$ với nhãn $y \in \{0, 1\}$. Đề tài không có bài toán phân loại nào, nên chỉ dùng MSE cho Autoencoder.

Làm nhỏ MSE tương đương tìm $\hat{x}$ có khả năng lớn nhất khi giả định ảnh thật bằng ảnh dựng lại cộng nhiễu Gauss độc lập ở từng điểm ảnh: $-\log p(x \mid \hat{x}) = \|x - \hat{x}\|^2/(2\sigma^2) +$ hằng số. Giả định này có hai hệ quả cho phát hiện bất thường. Thứ nhất, MSE phạt nặng sai số lớn và nhẹ sai số nhỏ, nên một vết xước mảnh chỉ lệch vài điểm ảnh đóng góp rất ít vào mất mát. Thứ hai, khi nhiều chi tiết đều có thể đúng (vị trí chính xác của từng sợi ren, từng hạt trên bề mặt kim loại), lời giải tốt nhất theo MSE là trung bình của chúng, nên ảnh dựng lại mờ ở cạnh và kết

cấu. Hàm mất mát dựa trên SSIM [19] so sánh độ sáng trung bình, độ tương phản và cấu trúc của từng vùng nhỏ thay vì từng điểm ảnh, và được đề xuất để khắc phục hai hạn chế trên; đề tài chưa thử hàm này.

1.2.4. Quá trình huấn luyện và lan truyền ngược

Huấn luyện là tìm tham số θ (toàn bộ trọng số và độ lệch) làm nhỏ hàm mất mát $L(\theta)$ trên dữ liệu huấn luyện [20]. Trọng số trước hết được khởi tạo ngẫu nhiên theo một phân phối phù hợp, để các nơ-ron không học giống hệt nhau. Mỗi vòng huấn luyện đưa một lô dữ liệu qua mạng để được đầu ra dự đoán (lan truyền tiến), rồi tính mất mát, tính gradient $\nabla_\theta L$ (lan truyền ngược) và cập nhật tham số ngược hướng gradient. Phương pháp hạ gradient cơ bản cập nhật với tốc độ học η:

$$\theta_{k+1} = \theta_k - \eta\, \nabla_\theta L(\theta_k). \tag{1.4}$$

Lan truyền ngược [9] tính gradient cho mọi lớp bằng quy tắc đạo hàm hàm hợp, đi từ lớp ra về lớp vào. Với mạng (1.2), đặt $\delta^{(l)} = \partial L / \partial a^{(l)}$, ta có

$$\delta^{(l)} = \left[\left(W^{(l+1)}\right)^\top \delta^{(l+1)}\right] \odot \varphi'\left(a^{(l)}\right), \qquad \frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} \left(h^{(l-1)}\right)^\top, \qquad \frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}, \tag{1.5}$$

với $\odot$ là phép nhân từng phần tử. Công thức cho thấy gradient ở lớp l là tích của các ma trận trọng số và các đạo hàm φ' của mọi lớp phía sau. Nếu các thừa số này nhỏ hơn 1, gradient giảm theo cấp số nhân khi đi về các lớp đầu; nếu lớn hơn 1, gradient tăng vọt. Đây là gốc của hai kỹ thuật trình bày ở mục sau, ReLU và chuẩn hóa theo lô, và của kết nối tắt trong ResNet (Mục 1.3.4). PyTorch tự tính (1.5) cho mọi phép toán, nên đề tài chỉ cần định nghĩa mạng và hàm mất mát.

1.2.5. Khởi tạo trọng số và tốc độ học

Cách khởi tạo trọng số ảnh hưởng trực tiếp đến việc gradient có truyền được qua mạng sâu hay không. Nếu mọi trọng số bằng nhau, mọi nơ-ron trong một lớp nhận cùng gradient và học giống hệt nhau; nếu trọng số quá lớn hoặc quá nhỏ, phương sai của tín hiệu tăng hoặc giảm theo cấp số nhân qua các lớp, đúng hiện tượng đã thấy ở (1.5). Khởi tạo Xavier [21] chọn phương sai của trọng số để giữ phương sai tín hiệu gần như không đổi qua mỗi lớp, $\mathrm{Var}(w) = 2/(n_{\text{vào}} + n_{\text{ra}})$, phù hợp với hàm kích hoạt đối xứng như tanh. Với ReLU, một nửa đầu vào bị đưa về 0, nên khởi tạo He [22] nhân đôi phương sai, $\mathrm{Var}(w) = 2/n_{\text{vào}}$. PyTorch mặc định khởi tạo các lớp tích chập theo một biến thể của cách thứ hai, và Autoencoder của đề tài dùng mặc định này. Với PaDiM và PatchCore, câu hỏi khởi tạo không đặt ra vì trọng số được lấy nguyên từ mạng đã học trên ImageNet.

Tốc độ học η trong (1.4) là siêu tham số quan trọng nhất của quá trình huấn luyện. η quá lớn làm mất mát dao động hoặc tăng vọt; η quá nhỏ làm mạng học rất chậm và dễ dừng ở một điểm kém. Nhiều quy trình huấn luyện giảm dần η theo một lịch cố định, để đi nhanh ở đầu và tinh chỉnh ở cuối. Đề tài giữ η cố định vì Adam đã tự điều chỉnh bước cho từng tham số và số

bước huấn luyện tương đối ít.

1.2.6. Thuật toán tối ưu

Tính gradient trên toàn bộ dữ liệu ở mỗi bước rất tốn, nên trong thực tế mỗi lần cập nhật dùng một lô nhỏ (mini-batch) B mẫu; gradient trên lô là ước lượng không chệch nhưng có nhiễu của gradient thật. Một lượt đi qua hết dữ liệu gọi là một epoch. Hạ gradient ngẫu nhiên (SGD) dùng trực tiếp ước lượng này trong (1.4), với cùng một tốc độ học cho mọi tham số. Biến thể có quán tính (momentum) cộng dồn gradient của các bước trước để đi nhanh theo hướng ổn định và giảm dao động theo hướng nhiễu.

Adam [14] kết hợp quán tính với một tốc độ học riêng cho từng tham số. Với gradient g_k ở bước k, Adam giữ trung bình động của gradient và của bình phương gradient, hiệu chỉnh độ chệch do khởi tạo bằng 0, rồi cập nhật:

$$\begin{aligned} m_k &= \beta_1 m_{k-1} + (1-\beta_1)\, g_k, & v_k &= \beta_2 v_{k-1} + (1-\beta_2)\, g_k^2, \\ \hat{m}_k &= \frac{m_k}{1-\beta_1^k}, & \hat{v}_k &= \frac{v_k}{1-\beta_2^k}, \qquad \theta_k = \theta_{k-1} - \eta\, \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon}. \end{aligned} \tag{1.6}$$

Tham số có gradient thường lớn được chia cho $\sqrt{\hat{v}_k}$ lớn nên bước đi nhỏ lại, và ngược lại; nhờ vậy Adam ít nhạy với việc chọn η hơn SGD. Autoencoder của đề tài dùng các giá trị mặc định $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, $\epsilon = 10^{-8}$; tốc độ học, cỡ lô và số bước nằm ở Mục 2.5.3.

Để huấn luyện nhanh hơn trên GPU, Autoencoder được tính ở độ chính xác hỗn hợp [23]: phần lớn phép tính chạy ở số thực 16 bit, còn trọng số được giữ và cập nhật ở 32 bit. Số thực 16 bit biểu diễn được ít giá trị nhỏ hơn, nên gradient rất nhỏ có thể bị làm tròn về 0; mất mát được nhân với một hệ số lớn trước khi lan truyền ngược và gradient được chia lại cho hệ số đó trước khi cập nhật.

1.2.7. Quá khớp và cách hạn chế

Quá khớp (overfitting) là khi mô hình học thuộc cả nhiễu và chi tiết riêng của dữ liệu huấn luyện: mất mát trên tập huấn luyện rất thấp nhưng kết quả trên dữ liệu mới kém [6]. Nguy cơ này cao khi mô hình có nhiều tham số mà dữ liệu ít, đúng tình huống của đề tài: mỗi loại sản phẩm chỉ có 137 đến 769 ảnh tốt để xây dựng mô hình.

Có bốn cách hạn chế thường dùng. Chính quy hóa L2 (weight decay) cộng vào mất mát một khoản tỷ lệ với bình phương độ lớn trọng số, để trọng số không quá lớn. Dropout [24] tắt ngẫu nhiên một phần nơ-ron khi huấn luyện để mạng không phụ thuộc vào vài nơ-ron. Dừng sớm (early stopping) ngừng huấn luyện khi kết quả trên tập kiểm định thôi cải thiện. Cách thứ tư là giảm số tham số cần học, bằng mạng nhỏ hơn hoặc bằng cách dùng lại đặc trưng học trước; PaDiM và PatchCore không học tham số mạng nào trên dữ liệu của đề tài, nên gần như không có nguy cơ quá khớp theo nghĩa này.

Với Autoencoder, quá khớp còn có một dạng riêng: mạng quá mạnh học được cách dựng

lại mọi thứ, kể cả lỗi, nên mất khả năng phát hiện. Cổ chai hẹp (Mục 2.5) là cách đề tài hạn chế điều này. Dừng sớm cũng khó áp dụng, vì mất mát thấp trên ảnh tốt kiểm định không có nghĩa là phát hiện lỗi tốt hơn; đề tài cố định 1500 bước cho mọi loại.

1.2.8. Autoencoder và các biến thể

Autoencoder là mạng được huấn luyện để đầu ra bằng chính đầu vào, gồm bộ mã hóa f biến ảnh x thành mã z và bộ giải mã g dựng lại $\hat{x} = g(f(x))$. Nếu không có ràng buộc nào, mạng có thể học hàm đồng nhất và không rút ra được gì về cấu trúc của dữ liệu; vì vậy luôn cần một ràng buộc buộc mạng chỉ giữ lại những gì quan trọng nhất [7]. Ràng buộc phổ biến nhất là cổ chai: z có số chiều nhỏ hơn nhiều so với x. Với ảnh $256 \times 256 \times 3$, tức 196 608 giá trị, và mã z 256 chiều như trong đề tài, mã nhỏ hơn đầu vào khoảng 770 lần.

Các biến thể khác dùng ràng buộc khác thay cho cổ chai. Autoencoder thưa phạt số thành phần khác 0 của z. Autoencoder khử nhiễu [25] nhận ảnh đã bị thêm nhiễu và học dựng lại ảnh sạch, nên phải học cấu trúc của dữ liệu để biết đâu là nhiễu; ý tưởng này gần với phát hiện bất thường, vì lỗi có thể coi là một thứ cần loại khỏi ảnh. Autoencoder biến phân (VAE) [26] coi z là biến ngẫu nhiên và ép phân phối của nó gần phân phối chuẩn, nên ngoài sai số tái tạo còn cho một ước lượng xác suất của mẫu. Đề tài dùng dạng đơn giản nhất, Autoencoder có cổ chai, để làm mốc so sánh.

Dùng Autoencoder để phát hiện bất thường dựa trên giả định rằng mạng chỉ học từ ảnh tốt nên dựng lại tốt ảnh tốt và dựng sai vùng lỗi. Giả định này có thể sai theo hai hướng. Mạng có thể khái quát quá tốt và dựng lại được cả lỗi, nhất là lỗi nhỏ hoặc lỗi có kết cấu giống nền, khi đó sai số ở vùng lỗi không nổi lên. Ngược lại, mạng có thể dựng ảnh tốt chưa đủ sắc ở cạnh và kết cấu chi tiết, khiến ảnh tốt cũng có sai số cao. Chương 3 cho thấy cả hai trường hợp đều xảy ra: lưới bị cong của grid được dựng lại gần như lưới thẳng nhưng sai số vẫn nhỏ, còn thân con vít có ren bị sai số cao dọc theo cả thân.

1.3. Mạng nơ-ron tích chập

1.3.1. Phép tích chập

Với ảnh, lớp kết nối đầy đủ có hai nhược điểm: số tham số rất lớn (ảnh $256 \times 256 \times 3$ có gần 200 nghìn giá trị đầu vào) và mất thông tin điểm ảnh nào nằm cạnh điểm ảnh nào. Lớp tích chập khắc phục cả hai bằng một bộ lọc nhỏ K trượt trên ảnh [10]:

$$h(u, v) = \sum_{i} \sum_{j} \sum_{c} K(i, j, c)\, x(u + i,\, v + j,\, c) + b \tag{1.7}$$

Cùng một bộ lọc được dùng ở mọi vị trí (chia sẻ trọng số), nên số tham số chỉ phụ thuộc kích thước bộ lọc và số kênh, không phụ thuộc kích thước ảnh. Một lớp có C_{ra} bộ lọc kích thước $k \times k$ trên $C_{\text{vào}}$ kênh có $k^2 C_{\text{vào}} C_{\text{ra}} + C_{\text{ra}}$ tham số; lớp đầu của bộ mã hóa Autoencoder ($k = 4$, 3 kênh

vào, 32 kênh ra) chỉ có 1568 tham số. Đầu ra vẫn là một bản đồ có tọa độ không gian, nên mỗi vị trí của bản đồ đặc trưng ứng với một vùng (patch) của ảnh gốc. Tính chất này là cơ sở để cả ba phương pháp trong đề tài tạo được bản đồ bất thường theo vị trí.

Với đầu vào cạnh I, bộ lọc cạnh k, bước trượt (stride) S và đệm (padding) P điểm ảnh mỗi bên, cạnh của bản đồ ra là

$$O = \left\lfloor \frac{I - k + 2P}{S} \right\rfloor + 1. \tag{1.8}$$

Các lớp mã hóa của Autoencoder dùng $k = 4$, $S = 2$, $P = 1$, nên mỗi lớp giảm đúng một nửa: $256 \to 128 \to \cdots \to 8$ sau năm lớp. Bảng 1.3 áp dụng (1.8) và công thức số tham số ở trên cho từng lớp của bộ mã hóa, cùng trường tiếp nhận tính theo (1.10) ở mục sau. Số tham số tăng nhanh theo số kênh, nhưng cả năm lớp tích chập cộng lại, khoảng 1,74 triệu tham số, vẫn chưa bằng một nửa lớp kết nối đầy đủ ở cổ chai.

Bảng 1.3. Kích thước, số tham số và trường tiếp nhận các lớp tích chập của bộ mã hóa Autoencoder

Lớp	Kênh vào $\to$ ra	Bản đồ ra	Số tham số	Trường tiếp nhận
Tích chập 1	$3 \to 32$	128×128	1 568	4
Tích chập 2	$32 \to 64$	64×64	32 832	10
Tích chập 3	$64 \to 128$	32×32	131 200	22
Tích chập 4	$128 \to 256$	16×16	524 544	46
Tích chập 5	$256 \to 256$	8×8	1 048 832	94

1.3.2. Gộp, tích chập chuyển vị và chuẩn hóa theo lô

Phép gộp (pooling) thay mỗi vùng $k \times k$ bằng một giá trị đại diện, giá trị lớn nhất (max pooling) hoặc trung bình (average pooling). Gộp không có tham số; với bước trượt lớn hơn 1 nó giảm kích thước bản đồ, còn với bước trượt 1 và đệm phù hợp nó giữ nguyên kích thước và chỉ làm mỗi vị trí mang thông tin của lân cận. PatchCore dùng cách thứ hai: gộp trung bình 3×3 với bước trượt 1 để mỗi vector đặc trưng tổng hợp cả vùng xung quanh, bền hơn với dịch chuyển nhỏ (Mục 2.7).

Tích chập chuyển vị (transposed convolution) làm điều ngược lại với tích chập có bước trượt: phóng to bản đồ, với cạnh ra $O = (I - 1)S - 2P + k$. Với cùng $k = 4$, $S = 2$, $P = 1$, mỗi lớp giải mã của Autoencoder nhân đôi kích thước, $8 \to 16 \to \cdots \to 256$, đối xứng với bộ mã hóa.

Chuẩn hóa theo lô (batch normalization) [13] chuẩn hóa đầu ra x của mỗi kênh theo trung bình μ_B và phương sai σ_B^2 trên lô hiện tại, rồi cho mạng học lại thang đo γ và độ dời β:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y = \gamma\,\hat{x} + \beta. \tag{1.9}$$

Nhờ đầu vào của mỗi lớp giữ được thang đo ổn định trong suốt quá trình huấn luyện, mạng dùng được tốc độ học lớn hơn và hội tụ nhanh hơn. Khi suy luận, μ_B và σ_B^2 được thay bằng trung bình tích lũy trong lúc huấn luyện. Điều này quan trọng với PaDiM và PatchCore: Wide-ResNet-50-2 chạy ở chế độ suy luận với thống kê học từ ImageNet, nên đặc trưng của một ảnh không phụ thuộc vào các ảnh khác trong cùng lô.

1.3.3. Trường tiếp nhận và thứ bậc đặc trưng

Mỗi vị trí ở bản đồ đặc trưng chỉ "nhìn" một vùng giới hạn của ảnh gốc, gọi là trường tiếp nhận. Với lớp l có bộ lọc cạnh k_l và bước trượt s_l, cạnh trường tiếp nhận r_l và khoảng cách j_l giữa hai vị trí liền kề (tính bằng điểm ảnh gốc) tăng dần theo

$$r_l = r_{l-1} + (k_l - 1)\, j_{l-1}, \qquad j_l = j_{l-1}\, s_l, \qquad r_0 = j_0 = 1. \tag{1.10}$$

Như vậy trường tiếp nhận tăng nhanh khi mạng sâu hơn, nhất là sau các lớp có bước trượt lớn hơn 1. Các lớp nông có trường tiếp nhận nhỏ, phản hồi với cạnh, màu và kết cấu đơn giản; các lớp sâu có trường tiếp nhận rộng, biểu diễn cấu trúc phức tạp hơn nhưng có độ phân giải không gian thấp hơn [27]. PaDiM và PatchCore chọn các tầng trung gian vì cần cả hai: đủ chi tiết để thấy lỗi nhỏ và đủ ngữ cảnh để phân biệt lỗi với biến thiên bình thường. Với Autoencoder, năm lớp bước trượt 2 cho mỗi vị trí ở bản đồ 8×8 một trường tiếp nhận rộng 94 điểm ảnh theo (1.10), và lớp kết nối đầy đủ ở cổ chai nối mọi vị trí với nhau, nên mỗi điểm ảnh dựng lại phụ thuộc vào toàn bộ ảnh.

1.3.4. Các kiến trúc mạng tích chập tiêu biểu

LeNet [10] là một trong những mạng tích chập đầu tiên được dùng thực tế, để nhận dạng chữ số viết tay. AlexNet [12] thắng cuộc thi phân loại ImageNet năm 2012 với cách biệt lớn, nhờ huấn luyện mạng sâu hơn trên GPU, dùng ReLU và dropout; từ đó học sâu trở thành hướng chính của thị giác máy tính. VGG [28] cho thấy xếp chồng nhiều lớp 3×3 nhỏ (16 đến 19 lớp) tốt hơn ít lớp có bộ lọc lớn. Tuy vậy, chỉ thêm lớp thì đến một mức nào đó mạng sâu hơn lại cho sai số huấn luyện cao hơn, do gradient khó truyền qua quá nhiều lớp.

ResNet [15] giải quyết vấn đề này bằng kết nối tắt: mỗi khối học phần chênh lệch $F(x)$ rồi cộng lại với đầu vào,

$$y = F(x) + x. \tag{1.11}$$

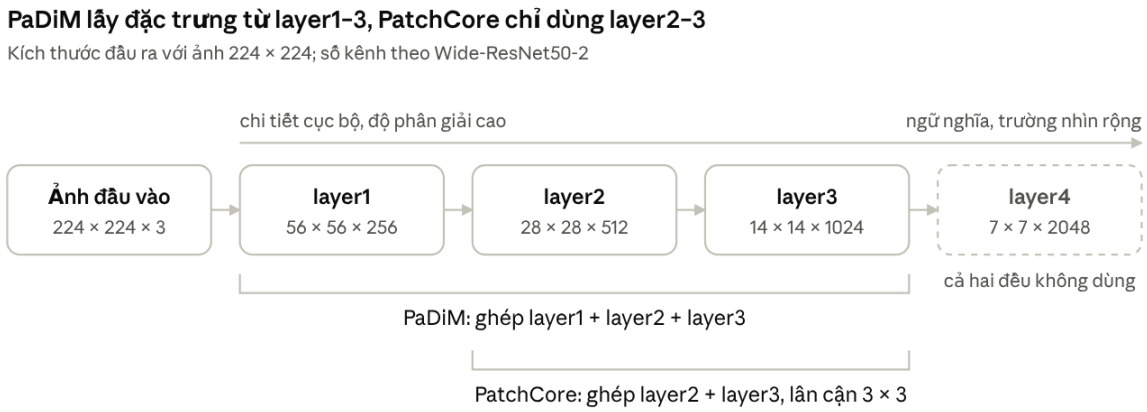
Đạo hàm của y theo x luôn có thành phần 1 từ nhánh tắt, nên gradient đi qua được hàng chục khối mà không bị triệt tiêu, và một khối không cần thiết chỉ việc học $F(x) \approx 0$. Các bản ResNet từ 50 lớp trở lên dùng khối cổ chai: tích chập 1×1 giảm số kênh, tích chập 3×3 trên số kênh nhỏ đó, rồi tích chập 1×1 tăng số kênh lại, để giảm chi phí tính toán. Tích chập 1×1 không nhìn lân cận mà chỉ trộn các kênh tại từng vị trí, nên rất rẻ. Với đầu vào 256 kênh, hai lớp 3×3 giữ nguyên 256 kênh cần khoảng $2 \cdot 9 \cdot 256^2 \approx 1{,}18$ triệu trọng số, trong khi khối cổ chai

$256 \to 64 \to 64 \to 256$ chỉ cần $256 \cdot 64 + 9 \cdot 64^2 + 64 \cdot 256 \approx 70$ nghìn. Wide-ResNet-50-2 dùng 128 thay vì 64 kênh bên trong khối này, nên mỗi khối cần khoảng 213 nghìn trọng số, gấp ba khối của ResNet-50; đây là lý do mạng có 68,9 triệu tham số so với khoảng 25,6 triệu của ResNet-50, đổi lại đặc trưng phong phú hơn ở mỗi tầng.

Các khối được nhóm thành bốn tầng `layer1`–`layer4`; mỗi tầng sau giảm một nửa kích thước không gian và tăng gấp đôi số kênh (Hình 1.5). Wide ResNet [16] giữ cấu trúc đó nhưng tăng gấp đôi số kênh bên trong mỗi khối cổ chai; số kênh ở đầu ra của mỗi tầng không đổi. Đề tài dùng Wide-ResNet-50-2 (khoảng 68,9 triệu tham số) cho cả PaDiM và PatchCore. Bảng 1.4 cho kích thước đặc trưng của từng tầng với ảnh vào 256×256. Cả hai phương pháp đều bỏ `layer4`: tầng này có độ phân giải quá thấp để định vị lỗi và mang đặc trưng thiên về nhiệm vụ phân loại ImageNet.

Bảng 1.4. Các tầng của Wide-ResNet-50-2 với ảnh vào 256×256

Tầng	Số khối	Kênh trong khối	Kênh ra	Bản đồ	Dùng trong
Tích chập đầu và gộp	–	–	64	64×64	–
`layer1`	3	128	256	64×64	PaDiM
`layer2`	4	256	512	32×32	PaDiM, PatchCore
`layer3`	6	512	1024	16×16	PaDiM, PatchCore
`layer4`	3	1024	2048	8×8	Không dùng



Hình 1.5. Các tầng của Wide-ResNet-50-2 và tầng đặc trưng mà PaDiM, PatchCore sử dụng (kích thước minh họa với ảnh 224×224; với ảnh 256×256 của đề tài là 64, 32, 16 và 8)

1.4. Học chuyển giao: dùng lại đặc trưng của mạng học trên ImageNet

1.4.1. ImageNet và mạng học trước

ImageNet là bộ dữ liệu khoảng 1,2 triệu ảnh thuộc 1000 lớp đối tượng [11]. Một mạng tích chập huấn luyện để phân loại ImageNet học được các bộ lọc cho cạnh, kết cấu và hình dạng; các bộ

lọc ở tầng nông và tầng giữa dùng lại được cho dữ liệu khác với hiệu quả tốt [27]. Học chuyển giao là việc lấy mạng đã học trên một bài toán có nhiều dữ liệu như vậy làm điểm xuất phát cho một bài toán khác có ít dữ liệu hơn.

1.4.2. Tinh chỉnh và trích đặc trưng cố định

Có hai cách dùng lại một mạng học trước. Tinh chỉnh (fine-tuning) tiếp tục huấn luyện một phần hoặc toàn bộ trọng số trên dữ liệu mới, thường với tốc độ học nhỏ; cách này cần một hàm mất mát có ý nghĩa cho bài toán mới, thường là nhãn phân loại. Trích đặc trưng cố định giữ nguyên mọi trọng số và chỉ dùng đầu ra của một số tầng làm biểu diễn của ảnh, rồi xây dựng mô hình đơn giản trên biểu diễn đó.

PaDiM và PatchCore dùng cách thứ hai: không cập nhật trọng số nào trên dữ liệu của đề tài. Lý do là bài toán không có nhãn lỗi để tinh chỉnh, còn tinh chỉnh chỉ trên ảnh tốt không có mục tiêu rõ ràng và dễ làm đặc trưng sụp về một điểm. Phần "học" trên ảnh tốt chỉ là ghi nhận đặc trưng của chúng, dưới dạng thống kê (PaDiM) hoặc tập vector tham chiếu (PatchCore). Nhờ đó hai phương pháp không cần lan truyền ngược, xây dựng mô hình trong một lượt duyệt ảnh và không cần nhiều ảnh tốt.

1.4.3. Khoảng cách miền

Ảnh ImageNet là ảnh chụp tự nhiên của động vật, đồ vật, phong cảnh, khác xa ảnh cận cảnh một tấm kim loại hay một bảng mạch dưới ánh sáng công nghiệp. Sự khác biệt giữa dữ liệu nguồn và dữ liệu đích gọi là khoảng cách miền (domain gap). Thực nghiệm của Yosinski và cộng sự [27] cho thấy khả năng chuyển giao giảm khi tầng càng sâu và khi hai bài toán càng khác nhau: tầng nông mang đặc trưng chung như cạnh, màu, kết cấu, còn tầng sâu chuyên biệt cho các lớp của ImageNet. Đây là thêm một lý do để PaDiM và PatchCore dùng `layer1` đến `layer3` và bỏ `layer4`.

Để đặc trưng có ý nghĩa, ảnh đầu vào phải có cùng phân phối với ảnh lúc mạng được huấn luyện; vì vậy mỗi kênh màu c của ảnh, đã đưa về $[0, 1]$, được chuẩn hóa theo trung bình m_c và độ lệch chuẩn d_c tính trên toàn bộ ImageNet,

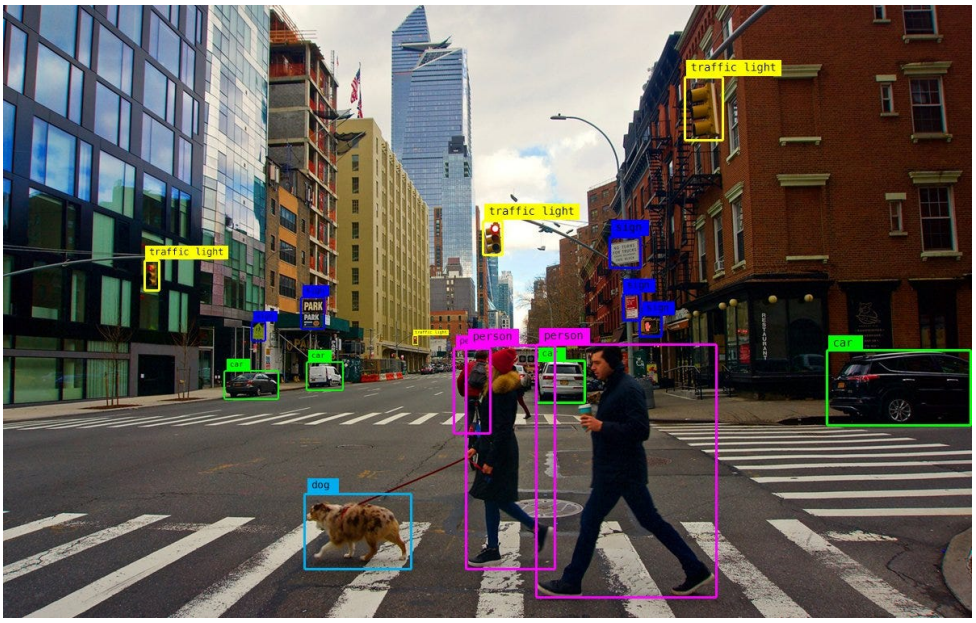
$$x'_c = \frac{x_c - m_c}{d_c}, \qquad m = (0{,}485;\ 0{,}456;\ 0{,}406), \qquad d = (0{,}229;\ 0{,}224;\ 0{,}225), \tag{1.12}$$

trước khi đưa vào Wide-ResNet-50-2 (Mục 2.3.3). Nếu bỏ bước này, đầu vào lệch khỏi phân phối mà các lớp chuẩn hóa theo lô của mạng đã ghi nhận, và đặc trưng của mọi tầng đều bị dịch đi. Chất lượng của hai phương pháp vẫn phụ thuộc vào việc đặc trưng ImageNet có phân biệt được lỗi công nghiệp hay không; Chương 3 cho thấy điều này đúng với phần lớn các loại, nhưng đặc trưng cũng thay đổi theo ánh sáng, nên ảnh chụp ở điều kiện khác lúc huấn luyện có thể bị coi là bất thường dù không có lỗi.

1.5. Thị giác máy tính và xử lý ảnh số

1.5.1. Các nhiệm vụ cơ bản

Thị giác máy tính là lĩnh vực của AI giúp máy tính hiểu nội dung ảnh và video. Ba nhiệm vụ cơ bản xếp theo mức chi tiết tăng dần: phân loại ảnh (ảnh thuộc lớp nào), phát hiện đối tượng (đối tượng nằm ở hộp nào trong ảnh, Hình 1.6) và phân đoạn (từng điểm ảnh thuộc đối tượng nào). Bài toán của đề tài ghép hai mức: phát hiện ảnh lỗi tương ứng phân loại ảnh, còn định vị vùng lỗi tương ứng phân đoạn ở mức điểm ảnh. Từ khi mạng tích chập ra đời, cả ba nhiệm vụ đều do học sâu đảm nhận là chính.



Hình 1.6. Phát hiện đối tượng, một nhiệm vụ của thị giác máy tính

1.5.2. Ảnh số và thay đổi kích thước ảnh

Một ảnh màu được lưu dưới dạng mảng $H \times W \times 3$, mỗi phần tử là cường độ của một kênh đỏ, lục hoặc lam tại một điểm ảnh, thường là số nguyên 8 bit từ 0 đến 255; ảnh xám chỉ có một kênh. Trước khi đưa vào mạng, giá trị được chia cho 255 để nằm trong $[0, 1]$. Grid và screw trong MVTec AD là ảnh xám, nên được nhân thành ba kênh giống nhau để dùng chung mạng với ảnh màu.

Ảnh gốc của ba bộ dữ liệu có cạnh từ khoảng 1000 đến hơn 4000 điểm ảnh, quá lớn để xử lý trực tiếp, nên được thu nhỏ. Phép thu nhỏ cần nội suy giá trị tại các vị trí mới. Nội suy lân cận gần nhất lấy giá trị của điểm ảnh gốc gần nhất; cách này không tạo giá trị mới, nên được dùng cho mặt nạ để mặt nạ vẫn chỉ có hai giá trị lỗi và không lỗi. Nội suy song tuyến tính lấy trung bình có trọng số của bốn điểm lân cận: với vị trí $(u+a, v+b)$, $0 \leq a, b < 1$, nằm giữa bốn điểm

nguyên,

$$I(u+a, v+b) = (1-a)(1-b)\, I(u,v) + a(1-b)\, I(u+1,v) \\ + (1-a)\, b\, I(u,v+1) + a\, b\, I(u+1,v+1). \tag{1.13}$$

Cách này cho kết quả mượt và phù hợp khi phóng to, như khi đưa bản đồ bất thường từ lưới đặc trưng 64×64 hay 32×32 về kích thước ảnh: mỗi vị trí đặc trưng trải ra một vùng 4 hoặc 8 điểm ảnh, và giá trị chuyển dần giữa các vùng thay vì thành từng khối vuông. Nội suy theo diện tích lấy trung bình mọi điểm ảnh gốc nằm trong vùng ứng với một điểm ảnh mới; khi thu nhỏ nhiều lần, cách này tránh hiện tượng răng cưa và chi tiết giả, nên được dùng cho ảnh đầu vào. Cái giá của việc thu nhỏ là mất chi tiết: ảnh sheet_metal 4224×1056 thu về 1024×256, tức mỗi chiều nhỏ đi khoảng 4 lần, nên một vết xước rộng 8 điểm ảnh chỉ còn khoảng 2 điểm ảnh.

1.5.3. Lọc Gauss

Lọc Gauss là phép tích chập ảnh với hàm Gauss hai chiều

$$G_\sigma(u,v) = \frac{1}{2\pi\sigma^2} \exp\!\left(-\frac{u^2+v^2}{2\sigma^2}\right), \tag{1.14}$$

tức thay mỗi điểm bằng trung bình có trọng số của lân cận, với trọng số giảm dần theo khoảng cách; σ càng lớn thì ảnh càng mượt. Trên thực tế bộ lọc được cắt ở bán kính khoảng 4σ. Cả ba phương pháp làm mượt bản đồ bất thường bằng lọc Gauss với $\sigma = 4$ (bộ lọc 33×33) trước khi lấy giá trị lớn nhất làm điểm của ảnh. Mục đích là để điểm ảnh phản ánh một vùng sai lệch có kích thước đáng kể, không bị quyết định bởi một điểm lẻ do nhiễu; đổi lại, một lỗi chỉ rộng vài điểm ảnh cũng bị làm yếu đi.

1.5.4. Ứng dụng trong kiểm tra chất lượng

Hai nhóm ứng dụng gần với đề tài nhất là kiểm tra chất lượng trong sản xuất và phân tích ảnh y học. Một hệ thống kiểm tra quang học tự động gồm camera, nguồn sáng và bộ xử lý: camera chụp từng sản phẩm trên dây chuyền ở vị trí và ánh sáng cố định, mô hình quyết định đạt hay không đạt và chỉ ra vị trí lỗi để truy nguyên nhân. Ánh sáng ổn định là giả định ngầm của hầu hết các phương pháp; khi nguồn sáng thay đổi, ảnh sản phẩm tốt cũng khác ảnh lúc huấn luyện. Thay đổi ánh sáng là một trong các tình huống mà MVTec AD 2 được xây dựng để đánh giá (Mục 2.3). Khác với nhiều ứng dụng khác, ở đây ảnh lỗi hiếm và kiểu lỗi khó lường trước, nên hướng chỉ học từ ảnh tốt được dùng rộng rãi. Trong y học, học sâu được dùng để phát hiện tổn thương trên ảnh X-quang, CT, MRI. Ảnh bệnh lý hiếm hơn ảnh bình thường và việc khoanh vùng tổn thương cần chuyên gia, nên lĩnh vực này gặp đúng khó khăn về dữ liệu như kiểm tra chất lượng công nghiệp.

1.6. Bài toán phát hiện bất thường không giám sát

1.6.1. Kiểm tra lỗi sản phẩm công nghiệp và các khó khăn

Trên dây chuyền sản xuất, mỗi sản phẩm cần được kiểm tra để loại bỏ những chiếc bị xước, nứt, nhiễm bẩn, biến dạng hay thiếu chi tiết. Kiểm tra bằng mắt người chậm hơn tốc độ dây chuyền, phụ thuộc kinh nghiệm và độ tập trung của người kiểm tra, và dễ bỏ sót lỗi nhỏ. Tự động hóa khâu này bằng thị giác máy tính gặp bốn khó khăn đặc thù.

Thứ nhất, ảnh lỗi hiếm: dây chuyền ổn định có tỷ lệ lỗi thấp, nên thu đủ ảnh lỗi để huấn luyện tốn nhiều thời gian, trong khi ảnh sản phẩm tốt có sẵn với số lượng lớn. Thứ hai, lỗi đa dạng và khó lường trước: không thể liệt kê trước mọi kiểu lỗi, và một kiểu lỗi mới có thể xuất hiện bất cứ lúc nào khi nguyên liệu hay máy móc thay đổi. Thứ ba, gán nhãn tốn kém: khoanh vùng lỗi chính xác đến từng điểm ảnh cần người có chuyên môn và nhiều công sức. Thứ tư, lỗi nhỏ nằm trên nền biến thiên: nhiều lỗi chỉ chiếm một phần rất nhỏ diện tích ảnh, trong khi ảnh tốt vẫn khác nhau tự nhiên về ánh sáng, tư thế và kết cấu bề mặt.

Ngay cả khi mô hình đạt AUROC cao trên bộ dữ liệu chuẩn, việc đưa vào dây chuyền còn gặp bốn thách thức khác. Thứ nhất là thay đổi điều kiện: ánh sáng, độ nét của camera, vị trí đặt sản phẩm hay nguyên liệu đầu vào thay đổi dần theo thời gian, khiến ảnh tốt mới khác ảnh dùng để xây dựng mô hình. Một mô hình phát hiện bất thường, đúng với bản chất của nó, sẽ coi mọi thay đổi như vậy là bất thường. Thứ hai là ngưỡng: ngưỡng đặt lúc xây dựng mô hình chỉ đúng khi phân phối điểm của ảnh tốt giữ nguyên, nên cần được theo dõi và đặt lại; một hệ thống báo nhầm quá nhiều sẽ bị người vận hành bỏ qua. Thứ ba là tốc độ và tài nguyên: mỗi sản phẩm thường chỉ có vài chục đến vài trăm mili giây để kiểm tra, và mô hình phải chạy trên phần cứng đặt cạnh dây chuyền. Thứ tư là khả năng giải thích: người vận hành cần biết vì sao một sản phẩm bị loại, nên bản đồ bất thường chỉ đúng vùng lỗi có giá trị không kém một điểm số chính xác.

Bốn thách thức này định hướng cách đề tài đánh giá ba phương pháp. Ngoài AUROC, đề tài đo tỷ lệ báo nhầm và bắt lỗi tại một ngưỡng chốt trước, thời gian suy luận và dung lượng mô hình, xem bản đồ bất thường trên từng ảnh, và thử ba phương pháp khi dữ liệu và điều kiện chụp thay đổi (Chương 3).

1.6.2. Phát hiện và định vị bất thường

Bài toán gồm hai nhiệm vụ trả lời hai câu hỏi khác nhau. Phát hiện trả lời "sản phẩm này có lỗi không": đầu ra là một số cho cả ảnh, điểm bất thường $s(x)$, và được đánh giá trên từng ảnh. Định vị trả lời "lỗi nằm ở đâu": đầu ra là bản đồ bất thường $A_x(u, v)$ cùng kích thước ảnh, mỗi điểm ảnh một giá trị, và được đánh giá trên từng điểm ảnh so với mặt nạ vùng lỗi. Điểm ảnh thường được suy ra từ bản đồ, ví dụ $s(x) = \max_{u,v} A_x(u, v)$. Điểm càng cao thì càng khác thường; điểm không phải xác suất có lỗi và không có thang đo chung giữa các phương pháp.

Hai nhiệm vụ có thể cho kết quả khác nhau trên cùng một mô hình. Với một hạt dẻ bị nứt, mô hình có thể khoanh đúng vết nứt trên bản đồ (định vị tốt), nhưng nếu bản đồ của ảnh tốt cũng

có vài điểm sáng thì điểm lớn nhất của ảnh tốt và ảnh lỗi khó phân biệt (phát hiện kém). Ngược lại, một mô hình có thể nhận ra ảnh lỗi nhờ một vùng sai lệch lớn mà vẫn khoanh lệch vị trí. Vì vậy đề tài đánh giá riêng hai nhiệm vụ (Mục 3.1). Trong ứng dụng thực tế, phát hiện quyết định loại hay giữ sản phẩm, còn định vị giúp người vận hành kiểm tra lại và truy nguyên nhân lỗi.

1.6.3. Vì sao chỉ học từ ảnh tốt

Một bộ phân loại có giám sát cần các ví dụ lỗi đã gán nhãn và có thể khó khái quát sang kiểu lỗi chưa xuất hiện trong tập huấn luyện; với ảnh lỗi hiếm và lỗi mới liên tục xuất hiện, điều kiện này khó đáp ứng. Hướng không giám sát đảo ngược cách đặt vấn đề: thay vì học "lỗi trông như thế nào", mô hình học "bình thường trông như thế nào" từ tập ảnh tốt $\mathcal{X}_{\text{train}} = \{x_1, \ldots, x_N\}$, rồi coi mọi sai khác đáng kể là bất thường [1]. Cách này chỉ cần ảnh tốt, vốn dễ thu thập, và về nguyên tắc phát hiện được cả kiểu lỗi chưa từng xuất hiện. Đổi lại, mô hình không biết đâu là lỗi "thật": mọi khác thường, kể cả một vết bẩn vô hại trên nền, đều có thể bị báo. Ảnh lỗi và mặt nạ vùng lỗi chỉ dùng ở bước đánh giá.

1.6.4. Hai loại bất thường

Trên ảnh công nghiệp, bất thường thường chia làm hai loại. Bất thường cấu trúc là thay đổi cục bộ về bề mặt hay hình dạng, như vết xước, vết nứt, lỗ thủng, vết bẩn. Bất thường logic là trường hợp từng chi tiết đều trông bình thường nhưng tổng thể sai, như thiếu linh kiện, lắp sai vị trí hay lật ngược. Phân biệt hai loại giúp giải thích vì sao một phương pháp so sánh từng vùng nhỏ có thể bỏ sót lỗi dù khoanh vùng tốt các vết trên bề mặt.

1.6.5. Cơ sở thống kê của điểm bất thường

Mọi phương pháp phát hiện bất thường đều ngầm ước lượng mức độ "hiếm" của một mẫu so với dữ liệu bình thường. Ba phương pháp của đề tài ứng với ba cách ước lượng khác nhau.

Cách thứ nhất là giả định dạng phân phối. Nếu vector đặc trưng $z \in \mathbb{R}^d$ của mẫu bình thường tuân theo phân phối Gauss nhiều chiều $\mathcal{N}(\mu, \Sigma)$, mật độ xác suất là

$$p(z) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\Big(-\tfrac{1}{2}(z-\mu)^\top \Sigma^{-1}(z-\mu)\Big), \tag{1.15}$$

nên $-\log p(z) = \frac{1}{2} d_M(z)^2 +$ hằng số, với $d_M(z) = \sqrt{(z-\mu)^\top \Sigma^{-1}(z-\mu)}$ là khoảng cách Mahalanobis [29]. Mẫu càng xa trung bình theo khoảng cách này thì mật độ càng thấp, tức càng bất thường. Khi $\Sigma = I$, khoảng cách Mahalanobis trở thành khoảng cách Euclid; khi Σ bất kỳ, nó co giãn từng hướng theo độ biến thiên của dữ liệu, nên lệch theo một hướng vốn ít dao động bị coi là nghiêm trọng hơn. PaDiM dùng đúng cách này cho từng vị trí trên ảnh (Mục 2.6).

Một ví dụ hai chiều cho thấy sự khác biệt. Giả sử đặc trưng bình thường có trung bình $\mu = (0, 0)$, độ lệch chuẩn 1 theo chiều thứ nhất, 0,1 theo chiều thứ hai, và hai chiều độc lập, tức $\Sigma = \text{diag}(1;\ 0{,}01)$. Điểm $z_1 = (2, 0)$ và điểm $z_2 = (0;\ 0{,}2)$ có khoảng cách Euclid tới μ lần lượt

là 2 và 0,2, nên theo Euclid z_1 bất thường hơn gấp 10 lần. Theo Mahalanobis, cả hai đều cách μ đúng 2 độ lệch chuẩn theo chiều của mình, nên có cùng khoảng cách $d_M = 2$. Với đặc trưng nhiều chiều của ảnh, nơi có chiều dao động mạnh theo kết cấu bình thường và có chiều gần như cố định, khoảng cách Euclid sẽ bị các chiều dao động mạnh chi phối, còn Mahalanobis đưa mọi chiều về cùng thang đo.

Để dùng (1.15), phải ước lượng Σ từ N mẫu và nghịch đảo nó. Ma trận hiệp phương sai ước lượng có hạng không quá $N - 1$, nên khi $N \leq d$ nó suy biến và không nghịch đảo được; khi N chỉ lớn hơn d không nhiều, nó nghịch đảo được nhưng các trị riêng nhỏ bị ước lượng thấp, khiến Σ^{-1} khuếch đại nhiễu. Với PaDiM, mỗi vị trí có $d = 100$ chiều và chỉ $N = 137$ đến 769 ảnh tốt, nên một khoản εI nhỏ được cộng vào đường chéo để ổn định phép nghịch đảo.

Cách thứ hai không giả định dạng phân phối mà dùng trực tiếp các mẫu: mật độ quanh một điểm tỷ lệ nghịch với khoảng cách từ điểm đó tới các mẫu bình thường gần nhất. Khoảng cách tới láng giềng gần nhất vì vậy là một điểm bất thường phi tham số, mô tả được cả phân phối nhiều đỉnh, khi tại cùng một vị trí có nhiều kiểu bình thường khác nhau. Cái giá là phải lưu và tìm kiếm trên toàn bộ tập mẫu; PatchCore giảm chi phí này bằng cách chỉ giữ một tập con đại diện (Mục 2.7).

Cả hai cách trên đều làm việc trong không gian nhiều chiều, nơi trực giác hai, ba chiều không còn đúng. Khi số chiều tăng, thể tích tăng theo hàm mũ nên dữ liệu trở nên thưa, và khoảng cách giữa các điểm có xu hướng gần bằng nhau; hiện tượng này thường gọi là lời nguyền số chiều. Ước lượng hiệp phương sai cần số mẫu tăng theo bình phương số chiều, còn tìm láng giềng gần nhất chậm dần theo số chiều. Vì vậy cả PaDiM và PatchCore đều giảm chiều bằng một phép chọn hoặc chiếu ngẫu nhiên: PaDiM giữ 100 trong 1792 chiều, PatchCore chiếu đặc trưng xuống 128 chiều khi chọn coreset. Bổ đề Johnson–Lindenstrauss [30] cho cơ sở của cách làm này: với n điểm bất kỳ, một phép chiếu ngẫu nhiên xuống k chiều, với k chỉ cần tỷ lệ với $\log n/\epsilon^2$, giữ mọi khoảng cách giữa các cặp điểm trong khoảng sai số tương đối ϵ với xác suất cao. Số chiều cần thiết chỉ phụ thuộc vào số điểm và sai số chấp nhận được, không phụ thuộc vào số chiều ban đầu.

Cách thứ ba là học một biểu diễn nén. Autoencoder có cổ chai nhỏ hơn đầu vào chỉ dựng lại tốt những ảnh nằm gần tập dữ liệu nó đã học; sai số tái tạo là thước đo khoảng cách từ ảnh tới tập đó. Với bộ giải mã tuyến tính và hàm mất mát MSE, Autoencoder học được đúng không gian con của phân tích thành phần chính (PCA) [7]; mạng phi tuyến nhiều lớp mở rộng ý tưởng đó sang các tập dữ liệu cong, phức tạp hơn một không gian con phẳng.

1.6.6. Ngưỡng quyết định và đánh đổi giữa báo nhầm và bỏ sót

Điểm bất thường chỉ là một con số; để kết luận một sản phẩm có lỗi hay không, cần so nó với một ngưỡng t. Hạ t thì bắt được nhiều lỗi hơn nhưng báo nhầm nhiều sản phẩm tốt hơn; nâng t thì ngược lại. Không có ngưỡng tốt nhất cho mọi tình huống. Trên dây chuyền, bỏ sót một sản phẩm lỗi có thể đưa hàng lỗi đến tay khách hàng, còn báo nhầm chỉ tốn công kiểm tra lại, nên ngưỡng thường được chọn theo mức báo nhầm mà nhà máy chấp nhận được.

Khi chỉ có ảnh tốt, ngưỡng phải đặt từ phân phối điểm của ảnh tốt. Một cách đơn giản là $t = \bar{s} + k\sigma_s$, với $\bar{s}$, σ_s là trung bình và độ lệch chuẩn của điểm trên một tập ảnh tốt không dùng để xây dựng mô hình. Nếu điểm của ảnh tốt có phân phối chuẩn, $k = 3$ cho xác suất một ảnh tốt vượt ngưỡng khoảng 0,13%. Nếu không giả định gì về phân phối, bất đẳng thức Cantelli chỉ bảo đảm xác suất này không quá $1/(1 + k^2)$, tức 10% với $k = 3$. Điểm bất thường lấy từ giá trị lớn nhất của bản đồ thường lệch về phía giá trị lớn, nên tỷ lệ báo nhầm thực tế nằm đâu đó giữa hai mức này. Ngoài ra, $\bar{s}$ và σ_s ước lượng từ vài chục ảnh có sai số đáng kể, và ngưỡng chỉ đúng khi ảnh mới có cùng phân phối với ảnh dùng để đặt ngưỡng; nếu điều kiện chụp thay đổi, cả phân phối điểm dịch chuyển theo. Mục 3.1.4 trình bày cách đề tài áp dụng ngưỡng này.

1.6.7. Các bộ dữ liệu chuẩn

Một bộ dữ liệu chuẩn cho bài toán này cần ảnh tốt để huấn luyện, ảnh test gồm cả tốt và lỗi, và mặt nạ khoanh vùng lỗi đến từng điểm ảnh để đánh giá định vị. Trước năm 2019, các nghiên cứu thường dùng bộ dữ liệu phân loại như MNIST hay CIFAR-10, coi một lớp là bình thường và các lớp khác là bất thường. Cách này khác xa bài toán công nghiệp, nơi bất thường là một vết nhỏ trên một sản phẩm vẫn thuộc đúng loại.

MVTec AD [1] là bộ dữ liệu đầu tiên được xây dựng riêng cho bài toán này, gồm 5354 ảnh độ phân giải cao của 15 loại, chia thành 5 loại bề mặt (carpet, grid, leather, tile, wood) và 10 loại vật thể, với 73 kiểu lỗi và mặt nạ cho mọi ảnh lỗi. Bộ này nhanh chóng trở thành thước đo chung, đến mức nhiều phương pháp gần đây đạt image-AUROC trung bình trên 0,99, và khoảng cách giữa các phương pháp không còn rõ. VisA [5] có 10 821 ảnh của 12 loại, trong đó 1200 ảnh lỗi, với các vật thể phức tạp hơn như bảng mạch nhiều linh kiện, nhiều vật thể trong một ảnh, hay vật thể không căn chỉnh. MVTec AD 2 [4] gồm 8 loại với các tình huống khó hơn: vật thể trong suốt hay phản quang, lỗi rất nhỏ trên ảnh lớn, và ảnh test chụp ở nhiều điều kiện ánh sáng khác lúc huấn luyện. Real-IAD [31] chụp mỗi sản phẩm từ 5 góc, với 30 loại vật thể, gần với điều kiện sản xuất thật hơn nữa.

Ba bộ dữ liệu đầu được dùng trong đề tài. MVTec AD cho phép so sánh trực tiếp với kết quả đã công bố; VisA và MVTec AD 2 thử ba phương pháp trong những điều kiện mà MVTec AD không có. Chương 2 trình bày chín loại sản phẩm được chọn và lý do chọn từng loại.

1.6.8. Các hướng tiếp cận và vị trí của ba phương pháp

Các phương pháp phát hiện bất thường không giám sát trên ảnh khác nhau ở chỗ chúng mô tả "bình thường" bằng gì và đo "khác thường" ra sao (Bảng 1.5). Đề tài chọn ba phương pháp thuộc ba hướng đầu: Autoencoder thuộc hướng tái tạo, so sánh trực tiếp trên điểm ảnh; PaDiM thuộc hướng mô hình hóa phân phối đặc trưng; PatchCore thuộc hướng bộ nhớ đặc trưng. Hai phương pháp sau cùng dựa trên đặc trưng học trước, nhưng PaDiM gắn thống kê với từng vị trí còn PatchCore so sánh không phụ thuộc vị trí. Hai hướng còn lại được nêu để thấy vị trí của ba phương pháp, không nằm trong phạm vi thực nghiệm.

Bảng 1.5. Các hướng tiếp cận phát hiện bất thường không giám sát trên ảnh

Hướng	Mô tả bình thường bằng	Điểm bất thường	Ví dụ
Tái tạo	Mạng học dựng lại ảnh tốt	Sai khác giữa ảnh và ảnh dựng lại	Autoencoder
Phân phối đặc trưng	Phân phối xác suất của đặc trưng học trước	Độ lệch khỏi phân phối	PaDiM
Bộ nhớ đặc trưng	Tập vector đặc trưng của ảnh tốt	Khoảng cách tới mẫu gần nhất	PatchCore
Chưng cất tri thức	Mạng học trò bắt chước mạng thầy trên ảnh tốt	Sai khác giữa đặc trưng thầy và trò	STFPM
Tự giám sát với lỗi giả	Bộ phân loại học trên lỗi tổng hợp	Đầu ra của bộ phân loại	CutPaste

1.6.9. Một số phương pháp tiêu biểu theo từng hướng

Ở hướng tái tạo, bên cạnh Autoencoder dùng MSE như trong đề tài, Bergmann và cộng sự đề xuất dùng hàm mất mát SSIM [19]. DRAEM [32] kết hợp tái tạo với lỗi giả: mạng tái tạo học bỏ đi các lỗi tổng hợp được dán lên ảnh tốt, rồi một mạng thứ hai học khoanh vùng khác biệt giữa ảnh vào và ảnh dựng lại, nhờ đó không còn phụ thuộc vào việc sai số từng điểm ảnh có đủ lớn hay không.

Ở hướng phân phối đặc trưng, PaDiM [2] là đại diện đơn giản nhất, với một phân phối Gauss cho mỗi vị trí. Các phương pháp dùng dòng chuẩn hóa (normalizing flow) như CFLOW-AD [33] học một phép biến đổi khả nghịch đưa đặc trưng bình thường về phân phối chuẩn, nên mô tả được phân phối phức tạp hơn một phân phối Gauss mà vẫn tính được mật độ chính xác.

Ở hướng bộ nhớ đặc trưng, SPADE [34] tìm các ảnh tốt gần nhất với ảnh kiểm thử, rồi so từng vị trí với mọi patch của các ảnh đó. Cách này cần lưu toàn bộ đặc trưng và tìm kiếm chậm. PatchCore [3] cải tiến trực tiếp từ SPADE bằng hai điểm: đặc trưng patch được gộp với lân cận để mang ngữ cảnh, và bộ nhớ được rút gọn bằng coreset.

Ở hướng chưng cất tri thức, mạng "thầy" là một mạng học trước, mạng "trò" được huấn luyện để cho đặc trưng giống thầy trên ảnh tốt; ở vùng lỗi, trò chưa từng học nên đặc trưng của hai mạng khác nhau. Phương pháp của Bergmann và cộng sự [35] và STFPM [36] thuộc hướng này. Ở hướng tự giám sát, CutPaste [37] cắt một mảnh của ảnh tốt rồi dán vào vị trí khác để tạo lỗi giả, huấn luyện một bộ phân loại phân biệt ảnh gốc với ảnh đã cắt dán, rồi dùng đặc trưng của bộ phân loại đó để phát hiện lỗi thật. Hai hướng sau đều phải huấn luyện mạng trên dữ liệu của từng loại sản phẩm, nên không được chọn trong phạm vi đề tài.

Tiểu kết chương

Chương này đã trình bày các nền tảng mà ba phương pháp của đề tài dựa vào. Autoencoder là một mạng nơ-ron tích chập được huấn luyện từ đầu bằng lan truyền ngược và Adam, với hàm mất mát MSE; cổ chai buộc mạng chỉ giữ cấu trúc chung của ảnh tốt, và chính tính chất của MSE

giải thích vì sao ảnh dựng lại mờ ở cạnh và kết cấu. PaDiM và PatchCore không huấn luyện mạng nào mà dùng lại đặc trưng của Wide-ResNet-50-2 học trên ImageNet, lấy ở các tầng giữa để cân bằng giữa chi tiết và ngữ cảnh. PaDiM mô tả đặc trưng bình thường tại mỗi vị trí bằng một phân phối Gauss và tính điểm bằng khoảng cách Mahalanobis; PatchCore lưu một tập con đại diện của các patch bình thường và tính điểm bằng khoảng cách tới láng giềng gần nhất. Cả ba cho một bản đồ bất thường, được làm mượt bằng lọc Gauss, và lấy giá trị lớn nhất làm điểm của ảnh; kết luận tốt hay lỗi cần thêm một ngưỡng đặt trên ảnh tốt.

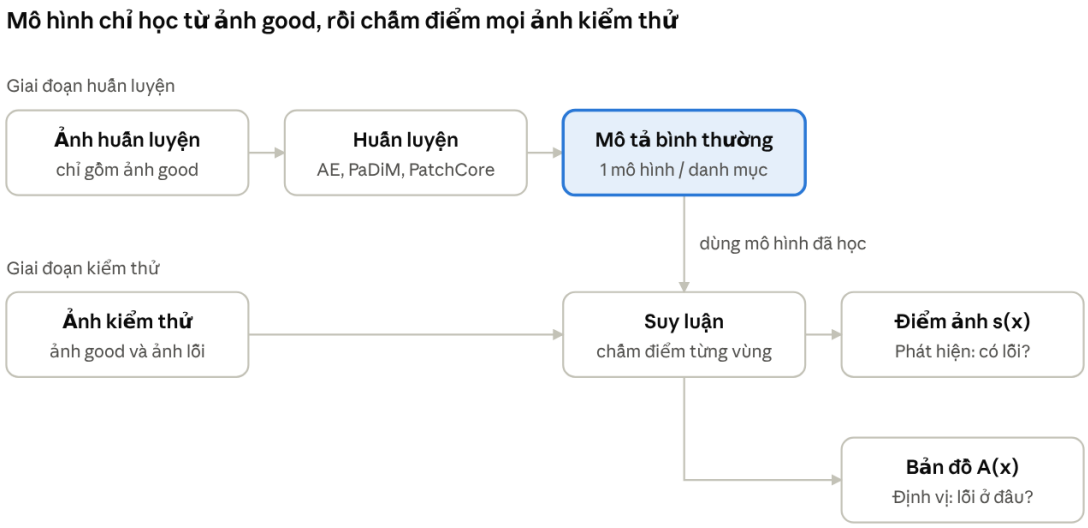
Chương cũng chỉ ra những giới hạn có thể đoán trước từ cơ chế: Autoencoder nhạy với cường độ điểm ảnh và dễ bỏ sót lỗi nhỏ, PaDiM phụ thuộc vào việc ảnh đã được căn chỉnh, PatchCore không dùng thông tin vị trí, còn cả hai phương pháp dùng đặc trưng học trước đều chịu khoảng cách miền giữa ImageNet và ảnh công nghiệp. Chương 2 trình bày chi tiết cách ba phương pháp được cài đặt, còn Chương 3 kiểm tra các dự đoán này bằng thực nghiệm.

CHƯƠNG 2. PHƯƠNG PHÁP PHÁT HIỆN BẤT THƯỜNG

Chương này trình bày bài toán cụ thể của đề tài, dữ liệu và cách tiền xử lý, lý do chọn ba phương pháp và cách triển khai từng phương pháp. Mọi cấu hình nêu ở đây là cấu hình đã chạy để ra kết quả ở Chương 3.

2.1. Phát biểu bài toán

Với mỗi loại sản phẩm, mô hình chỉ được học từ ảnh tốt của loại đó, sau đó phải cho mỗi ảnh kiểm thử hai đầu ra: điểm bất thường của ảnh $s(x)$ để trả lời "sản phẩm có lỗi không", và bản đồ bất thường $A_x(u, v)$ để trả lời "lỗi nằm ở đâu" (Hình 2.1; định nghĩa ở Mục 1.6). Mỗi loại có một mô hình riêng, vì "bình thường" của một lọ thủy tinh khác hẳn "bình thường" của một bảng mạch. Để có kết luận tốt/lỗi, mỗi mô hình còn có một ngưỡng t, đặt trên ảnh tốt không tham gia xây dựng mô hình; ảnh có $s(x) \geq t$ bị coi là lỗi.



Hình 2.1. Thiết lập bài toán: học từ ảnh tốt, mỗi ảnh kiểm thử cho điểm ảnh và bản đồ bất thường

2.2. Tổng quan giải pháp

Giải pháp của đề tài là một chuỗi xử lý chung cho cả ba phương pháp; chỉ bước xây dựng mô hình và suy luận là khác nhau. Đầu vào của mỗi loại sản phẩm gồm ảnh tốt để xây dựng mô hình, ảnh tốt tách riêng để đặt ngưỡng, ảnh test và mặt nạ để đánh giá (Mục 2.3). Ảnh được đổi về chiều cao 256 điểm ảnh, giữ gần đúng tỷ lệ gốc, chuẩn hóa theo yêu cầu của từng phương pháp, mặt nạ được đổi cùng kích thước (Mục 2.3.3).

Từ ảnh tốt, mỗi phương pháp xây dựng mô hình theo cách riêng: Autoencoder được huấn luyện để dựng lại ảnh; PaDiM trích đặc trưng Wide-ResNet-50-2 và ước lượng phân phối Gauss theo vị trí; PatchCore trích đặc trưng cùng mạng đó và chọn coreset (Mục 2.5 đến 2.7). Khi suy luận, mỗi ảnh cho sai số tái tạo, khoảng cách Mahalanobis hoặc khoảng cách tới patch gần nhất tại từng vị trí; giá trị này được đưa về bản đồ cùng kích thước ảnh, làm mượt, rồi lấy giá trị lớn nhất làm điểm của ảnh. Ngưỡng là trung bình cộng ba độ lệch chuẩn của điểm trên ảnh tốt tách riêng. Cuối cùng, mô hình được đánh giá bằng image-AUROC, pixel-AUROC, AUPRO trên tập test, cùng tỷ lệ báo nhầm và tỷ lệ bắt lỗi tại ngưỡng đã đặt (Mục 3.2).

Mỗi lựa chọn cấu hình trong chuỗi trên đều có lý do (Bảng 2.1). Nguyên tắc chung là giữ giá trị mặc định của bài báo gốc khi không có lý do để đổi, và không chỉnh bất kỳ giá trị nào theo kết quả trên tập test.

Bảng 2.1. Các lựa chọn cấu hình và lý do

Lựa chọn	Giá trị	Lý do
Kích thước ảnh	Cao 256, rộng theo tỷ lệ gốc; không cắt	Đủ chi tiết để thấy lỗi nhỏ mà chi phí tính toán vừa phải. Giữ tỷ lệ để không méo ảnh dài như sheet_metal (4224×1056), nơi lỗi chỉ chiếm khoảng 0,3% diện tích. Không cắt vùng giữa để giữ lỗi sát mép.
Số mô hình	Một mô hình cho mỗi loại	"Bình thường" của mỗi loại sản phẩm khác nhau; đây cũng là giao thức chuẩn của cả ba bộ dữ liệu.
Autoencoder	Tự xây dựng, học từ đầu	Đại diện cho hướng học từ ảnh mà không dùng đặc trưng học trước, làm mốc để thấy giá trị của đặc trưng học trước.
Backbone	Wide-ResNet-50-2 cho cả PaDiM và PatchCore	Cùng backbone thì chênh lệch giữa hai phương pháp chỉ do cơ chế. Đây cũng là backbone mạnh nhất trong bài báo gốc của cả hai.
Số chiều PaDiM	100 chiều chọn ngẫu nhiên từ 1792	Giữ ma trận hiệp phương sai ở cỡ 100×100; mô hình một loại 256×256 khoảng 160 MB thay vì vài GB.
Coreset	10% (MVTec AD); 1% (VisA, MVTec AD 2)	Mặc định 10% của bài báo. VisA có khoảng 770 ảnh huấn luyện mỗi loại, MVTec AD 2 có ảnh rất rộng, nên bộ nhớ đầy đủ lớn gấp 3 đến 4 lần; 1% vẫn nằm trong khoảng bài báo cho thấy gần như không mất độ chính xác.
Điểm ảnh	Giá trị lớn nhất của bản đồ	Một sản phẩm chỉ cần một vết lỗi nhỏ là đã lỗi, nên điểm của ảnh phải nhạy với vùng sai lệch mạnh nhất. Dùng chung một cách cho cả ba phương pháp.
Làm mượt bản đồ	Bộ lọc Gauss $\sigma = 4$	Giảm nhiễu từng điểm ảnh để giá trị lớn nhất không bị một điểm lẻ quyết định.
Ngưỡng	Trung bình + 3 độ lệch chuẩn trên ảnh tốt tách riêng	Cách đặt ngưỡng của bài báo MVTec AD 2 [4]. Không dùng nhãn test, nên tỷ lệ báo nhầm đo trên test là ước lượng trung thực cho ảnh mới.
Tăng cường dữ liệu	Không dùng	Xoay, lật có thể biến ảnh tốt thành ảnh giống lỗi (transistor lệch vị trí là một kiểu lỗi).

2.3. Dữ liệu thực nghiệm và cách tiền xử lý ảnh

2.3.1. Dữ liệu ảnh và các bước tiền xử lý

Dữ liệu quyết định chất lượng của mô hình học máy. Dữ liệu thường được chia làm ba loại: có cấu trúc (bảng với các cột rõ ràng, như cơ sở dữ liệu), phi cấu trúc (ảnh, âm thanh, văn bản) và bán cấu trúc (JSON, XML, nhật ký hệ thống). Ảnh sản phẩm của đề tài là dữ liệu phi cấu trúc: mỗi ảnh là một ma trận giá trị điểm ảnh, ý nghĩa nằm ở cách các điểm ảnh sắp xếp chứ không ở từng giá trị riêng lẻ. Đây là loại dữ liệu mà học sâu xử lý tốt hơn hẳn các phương pháp dùng đặc trưng thủ công.

Trước khi đưa vào mô hình, dữ liệu thường qua bốn bước. Làm sạch loại bỏ mẫu hỏng, thiếu hoặc sai; ba bộ dữ liệu đề tài dùng đều đã được làm sạch nên đề tài chỉ kiểm tra đủ số ảnh. Chuẩn hóa đưa dữ liệu về cùng thang đo, như đổi ảnh về cùng kích thước và đưa giá trị điểm ảnh về cùng phân phối; với mạng học trước, dữ liệu phải được chuẩn hóa giống lúc mạng đó được huấn luyện. Chia tập tách phần dữ liệu để xây dựng mô hình khỏi phần dùng để đánh giá; tập kiểm thử không được dùng khi xây dựng hay chọn mô hình, nếu không kết quả đánh giá sẽ lạc quan hơn thực tế. Tăng cường dữ liệu tạo thêm mẫu bằng cách xoay, lật, đổi độ sáng; kỹ thuật này chỉ hữu ích khi phép biến đổi không làm thay đổi bản chất mẫu, còn với bài toán tìm lỗi, nó có thể biến ảnh tốt thành ảnh trông giống lỗi, nên đề tài không dùng.

2.3.2. Ba bộ dữ liệu và chín loại sản phẩm

Đề tài dùng 9 loại sản phẩm từ ba bộ dữ liệu công khai (Bảng 2.2). Mỗi loại được chọn vì làm lộ một giả định của ba phương pháp hoặc phục vụ một thí nghiệm ở Mục 3.5, thay vì chạy toàn bộ một bộ dữ liệu.

MVTec AD [1] là bộ dữ liệu chuẩn cho phát hiện bất thường không giám sát trên ảnh công nghiệp, gồm 15 loại. Đề tài lấy ba loại: grid là bề mặt lưới lặp lại, ảnh xám; screw là con vít nhỏ nằm xoay tự do giữa các ảnh, phá giả định ảnh đã căn chỉnh; transistor là vật thể ở vị trí cố định, có kiểu lỗi logic như thiếu hay lệch linh kiện. Transistor thỏa giả định căn chỉnh nên được dùng làm loại đối chứng: khác biệt ở các thí nghiệm có thể quy cho điều kiện thay đổi chứ không phải cho bản thân loại sản phẩm. Đường tải chính thức không còn hoạt động khi thực hiện đề tài, nên nhóm dùng bản sao trên Hugging Face, đã đối chiếu số ảnh và kiểu lỗi với bài báo gốc.

MVTec AD 2 [4] là bộ dữ liệu mới hơn, cũng của MVTec, có ảnh độ phân giải cao hơn và tập test chụp cùng cảnh ở nhiều điều kiện ánh sáng: bình thường, quá sáng, quá tối và các mức dịch ánh sáng. Bộ này có sẵn thư mục `validation/good`, dùng để đặt ngưỡng. Đề tài lấy vial (lọ thủy tinh trong suốt) và sheet_metal (tấm kim loại dài, lỗi rất nhỏ), dùng tập `test_public` có mặt nạ. Đây là bộ duy nhất trong ba bộ có ảnh kiểm thử chụp ở điều kiện ánh sáng thật khác lúc huấn luyện, nên được dùng để thử ba phương pháp khi điều kiện chụp thay đổi (Mục 3.5.5).

VisA [5] gồm 12 loại vật thể có cấu trúc phức tạp. Đề tài lấy bốn loại bảng mạch in pcb1–pcb4, vì đây là nhiều mã sản phẩm gần giống nhau, giống tình huống một nhà máy sản

xuất nhiều loại bảng mạch, nên dùng được để kiểm tra mô hình của sản phẩm này có dùng được cho sản phẩm khác hay không (Mục 3.5.4). Dữ liệu dùng cách chia một lớp chính thức (`split_csv/1cls.csv`). Mặt nạ của VisA lưu giá trị 0/1 thay vì 0/255.

Bảng 2.2. Chín loại sản phẩm sử dụng (fit: ảnh tốt xây dựng mô hình; val: ảnh tốt đặt ngưỡng)

Bộ	Loại	Fit	Val	Test tốt	Test lỗi	Ảnh gốc	Đặc điểm
MVTec AD	grid	224	40	21	57	1024×1024	bề mặt lặp, ảnh xám, 5 kiểu lỗi
	screw	272	48	41	119	1024×1024	vật thể xoay tự do, lỗi nhỏ
	transistor	181	32	60	40	1024×1024	có lỗi logic (thiếu, lệch linh kiện)
MVTec AD 2	vial	291	41	35	105	1400×1900	trong suốt, 7 điều kiện chụp
	sheet_metal	137	19	24	90	4224×1056	ảnh dài, lỗi rất nhỏ, 6 điều kiện chụp
VisA	pcb1	768	136	100	100	1404×1070	bảng mạch nhiều linh kiện
	pcb2	766	135	100	100	1404×1070	bảng mạch nhiều linh kiện
	pcb3	769	136	101	100	1562×960	bảng mạch nhiều linh kiện
	pcb4	768	136	101	100	1358×1104	bảng mạch, nhiều kiểu lỗi kết hợp

Ba đặc điểm của dữ liệu được dùng để phân tích kết quả ở Chương 3. Screw xoay ngẫu nhiên giữa các ảnh, kể cả ảnh tốt, còn các loại khác có vị trí gần cố định. Tập test của MVTec AD 2 trộn nhiều điều kiện ánh sáng, trong khi ảnh huấn luyện chỉ chụp ở điều kiện bình thường; mỗi điều kiện chỉ có 4–5 ảnh tốt và 15 ảnh lỗi. VisA có số ảnh huấn luyện gấp 3 đến 4 lần các loại MVTec AD, và ảnh lỗi của pcb4 thường có nhiều kiểu lỗi cùng lúc.

2.3.3. Tiền xử lý ảnh và chia tập fit, validation, test

Để so sánh công bằng, ba phương pháp dùng chung cách xử lý dữ liệu và cách đánh giá; chỉ phần mô hình bên trong là khác nhau. Ảnh được thu nhỏ bằng nội suy theo diện tích (`INTER_AREA` của OpenCV) về 256×256 (MVTec AD), 256×320 (VisA), 256×192 (vial) và 256×1024 (sheet_metal), tính theo chiều cao $\times$ chiều rộng, và không bị cắt. Ảnh xám được nhân thành 3 kênh để dùng chung mạng với ảnh màu.

PaDiM và PatchCore chuẩn hóa giá trị điểm ảnh theo trung bình (0,485; 0,456; 0,406) và độ lệch chuẩn (0,229; 0,224; 0,225) của ImageNet, đúng phân phối mà backbone đã học. Autoencoder giữ giá trị trong $[0, 1]$ để so sánh trực tiếp với đầu ra sigmoid; ảnh không vuông được đổi về 256×256 trước khi vào mạng và bản đồ được phóng lại về kích thước ảnh. Mặt nạ được đổi kích thước bằng nội suy lân cận gần nhất rồi nhị phân hóa; ảnh test tốt được gán mặt

nạ toàn 0. Cả ba phương pháp đều không xoay hay lật ảnh.

MVTec AD và VisA không có tập validation, nên 15% ảnh huấn luyện tốt được tách ngẫu nhiên (seed 0) để đặt ngưỡng, 85% còn lại dùng xây dựng mô hình. MVTec AD 2 dùng thư mục `validation/good` có sẵn và xây dựng mô hình trên toàn bộ `train/good`. Ảnh test chỉ dùng để đánh giá lần cuối. Mọi chỉ số tính bằng cùng một module, riêng cho từng loại, rồi lấy trung bình theo loại (Mục 3.2).

2.4. Lựa chọn phương pháp và giả thuyết

Ba phương pháp được chọn vì ba lý do. Thứ nhất, chúng thuộc ba hướng khác nhau trong Bảng 1.5: tái tạo, mô hình hóa phân phối và bộ nhớ đặc trưng, nên so sánh chúng cho thấy ảnh hưởng của không gian so sánh (điểm ảnh hay đặc trưng) và cách mô tả "bình thường". Thứ hai, chúng tạo thành một chuỗi mà mỗi phương pháp khắc phục hạn chế chính của phương pháp trước (Hình 2.2): PaDiM chuyển từ so sánh điểm ảnh sang so sánh đặc trưng học trước; PatchCore bỏ ràng buộc theo vị trí và giả định phân phối Gauss của PaDiM. Thứ ba, cả ba đều đủ gọn để nhóm tự cài đặt trong phạm vi môn học, và đều có kết quả công bố để đối chiếu.



Hình 2.2. Ba phương pháp theo không gian biểu diễn, cách mô tả "bình thường" và cách tính điểm

Từ cơ chế của ba phương pháp, nhóm đặt ra ba giả thuyết và kiểm chứng ở Chương 3. Giả thuyết G1: PaDiM và PatchCore vượt Autoencoder ở hầu hết các loại, vì đặc trưng học trước phân biệt lỗi tốt hơn sai số từng điểm ảnh. Giả thuyết G2: khoảng cách giữa PatchCore và PaDiM lớn nhất ở screw, loại duy nhất xoay tự do, vì PaDiM gắn thống kê với vị trí còn PatchCore thì không; hai phương pháp dùng cùng backbone nên chênh lệch chỉ do cơ chế. Giả thuyết G3: Autoencoder làm tốt hơn ở các loại có nền đơn giản và lỗi lớn, kém ở các loại có lỗi nhỏ hoặc nhiều chi tiết khó tái tạo.

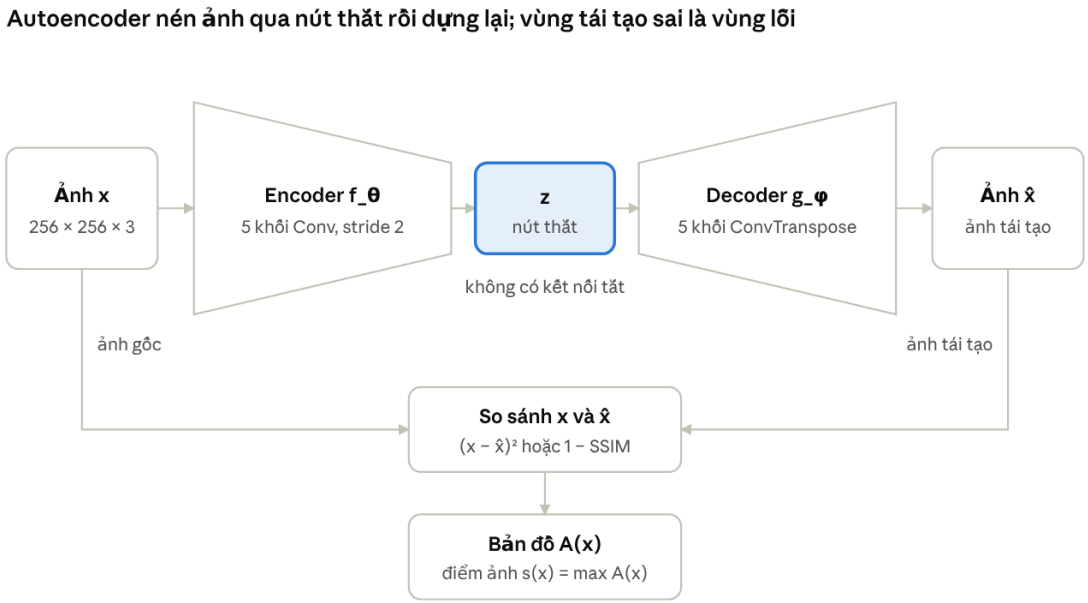
2.5. Autoencoder

2.5.1. Nguyên lý

Autoencoder gồm bộ mã hóa f nén ảnh x thành vector tiềm ẩn z và bộ giải mã g dựng lại ảnh từ z [7]:

$$z = f(x), \qquad \hat{x} = g(z) = g\big(f(x)\big). \tag{2.1}$$

Vector z nhỏ hơn nhiều so với ảnh, nên mạng không chép được nguyên ảnh mà phải học những cấu trúc chung của ảnh tốt. Khi gặp ảnh lỗi, mạng dựng lại theo kiểu ảnh tốt; vùng lỗi bị dựng sai nên có sai số tái tạo lớn (Hình 2.3).



Hình 2.3. Autoencoder và cách tính bản đồ bất thường (đề tài dùng sai số bình phương, không dùng SSIM)

2.5.2. Kiến trúc

Nhóm tự xây dựng một Autoencoder tích chập bằng PyTorch (Bảng 2.3). Bộ mã hóa gồm 5 lớp tích chập bước 2 thu ảnh từ 256×256 về 8×8, rồi một lớp kết nối đầy đủ nén $256 \times 8 \times 8 = 16384$ giá trị thành vector 256 chiều. Bộ giải mã đối xứng với bộ mã hóa. Mô hình có khoảng 11,9 triệu tham số (45 MB); hai lớp kết nối đầy đủ ở cổ chai chiếm khoảng 70%.

Hai lựa chọn thiết kế quyết định khả năng phát hiện lỗi. Thứ nhất, mạng không có kết nối tắt giữa bộ mã hóa và bộ giải mã như kiến trúc U-Net: kết nối tắt cho phép chép thẳng chi tiết từ đầu vào sang đầu ra, khi đó cả vùng lỗi cũng được dựng lại đúng và tín hiệu bất thường mất đi. Thứ hai, cổ chai là một vector chứ không phải bản đồ không gian. Cổ chai quá rộng thì mạng dựng lại được cả lỗi; quá hẹp thì ảnh tốt cũng bị dựng mờ và gây báo nhầm. Vector 256 chiều là

Bảng 2.3. Kiến trúc Autoencoder (mỗi lớp dùng bộ lọc 4×4 và chuẩn hóa theo lô; LeakyReLU ở bộ mã hóa, ReLU ở bộ giải mã)

Phần	Lớp	Kích thước đầu ra
Mã hóa	5 lớp tích chập bước 2, số kênh 32, 64, 128, 256, 256	$256 \times 8 \times 8$
	Kết nối đầy đủ $16384 \to 256$	256
Giải mã	Kết nối đầy đủ $256 \to 16384$	$256 \times 8 \times 8$
	4 lớp tích chập chuyển vị bước 2, số kênh 256, 128, 64, 32	$32 \times 128 \times 128$
	Tích chập chuyển vị bước 2 về 3 kênh, sigmoid	$3 \times 256 \times 256$

điểm cân bằng nhóm chọn và giữ cố định cho cả 9 loại.

2.5.3. Huấn luyện

Hàm mất mát là sai số bình phương trung bình trên mọi điểm ảnh và kênh màu:

$$L(x, \hat{x}) = \frac{1}{HWC} \sum_{u=1}^{H} \sum_{v=1}^{W} \sum_{c=1}^{C} \big[x(u, v, c) - \hat{x}(u, v, c)\big]^2. \tag{2.2}$$

Mỗi loại được huấn luyện cố định 1500 bước bằng Adam [14], tốc độ học $5 \cdot 10^{-4}$, mỗi bước một lô 16 ảnh lấy ngẫu nhiên có hoàn lại từ tập fit, tính toán ở độ chính xác hỗn hợp 16 bit. Số bước được cố định thay vì số epoch để mọi loại được huấn luyện cùng một lượng tính toán, dù số ảnh khác nhau từ 137 đến 769. Hàm mất mát dựa trên SSIM [19], so sánh cấu trúc cục bộ thay vì từng điểm ảnh, là một lựa chọn khác được báo cáo tốt hơn ở một số loại; đề tài chưa thử và ghi nhận ở phần Hạn chế.

2.5.4. Bản đồ và điểm bất thường

Bản đồ bất thường là sai số bình phương tại từng điểm ảnh, lấy trung bình theo kênh:

$$A(u, v) = \frac{1}{C} \sum_{c=1}^{C} \big[x(u, v, c) - \hat{x}(u, v, c)\big]^2. \tag{2.3}$$

Bản đồ được phóng về kích thước ảnh, làm mượt bằng bộ lọc Gauss với $\sigma = 4$, rồi lấy giá trị lớn nhất làm điểm của ảnh.

2.5.5. Ưu điểm và hạn chế

Autoencoder đơn giản, dễ giải thích (xem trực tiếp được ảnh dựng lại), mô hình nhỏ và không phụ thuộc mạng học trước. Hạn chế nằm ở chính không gian so sánh: ảnh dựng lại thường mờ ở cạnh và kết cấu chi tiết, nên sai số xuất hiện cả ở vùng bình thường; mạng đủ mạnh có thể dựng lại được cả lỗi; và sai khác từng điểm ảnh nhạy với ánh sáng hay dịch chuyển nhỏ nhưng kém nhạy với lỗi mang tính ngữ nghĩa. Ngoài ra, mạng phải huấn luyện từ đầu và kết quả phụ thuộc

nhiều vào siêu tham số.

2.6. PaDiM

2.6.1. Nguyên lý

PaDiM [2] chia ảnh thành lưới các vùng nhỏ (patch). Mỗi patch được mô tả bằng vector đặc trưng lấy từ mạng tích chập học trước. Phương pháp giả định rằng, trên ảnh tốt, các vector tại cùng một vị trí tuân theo một phân phối Gauss nhiều chiều; patch nào lệch xa phân phối của vị trí đó là bất thường (Hình 2.4). PaDiM không lan truyền ngược: phần "huấn luyện" chỉ là một lượt duyệt ảnh tốt để ước lượng thống kê.



Hình 2.4. Quy trình PaDiM (với ảnh 256×256 của đề tài, lưới là 64×64 và $d = 100$)

2.6.2. Trích xuất đặc trưng

Đặc trưng lấy từ ba tầng `layer1`, `layer2`, `layer3` của Wide-ResNet-50-2 [16] (Mục 1.3). Tầng nông giữ chi tiết cục bộ, tầng sâu hơn mang ngữ cảnh rộng; kết hợp cả ba để bắt được cả lỗi nhỏ lẫn lỗi lớn. Bản đồ của `layer2` và `layer3` được phóng về kích thước của `layer1` rồi ghép theo kênh, cho lưới bằng một phần tư kích thước ảnh (64×64 với ảnh 256×256), mỗi vị trí một vector $256 + 512 + 1024 = 1792$ chiều.

Kích thước ma trận hiệp phương sai tăng theo bình phương số chiều, nên PaDiM giữ lại $d = 100$ chiều chọn ngẫu nhiên (cố định theo seed 1234, dùng chung cho mọi loại, cho huấn luyện và kiểm thử). Bài báo gốc cho thấy chọn ngẫu nhiên cho kết quả tương đương phân tích thành phần chính nhưng nhanh hơn nhiều [2].

2.6.3. Ước lượng phân phối và tính điểm

Gọi $z_k(p)$ là vector đặc trưng của ảnh tốt thứ k tại vị trí p và N là số ảnh tốt:

$$\mu_p = \frac{1}{N}\sum_{k=1}^{N} z_k(p), \tag{2.4}$$

$$\Sigma_p = \frac{1}{N-1}\sum_{k=1}^{N}\big[z_k(p)-\mu_p\big]\big[z_k(p)-\mu_p\big]^\top + \varepsilon I. \tag{2.5}$$

Thành phần εI với $\varepsilon = 0{,}01$ bảo đảm Σ_p khả nghịch. Với ảnh kiểm thử, điểm tại p là khoảng cách Mahalanobis từ $z(p)$ tới phân phối bình thường:

$$d_p = \sqrt{\big[z(p)-\mu_p\big]^\top \Sigma_p^{-1}\big[z(p)-\mu_p\big]}. \tag{2.6}$$

Khác khoảng cách Euclid, khoảng cách này tính đến độ biến thiên và tương quan giữa các chiều: lệch trên một chiều vốn ổn định bị phạt nặng hơn lệch cùng độ lớn trên một chiều vốn dao động nhiều. Bản đồ các giá trị d_p được phóng về kích thước ảnh và làm mượt Gauss $\sigma = 4$; điểm ảnh là giá trị lớn nhất của bản đồ.

2.6.4. Triển khai và bộ nhớ

Nhóm tự cài đặt PaDiM bằng PyTorch, giữ nguyên trọng số Wide-ResNet-50-2 học trước trên ImageNet. Mô hình của mỗi loại gồm một vector trung bình và một ma trận nghịch đảo 100×100 cho mỗi vị trí, lưu ở dạng số thực 32 bit: 158 MB với lưới 64×64 (MVTec AD), 197 MB với 64×80 (VisA), 118 MB với vial và 631 MB với sheet_metal (64×256). Nếu giữ đủ 1792 chiều thì mỗi loại 256×256 cần khoảng 50 GB, nên bước giảm chiều là bắt buộc.

2.6.5. Ưu điểm và hạn chế

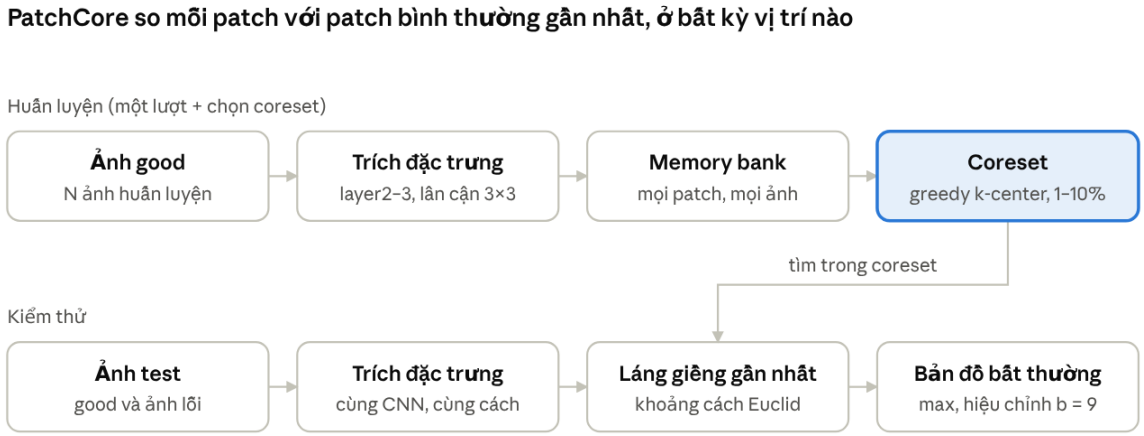
PaDiM không cần huấn luyện mạng, chỉ một lượt duyệt dữ liệu nên xây dựng mô hình nhanh, và tận dụng được đặc trưng học trước nên khoanh vùng lỗi tốt. Hạn chế chính là giả định ảnh đã căn chỉnh: khi vật thể xoay hay dịch chuyển, phân phối tại một vị trí trộn nhiều bộ phận khác nhau và kém phân biệt hơn. Ngoài ra, bộ nhớ tăng theo số vị trí và bình phương số chiều, và một phân phối Gauss duy nhất không mô tả đủ khi tại cùng vị trí có nhiều kiểu bình thường khác nhau.

2.7. PatchCore

2.7.1. Nguyên lý

PatchCore [3] coi "bình thường" là những gì đã xuất hiện trong ảnh tốt: một patch kiểm thử là bình thường nếu nó giống ít nhất một patch của ảnh tốt, ở bất kỳ vị trí nào trong bất kỳ ảnh nào

(Hình 2.5). So với PaDiM, phương pháp có hai khác biệt: không phụ thuộc vị trí, và không giả định dạng phân phối mà dùng trực tiếp các vector mẫu.



Hình 2.5. Quy trình PatchCore theo bài báo gốc (đề tài lấy giá trị lớn nhất của bản đồ làm điểm ảnh, không dùng bước hiệu chỉnh trọng số)

2.7.2. Đặc trưng patch có ngữ cảnh lân cận

Đặc trưng lấy từ `layer2` (512 kênh) và `layer3` (1024 kênh) của Wide-ResNet-50-2, cùng mạng với PaDiM. `layer1` quá gần mức điểm ảnh, `layer4` quá thiên về phân loại ImageNet. Tại mỗi vị trí, đặc trưng được lấy trung bình trên lân cận 3×3 để mỗi vector mang thông tin vùng xung quanh và bền hơn với dịch chuyển nhỏ. Bản đồ của `layer3` được phóng về lưới của `layer2` (một phần tám kích thước ảnh, tức 32×32 với ảnh 256×256) rồi ghép theo kênh, sau đó vector 1536 chiều được lấy trung bình theo nhóm kênh còn 1024 chiều như bài báo gốc.

2.7.3. Bộ nhớ và chọn coreset

Gom vector của mọi patch trong mọi ảnh tốt được bộ nhớ đầy đủ $\mathcal{M}$: một loại vài trăm ảnh có vài trăm nghìn vector, tìm láng giềng gần nhất trên đó rất chậm. PatchCore giữ lại một tập con $\mathcal{M}_C$ sao cho mọi vector trong $\mathcal{M}$ đều có một đại diện gần nó trong $\mathcal{M}_C$:

$$\mathcal{M}_C^* = \arg \min_{\mathcal{M}_C \subset \mathcal{M}} \max_{m \in \mathcal{M}} \min_{n \in \mathcal{M}_C} \|m - n\|_2. \tag{2.7}$$

Tìm nghiệm đúng của bài toán này quá tốn kém, nên PatchCore giải gần đúng bằng thuật toán tham lam k-center [38]. Mọi vector trong $\mathcal{M}$ trước hết được chiếu ngẫu nhiên xuống 128 chiều, để tính khoảng cách nhanh hơn mà vẫn giữ gần đúng khoảng cách giữa chúng. $\mathcal{M}_C$ được khởi tạo bằng một vector chọn ngẫu nhiên. Ở mỗi bước, thuật toán cập nhật khoảng cách từ mỗi vector trong $\mathcal{M}$ tới vector gần nhất của $\mathcal{M}_C$, rồi thêm vào $\mathcal{M}_C$ vector có khoảng cách đó lớn nhất, tức vector đang được đại diện kém nhất; việc này lặp lại đến khi $\mathcal{M}_C$ đủ kích thước mong muốn.

Khác lấy mẫu ngẫu nhiên, cách chọn này giữ được cả những kiểu patch bình thường hiếm gặp, nhờ đó giảm báo nhầm. Bài báo gốc cho thấy giữ 1% đến 10% số vector vẫn cho kết quả gần như bộ nhớ đầy đủ [3].

Đề tài giữ 10% số vector với MVTec AD và 1% với VisA, MVTec AD 2. Ví dụ grid có 224 ảnh fit, tức khoảng 229 nghìn vector; coreset còn 22,9 nghìn vector, chiếm 45 MB ở dạng số thực 16 bit. Mỗi loại pcb có khoảng 980 nghìn vector, coreset 1% còn khoảng 9,8 nghìn vector (19 MB).

2.7.4. Tính điểm

Điểm của một patch q trong ảnh kiểm thử là khoảng cách tới patch bình thường gần nhất trong coreset:

$$a(q) = \min_{m \in \mathcal{M}_C} \|q - m\|_2. \tag{2.8}$$

Bản đồ các giá trị $a(q)$ được phóng về kích thước ảnh và làm mượt Gauss $\sigma = 4$; điểm ảnh là giá trị lớn nhất của bản đồ. Bài báo gốc nhân thêm một trọng số theo mật độ láng giềng của patch có điểm cao nhất; đề tài bỏ bước này để ba phương pháp dùng chung một cách tính điểm ảnh.

2.7.5. Ưu điểm và hạn chế

PatchCore không phụ thuộc vị trí nên chịu được vật thể xoay hay lệch, không cần huấn luyện mạng và không giả định dạng phân phối. Đổi lại, mỗi patch kiểm thử phải tìm láng giềng gần nhất trong coreset nên suy luận chậm hơn khi bộ nhớ lớn, và bước chọn coreset tốn thời gian khi nhiều ảnh. Việc bỏ thông tin vị trí cũng có mặt trái: từng patch của một vật thể thiếu linh kiện hay lắp sai chỗ vẫn có thể giống một patch bình thường nào đó, nên PatchCore dễ bỏ sót lỗi logic. Cũng như PaDiM, chất lượng phụ thuộc vào đặc trưng học từ ImageNet.

CHƯƠNG 3. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này trình bày các chỉ số đánh giá, tiêu chí so sánh, môi trường và cách chạy thực nghiệm, rồi đến kết quả của ba phương pháp trên 9 loại, năm thí nghiệm chuyên sâu kiểm tra ba phương pháp trong điều kiện khác với giả định của bộ dữ liệu, chi phí tính toán và ứng dụng minh họa.

3.1. Chỉ số đánh giá

3.1.1. Các chỉ số phân loại thông thường

Với bài toán phân loại hai lớp (lỗi là lớp dương, tốt là lớp âm), mỗi dự đoán rơi vào một trong bốn ô của ma trận nhầm lẫn: dương thật (TP), dương giả (FP), âm thật (TN), âm giả (FN). Từ đó có các chỉ số thường dùng:

$$\begin{aligned} \text{Accuracy} &= \frac{\text{TP}+\text{TN}}{\text{TP}+\text{TN}+\text{FP}+\text{FN}}, & \text{Precision} &= \frac{\text{TP}}{\text{TP}+\text{FP}}, \\ \text{Recall} &= \frac{\text{TP}}{\text{TP}+\text{FN}}, & F_1 &= \frac{2\,\text{Precision}\cdot\text{Recall}}{\text{Precision}+\text{Recall}}. \end{aligned} \tag{3.1}$$

Accuracy là tỷ lệ dự đoán đúng; Precision cho biết trong các mẫu bị báo lỗi có bao nhiêu là lỗi thật; Recall cho biết bắt được bao nhiêu phần lỗi thật; F_1 cân bằng hai chỉ số sau. Tất cả các chỉ số phân loại trên đều cần một ngưỡng quyết định cố định, và Accuracy còn bị lệch khi hai lớp chênh lệch số lượng, như ở mức điểm ảnh, nơi gần như mọi điểm ảnh đều bình thường.

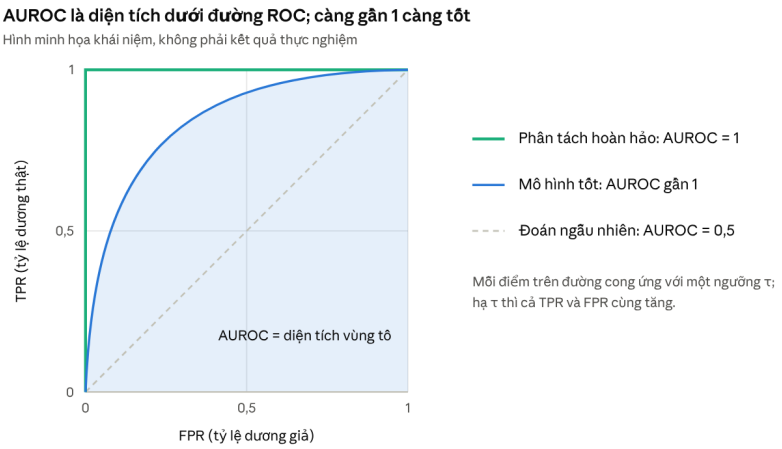
3.1.2. Đường cong ROC và AUROC

Để quyết định tốt hay lỗi, cần một ngưỡng t: ảnh có $s(x) \geq t$ bị coi là lỗi. Với mỗi ngưỡng, tỷ lệ phát hiện đúng (TPR, chính là Recall) và tỷ lệ báo nhầm (FPR) là

$$\text{TPR}(t) = \frac{\text{TP}(t)}{\text{TP}(t)+\text{FN}(t)}, \qquad \text{FPR}(t) = \frac{\text{FP}(t)}{\text{FP}(t)+\text{TN}(t)}. \tag{3.2}$$

Đường cong ROC vẽ TPR theo FPR khi t chạy qua mọi giá trị; AUROC là diện tích dưới đường cong đó [39]. AUROC bằng xác suất một mẫu lỗi chọn ngẫu nhiên có điểm cao hơn một mẫu bình thường chọn ngẫu nhiên. AUROC bằng 1 nghĩa là có ngưỡng tách hoàn toàn hai nhóm; bằng 0,5 nghĩa là không hơn đoán ngẫu nhiên (Hình 3.1).

Đề tài dùng AUROC để so sánh khả năng xếp hạng của các phương pháp mà không phụ thuộc ngưỡng, và để đối chiếu với kết quả đã công bố. Khác với Accuracy, AUROC không phụ thuộc tỷ lệ mẫu tốt/lỗi khi phân phối điểm của từng lớp giữ nguyên. Image-AUROC tính trên điểm $s(x)$ và nhãn tốt/lỗi của các ảnh test; pixel-AUROC tính trên giá trị $A_x(u, v)$ và mặt nạ vùng lỗi của mọi điểm ảnh test. Cả hai được tính riêng cho từng loại sản phẩm rồi lấy trung bình,



Hình 3.1. Đường cong ROC và AUROC (hình minh họa khái niệm, không phải kết quả thực nghiệm)

vì điểm của các mô hình khác loại không cùng thang đo.

3.1.3. AUPRO

Pixel-AUROC tính trên từng điểm ảnh nên vùng lỗi lớn đóng góp nhiều hơn vùng lỗi nhỏ: một mô hình khoanh trúng một vết lớn mà bỏ sót mọi vết nhỏ vẫn có pixel-AUROC cao. AUPRO [1] khắc phục điều đó. Với mỗi ngưỡng, chỉ số tính tỷ lệ phủ của từng vùng lỗi liên thông rồi lấy trung bình, sao cho mọi vùng lỗi có trọng số như nhau; AUPRO là diện tích dưới đường cong tỷ lệ phủ theo tỷ lệ điểm ảnh tốt bị báo nhầm, tính tới mức báo nhầm 30% và chuẩn hóa về $[0, 1]$.

3.1.4. Đặt ngưỡng quyết định

AUROC và AUPRO không cần ngưỡng, nhưng khi dùng thật, mỗi ảnh phải được kết luận tốt hay lỗi. Ngưỡng của mỗi loại và mỗi phương pháp được đặt từ điểm $s_1, \ldots, s_n$ của các ảnh tốt tách riêng, không dùng ảnh test:

$$t = \bar{s} + 3\,\sigma_s, \tag{3.3}$$

với $\bar{s}$ và σ_s là trung bình và độ lệch chuẩn của s_i. Nếu điểm ảnh tốt có phân phối gần chuẩn thì ngưỡng này để lọt khoảng 0,1% ảnh tốt. Đây là cách đặt ngưỡng của bài báo MVTec AD 2 [4]. Ngưỡng điểm ảnh của bản đồ tính cùng cách trên mọi điểm ảnh của ảnh tốt tách riêng, dùng làm mốc thang màu khi vẽ bản đồ. Tại ngưỡng này, đề tài báo cáo FPR và TPR trên tập test theo công thức (3.2).

3.2. Tiêu chí so sánh và cấu hình thực nghiệm

Độ chính xác chưa đủ để chọn phương pháp cho ứng dụng thực tế, nên ba phương pháp được so sánh trên năm nhóm tiêu chí (Bảng 3.1). Bảng 3.2 tóm tắt cấu hình đã chạy của ba phương pháp;

lý do từng lựa chọn ở Bảng 2.1.

Bảng 3.1. Tiêu chí so sánh và cách đo

Tiêu chí	Đại lượng đo	Trả lời câu hỏi
Phát hiện	Image-AUROC theo từng loại và trung bình	Phân biệt ảnh lỗi với ảnh tốt tốt đến đâu
Định vị	Pixel-AUROC và AUPRO theo từng loại và trung bình	Khoanh đúng vùng lỗi đến đâu
Tại ngưỡng	Tỷ lệ ảnh tốt bị báo lỗi (FPR) và tỷ lệ ảnh lỗi bị bắt (TPR) tại ngưỡng đặt trên ảnh tốt tách riêng	Dùng thật thì báo nhầm và bỏ sót bao nhiêu
Chi phí	Thời gian suy luận mỗi ảnh; dung lượng mô hình lưu lại	Có theo kịp tốc độ dây chuyền, cần bao nhiêu bộ nhớ
Định tính	Bản đồ bất thường trên cùng một ảnh lỗi, cùng thang màu theo ngưỡng	Vì sao phương pháp đúng hoặc sai

Bảng 3.2. Cấu hình đã chạy của ba phương pháp

Phương pháp	Cấu hình
Autoencoder	Đầu vào 256×256 trong $[0, 1]$; cổ chai vector 256 chiều; MSE; Adam, tốc độ học $5 \cdot 10^{-4}$, lô 16, 1500 bước; làm mượt $\sigma = 4$; điểm ảnh là giá trị lớn nhất
PaDiM	Wide-ResNet-50-2 học trước; `layer1`–`layer3`, giữ ngẫu nhiên $d = 100$ chiều (seed 1234); $\varepsilon = 0{,}01$; làm mượt $\sigma = 4$; điểm ảnh là giá trị lớn nhất
PatchCore	Wide-ResNet-50-2 học trước; `layer2`–`layer3`, lân cận 3×3, 1024 chiều; coreset 10% (MVTec AD) hoặc 1% (VisA, MVTec AD 2), chiếu ngẫu nhiên 128 chiều; làm mượt $\sigma = 4$; điểm ảnh là giá trị lớn nhất
Chung	Ngưỡng $\bar{s} + 3\sigma_s$ trên ảnh tốt tách riêng; tách 15% `train/good` với seed 0 (MVTec AD, VisA)

3.3. Công cụ, môi trường và cách chạy thực nghiệm

3.3.1. Công cụ và thư viện

Cả ba phương pháp đều do nhóm tự cài đặt bằng PyTorch, không dùng thư viện phát hiện bất thường có sẵn. Bảng 3.3 liệt kê các thư viện được dùng và vai trò của chúng.

3.3.2. Môi trường và thiết lập chung

Kết quả độ chính xác trong chương này lấy từ lần chạy đầy đủ trên Google Colab ngày 03/11/2026; máy cá nhân dùng để chạy thử và đo thời gian suy luận (Bảng 3.4 và Bảng 3.5).

3.3.3. Tổ chức mã nguồn

Mã nguồn viết bằng Python, nằm trong thư mục `src/tn` của kho mã nguồn, tổ chức theo nguyên tắc "cố định cách đánh giá, tự do cách triển khai". Module dữ liệu đọc cả ba bộ dữ liệu về cùng

Bảng 3.3. Công cụ và thư viện sử dụng trong đề tài

Thư viện	Vai trò trong đề tài
PyTorch, torchvision [40]	Cài đặt cả ba phương pháp; huấn luyện Autoencoder; nạp Wide-ResNet-50-2 học trước; làm mượt bản đồ
OpenCV	Đọc và thu nhỏ ảnh, mặt nạ
scikit-learn [41]	Tính image-AUROC và pixel-AUROC
NumPy [42], SciPy	Xử lý mảng bản đồ bất thường; tìm vùng lỗi liên thông khi tính AUPRO
Matplotlib [43]	Vẽ ảnh, mặt nạ và bản đồ bất thường minh họa
pandas	Ghi và tổng hợp bảng kết quả
Gradio	Giao diện web của ứng dụng minh họa
Google Colab	Môi trường chạy thực nghiệm với GPU Tesla T4

Bảng 3.4. Môi trường chạy thực nghiệm

Hạng mục	Xây dựng mô hình và đánh giá (Google Colab)	Đo thời gian suy luận (máy cá nhân)
Phần cứng	GPU NVIDIA Tesla T4 16 GB	GPU NVIDIA RTX 3050 Laptop 4 GB; RAM 16 GB
PyTorch	2.11.0 (CUDA 13), bản cài sẵn của Colab	2.11.0 (CUDA 12.8), torchvision 0.26.0
Thư viện khác	OpenCV, scikit-learn, SciPy, NumPy, pandas bản cài sẵn của Colab	Như bên trái, cài từ `requirements.txt`

một dạng (danh sách ảnh fit, ảnh validation, ảnh test kèm nhãn và mặt nạ) và tiền xử lý như Mục 2.3.3. Module đánh giá tính mọi chỉ số ở Mục 3.1. Ba phương pháp cùng một giao diện: một hàm xây dựng mô hình từ ảnh tốt và một hàm trả về bản đồ bất thường cùng kích thước ảnh. Nhờ vậy, đổi cách cài đặt một phương pháp không làm thay đổi cách đánh giá. Cấu trúc thư mục chi tiết nằm ở Phụ lục.

3.3.4. Quy trình chạy một loại sản phẩm

Chương trình `tools/build_tn_models.py` xử lý lần lượt từng loại. Với mỗi loại, chương trình đọc danh sách ảnh fit, ảnh validation và tập test, nạp ảnh và mặt nạ ở kích thước của loại đó. Sau đó, với từng phương pháp, nó xây dựng mô hình từ ảnh fit, chạy trên ảnh validation để đặt

Bảng 3.5. Các thiết lập dùng chung cho mọi lần chạy

Hạng mục	Thiết lập
Dữ liệu	9 loại của MVTec AD, MVTec AD 2 và VisA (Bảng 2.2); kích thước ảnh theo Mục 2.3.3
Cấu hình	Bảng 3.2; lý do từng lựa chọn ở Bảng 2.1
Seed	Tách validation: 0; chọn chiều PaDiM: 1234; Autoencoder và coreset: 0. Mỗi phương pháp chạy một lần
Mã nguồn	Commit `d0216d1`; lệnh chạy ở Phụ lục

ngưỡng và chạy trên tập test để lấy bản đồ và điểm của từng ảnh. Cuối cùng, chương trình tính các chỉ số và lưu mô hình, ngưỡng, chỉ số cùng điểm của từng ảnh test vào thư mục riêng của loại đó. Ứng dụng minh họa (Mục 3.8) nạp lại đúng các mô hình và ngưỡng đã lưu này.

3.3.5. Cách chạy trên Colab

Thực nghiệm đầy đủ chạy trên Google Colab với GPU Tesla T4. Mã nguồn được nhúng sẵn trong một notebook, nên chỉ cần tải một tệp lên Colab. Dữ liệu được tải về ổ cục bộ của máy Colab, còn kết quả ghi thẳng lên Google Drive, nên khi phiên Colab bị ngắt thì không mất gì. Mỗi loại chạy xong được lưu ngay; khi chạy lại, chương trình bỏ qua các loại đã có kết quả và chạy tiếp từ loại còn thiếu.

3.3.6. Các vấn đề kỹ thuật đã xử lý

Ba vấn đề phát sinh khi chạy trên Colab. Phép nghịch đảo đồng thời hàng nghìn ma trận hiệp phương sai của PaDiM trên GPU Tesla T4 gây lỗi truy cập bộ nhớ trong thư viện CUDA đi kèm PyTorch trên Colab; nhóm tính nghịch đảo trên CPU ở độ chính xác 64 bit, theo từng khối vị trí, rồi mới chuyển kết quả lên GPU. Bộ nhớ đầy đủ của PatchCore cho một loại pcb lên tới khoảng 2 GB; nhóm giữ đặc trưng trên RAM ở dạng số thực 16 bit và chỉ đưa bản chiếu 128 chiều lên GPU khi chọn coreset, thuật toán không đổi. Cuối cùng, đường tải bản nén đầy đủ của MVTec AD trả về lỗi "không tồn tại", nên nhóm tải ba loại cần dùng từ bản sao trên Hugging Face.

3.3.7. Đo thời gian và vẽ hình minh họa

Thời gian suy luận được đo lại trên máy cá nhân (GPU RTX 3050 Laptop, 4 GB) bằng đúng mã của ứng dụng minh họa: mỗi mô hình chạy lần lượt từng ảnh trên 32 ảnh test đầu tiên, sau một lượt chạy khởi động, và chờ GPU làm xong mọi phép tính trước khi dừng đồng hồ. Thời gian gồm cả trích đặc trưng và làm mượt bản đồ, không gồm đọc ảnh từ đĩa.

Hình minh họa gồm ảnh, mặt nạ lỗi và bản đồ của ba phương pháp trên cùng một ảnh lỗi (ảnh đầu tiên của kiểu lỗi đó). Thang màu của mỗi phương pháp đi từ giá trị trung bình của bản đồ trên ảnh tốt tách riêng (tối) tới ngưỡng điểm ảnh của bản đồ (sáng nhất), nên vùng sáng nhất là vùng vượt ngưỡng, và màu so sánh được giữa ba phương pháp.

3.4. Kết quả trên 9 loại sản phẩm

3.4.1. Kết quả tổng hợp

Bảng 3.6 cho kết quả trung bình theo bộ dữ liệu.

PatchCore cao nhất ở cả ba chỉ số và cả ba bộ dữ liệu. Hai điểm khác với dự đoán ban đầu. Thứ nhất, PaDiM và Autoencoder gần bằng nhau ở image-AUROC trung bình (0,828 và 0,820), vì trên VisA Autoencoder đạt 0,925, cao hơn PaDiM 0,068. Thứ hai, cả ba phương pháp đều

Bảng 3.6. Kết quả trung bình theo bộ dữ liệu (cao nhất in đậm)

Chỉ số	Bộ dữ liệu	Autoencoder	PaDiM	PatchCore
Image-AUROC	MVTec AD (3 loại)	0,796	0,870	**0,977**
	MVTec AD 2 (2 loại)	0,644	0,705	**0,806**
	VisA (4 loại)	0,925	0,857	**0,972**
	Cả 9 loại	0,820	0,828	**0,937**
Pixel-AUROC	MVTec AD	0,904	0,962	**0,984**
	MVTec AD 2	0,789	0,862	**0,908**
	VisA	0,959	0,980	**0,989**
	Cả 9 loại	0,903	0,948	**0,969**
AUPRO	MVTec AD	0,744	0,896	**0,935**
	MVTec AD 2	0,510	0,573	**0,614**
	VisA	0,794	0,849	**0,900**
	Cả 9 loại	0,714	0,803	**0,848**

giảm mạnh trên MVTec AD 2: image-AUROC của PatchCore còn 0,806 và AUPRO chỉ 0,614. Hai loại của bộ này giảm vì hai lý do khác nhau: vial bị ảnh hưởng bởi ánh sáng lúc kiểm thử khác lúc huấn luyện (Mục 3.4.3), còn sheet_metal kém ở mọi điều kiện chụp vì vết xước quá mảnh so với độ phân giải của đặc trưng (Mục 3.5.5).

3.4.2. Kết quả theo từng loại

Bảng 3.7. Image-AUROC và pixel-AUROC theo từng loại (AE: Autoencoder; cao nhất mỗi chỉ số in đậm)

		Image-AUROC			Pixel-AUROC		
Bộ	Loại	AE	PaDiM	PatchCore	AE	PaDiM	PatchCore
MVTec AD	grid	0,836	0,935	**0,988**	0,834	0,946	**0,981**
	screw	0,766	0,708	**0,943**	0,978	0,970	**0,991**
	transistor	0,786	0,967	**0,999**	0,900	0,970	**0,979**
MVTec AD 2	vial	0,686	0,633	**0,842**	0,887	0,882	**0,923**
	sheet_metal	0,601	**0,778**	0,771	0,692	0,842	**0,893**
VisA	pcb1	0,945	0,865	**0,974**	0,957	0,988	**0,998**
	pcb2	0,928	0,836	**0,955**	0,985	0,981	**0,987**
	pcb3	0,874	0,764	**0,966**	0,974	0,986	**0,989**
	pcb4	0,954	0,963	**0,994**	0,919	0,966	**0,984**

PatchCore cao nhất ở 8/9 loại theo image-AUROC và đủ 9/9 loại theo pixel-AUROC và AUPRO. Ngoại lệ duy nhất là sheet_metal, nơi PaDiM nhỉnh hơn 0,007. Trên MVTec AD và VisA, image-AUROC của PatchCore từ 0,943 (screw) đến 0,999 (transistor).

PaDiM thấp nhất trong ba phương pháp ở screw (0,708), vial (0,633) và pcb1–pcb3. Pixel-AUROC của nó vẫn từ 0,946 trở lên ở MVTec AD và VisA, tức bản đồ vẫn khoanh đúng vùng

Bảng 3.8. AUPRO theo từng loại (cao nhất in đậm)

	grid	screw	transistor	vial	sheet_metal	pcb1	pcb2	pcb3	pcb4
AE	0,607	0,908	0,718	0,775	0,245	0,866	0,883	0,838	0,586
PaDiM	0,870	0,889	0,929	0,754	0,393	0,897	0,840	0,824	0,835
PatchCore	**0,902**	**0,949**	**0,952**	**0,797**	**0,431**	**0,940**	**0,901**	**0,866**	**0,892**

lỗi, nhưng điểm ảnh lấy giá trị lớn nhất của bản đồ nên chỉ cần vài vị trí phản hồi mạnh trên ảnh tốt là điểm của ảnh tốt bị đẩy lên. Với screw, đó là hệ quả của việc con vít xoay: tại một vị trí cố định, ảnh tốt lúc là thân vít, lúc là nền, nên phân phối Gauss tại vị trí đó rất rộng và kém phân biệt.

Autoencoder tốt bất ngờ trên VisA (image-AUROC 0,874 đến 0,954), cao hơn PaDiM ở pcb1, pcb2, pcb3. Bảng mạch được chụp trên nền phẳng, lỗi thường là vết cháy, xước hay thiếu linh kiện làm đổi màu cả một vùng, nên sai số tái tạo đủ lớn. Ngược lại, Autoencoder gần mức ngẫu nhiên ở sheet_metal (0,601) và vial (0,686), và thấp nhất ở mọi loại MVTec AD trừ screw. Xét theo kiểu lỗi, AUROC của Autoencoder chỉ 0,563 với screw/thread_side (ren bên thân, vùng lỗi vài chục điểm ảnh) và 0,550 với transistor/damaged_case, nhưng đạt 0,965 với transistor/misplaced, kiểu lỗi mà cả linh kiện lệch khỏi chỗ. Các số này lấy từ lần chạy ở mục này; Bảng 3.21 và Bảng 3.22 dùng mốc huấn luyện trên toàn bộ `train/good` (Mục 3.5.1) nên lệch nhẹ (0,643; 0,542; 0,978), nhưng thứ tự giữa các kiểu lỗi không đổi.

Đối chiếu với ba giả thuyết đặt ra ở Mục 2.4, giả thuyết G1 đúng với PatchCore nhưng sai với PaDiM. PatchCore cao hơn Autoencoder ở cả 9 loại theo image-AUROC và pixel-AUROC, còn PaDiM chỉ cao hơn Autoencoder ở 4/9 loại theo image-AUROC và 6/9 loại theo pixel-AUROC; trên VisA, đặc trưng học trước không đủ bù cho ràng buộc theo vị trí.

Giả thuyết G2 đúng. Khoảng cách image-AUROC lớn nhất giữa PatchCore và PaDiM là ở screw (0,235), tiếp theo là vial (0,209) và pcb3 (0,202). Vì hai phương pháp dùng cùng backbone và cùng cách tính điểm ảnh, chênh lệch này đến từ cơ chế: PatchCore so mỗi patch với mọi patch bình thường, không phụ thuộc vị trí.

Giả thuyết G3 đúng một phần. Autoencoder tốt ở VisA và transistor/misplaced, nơi lỗi lớn hoặc đổi màu rõ, và kém ở các lỗi nhỏ (screw/thread_side, sheet_metal). Nhưng screw lại là loại MVTec AD duy nhất Autoencoder hơn PaDiM, vì nền trắng đơn giản và Autoencoder không bị ảnh hưởng bởi việc vít xoay.

3.4.3. Kết quả tại ngưỡng

AUROC chỉ cho biết khả năng xếp hạng. Khi dùng thật, mỗi ảnh phải được kết luận tốt hay lỗi theo một ngưỡng chốt trước. Bảng 3.9 cho tỷ lệ báo nhầm (FPR, ảnh tốt bị báo lỗi) và tỷ lệ bắt lỗi (TPR, ảnh lỗi bị phát hiện) trên tập test, tại ngưỡng $\bar{s} + 3\sigma_s$ đặt trên ảnh tốt tách riêng (Mục 3.1.4).

Ngưỡng $\bar{s} + 3\sigma_s$ giữ được tỷ lệ báo nhầm thấp: trung bình 2% với Autoencoder và PaDiM,

Bảng 3.9. Tỷ lệ báo nhầm (FPR) và tỷ lệ bắt lỗi (TPR) tại ngưỡng đặt trên ảnh tốt tách riêng

	FPR			TPR		
Loại	AE	PaDiM	PatchCore	AE	PaDiM	PatchCore
grid	0,00	0,05	0,05	0,00	0,56	0,96
screw	0,00	0,02	0,05	0,08	0,08	0,72
transistor	0,02	0,00	0,03	0,15	0,68	1,00
vial	0,00	0,00	0,14	0,04	0,00	0,52
sheet_metal	0,00	0,00	0,13	0,21	0,00	0,41
pcb1	0,04	0,03	0,04	0,43	0,13	0,73
pcb2	0,00	0,03	0,04	0,40	0,17	0,77
pcb3	0,08	0,01	0,00	0,41	0,17	0,67
pcb4	0,03	0,02	0,03	0,49	0,52	0,96
Trung bình	0,02	0,02	0,06	0,25	0,26	0,75

6% với PatchCore. Cái giá là bỏ sót: Autoencoder và PaDiM chỉ bắt được khoảng một phần tư ảnh lỗi, PatchCore bắt được 75%. Ở grid, transistor và pcb4, PatchCore bắt được từ 96% ảnh lỗi trở lên với không quá 5% báo nhầm. Bảng này cho thấy rõ hơn AUROC khoảng cách giữa các phương pháp khi dùng thật: PaDiM có image-AUROC 0,935 ở grid nhưng tại ngưỡng chỉ bắt được 56% ảnh lỗi, vì điểm của ảnh lỗi và ảnh tốt nằm sát nhau ở vùng ngưỡng.

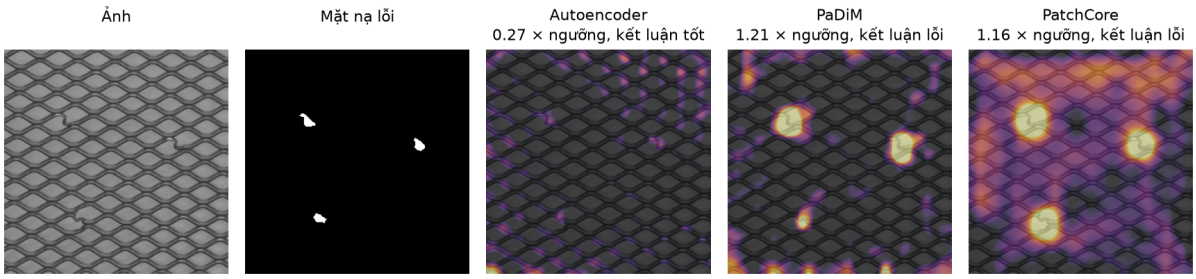
Bảng 3.10. PatchCore trên vial theo điều kiện chụp (mỗi điều kiện 5 ảnh tốt, 15 ảnh lỗi)

	bình thường	quá sáng	quá tối	dịch 1	dịch 2	dịch 3	dịch 4
Image-AUROC	0,907	0,920	0,667	0,907	0,920	0,920	0,907
FPR	0,00	0,00	1,00	0,00	0,00	0,00	0,00
TPR	0,40	0,47	0,93	0,53	0,33	0,47	0,53

Tỷ lệ báo nhầm 14% của PatchCore trên vial đến hoàn toàn từ một điều kiện (Bảng 3.10): khi ảnh quá tối, cả 5 ảnh tốt đều vượt ngưỡng, nên "bắt lỗi" 93% ở điều kiện này không có ý nghĩa. Ở các điều kiện còn lại, PatchCore không báo nhầm ảnh nào và image-AUROC giữ quanh 0,91. Như vậy, đặc trưng học trước chịu được phần lớn thay đổi ánh sáng, nhưng ngưỡng đặt trên ảnh chụp ở điều kiện bình thường thì không: ảnh quá tối làm điểm của mọi ảnh tăng lên. Trên sheet_metal, báo nhầm 13% của PatchCore rải đều ở ba mức dịch ánh sáng (mỗi mức 1/4 ảnh tốt). Autoencoder và PaDiM có FPR bằng 0 ở cả hai loại, nhưng ngưỡng của chúng cao hơn điểm của hầu hết ảnh lỗi, nên chỉ bắt được 0–21% ảnh lỗi.

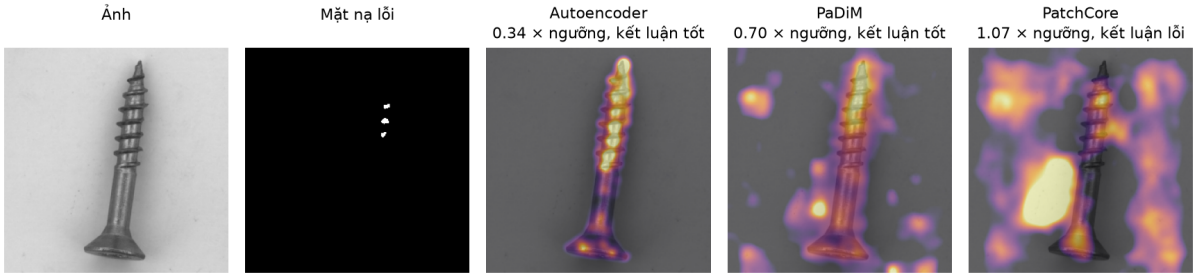
3.4.4. Phân tích định tính

Các hình dưới đây so sánh bản đồ của ba phương pháp trên cùng một ảnh lỗi. Thang màu của mỗi phương pháp đi từ mức trung bình trên ảnh tốt tách riêng (tối) tới ngưỡng điểm ảnh (sáng nhất), nên vùng sáng là vùng vượt ngưỡng và màu so sánh được giữa ba phương pháp. Tiêu đề mỗi bản đồ ghi điểm của ảnh chia cho ngưỡng ảnh và kết luận tương ứng.



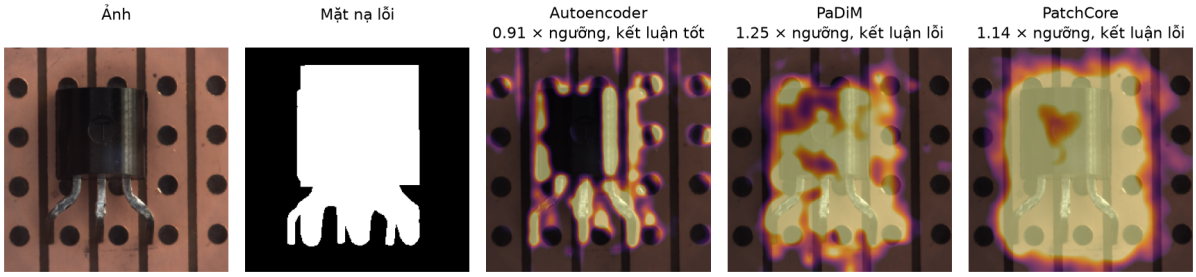
Hình 3.2. Grid, lỗi cong lưới

Với lưới bị cong ở ba chỗ (Hình 3.2), PaDiM và PatchCore sáng đúng cả ba vết, nền gần như tối. Autoencoder chỉ hơi sáng ở viền vết cong: lưới lặp lại được dựng lại khá tốt ở mọi nơi, kể cả chỗ cong, nên sai số tại vùng lỗi không nổi hơn nền và điểm ảnh chỉ bằng 0,27 lần ngưỡng.



Hình 3.3. Screw, lỗi ren bên thân

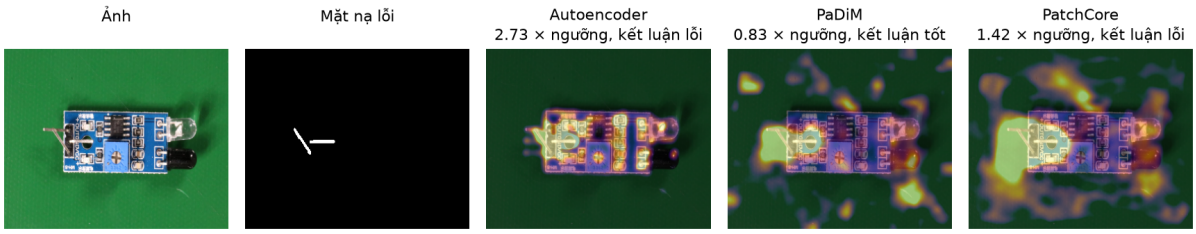
Với con vít lỗi ren bên thân (Hình 3.3), vùng lỗi chỉ là ba chấm nhỏ. Autoencoder sáng dọc cả thân vít, nơi ren có nhiều cạnh sắc khó tái tạo. PaDiM có nhiều đốm sáng rải rác trên nền, đúng kiểu phản hồi sai do vít xoay đã nêu ở mục trước. PatchCore vượt ngưỡng nhưng vùng sáng nhất nằm trên nền bên trái con vít chứ không ở vết ren; ảnh này được kết luận đúng là lỗi nhưng vì lý do sai. Đây là giới hạn của việc chỉ nhìn điểm ảnh: cần xem cả bản đồ.



Hình 3.4. Transistor, lỗi lệch vị trí (cả linh kiện bị đặt lệch khỏi chỗ)

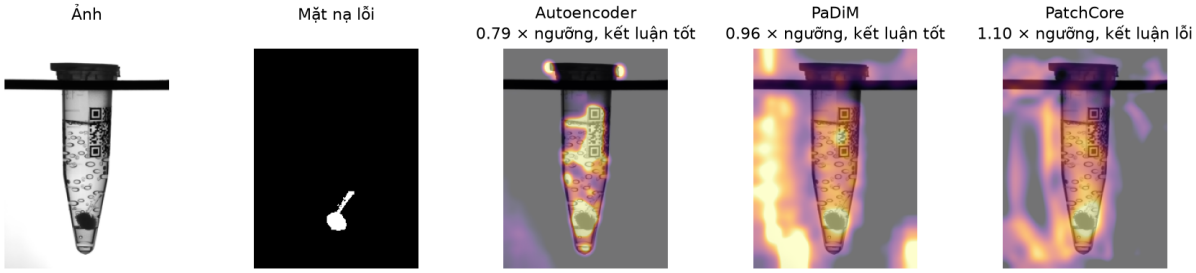
Với transistor bị đặt lệch (Hình 3.4), mặt nạ đánh dấu cả vùng linh kiện lẽ ra phải nằm. PatchCore và PaDiM phủ gần trọn vùng đó. Autoencoder sáng ở viền chân và viền thân linh kiện, nơi ảnh dựng lại khác ảnh thật nhiều nhất, nhưng vẫn dưới ngưỡng (0,91 lần).

Với bảng mạch có chân linh kiện bị cong (Hình 3.5), Autoencoder cho điểm cao nhất (2,73 lần ngưỡng) và vùng sáng nằm đúng quanh chân cong ở mép trái bảng mạch. PatchCore cũng



Hình 3.5. Pcb3, chân linh kiện bị cong

sáng đúng vùng đó. PaDiM có phản hồi ở vùng lỗi nhưng cả nền xanh phía trên và dưới bảng mạch cũng sáng, nên điểm ảnh không vượt ngưỡng. Ví dụ này minh họa vì sao Autoencoder hơn PaDiM trên VisA.



Hình 3.6. Vial, ảnh lỗi chụp ở điều kiện quá sáng

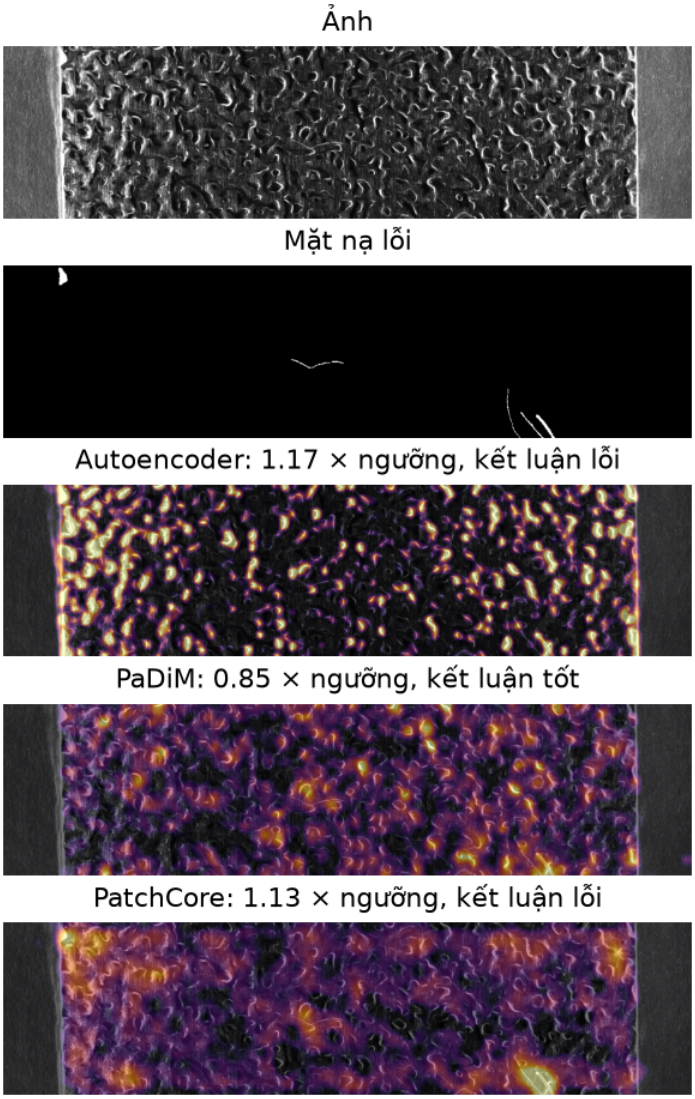
Hai loại của MVTec AD 2 cho thấy giới hạn chung (Hình 3.6 và 3.7). Với vial chụp quá sáng, PaDiM sáng mạnh ở hai bên nền, nơi ánh sáng khác lúc huấn luyện, và không thấy cục lắng ở đáy lọ. PatchCore vượt ngưỡng (1,10 lần) nhưng vùng sáng cũng nằm chủ yếu ở nền. Với sheet_metal, các vết xước mảnh vài điểm ảnh sau khi thu nhỏ, trên một bề mặt vốn lấm tấm; cả ba bản đồ đều lốm đốm trên toàn tấm, không phương pháp nào khoanh rõ vết xước. Điều này khớp với AUPRO chỉ 0,25–0,43 ở loại này.

3.5. Các thí nghiệm chuyên sâu

3.5.1. Thiết lập các thí nghiệm

Trên dữ liệu chuẩn, PatchCore đã đạt image-AUROC trên 0,95 ở hầu hết các loại, nên con số này không còn phân biệt được các phương pháp. Câu hỏi thực tế hơn là khi điều kiện nhà máy khác với giả định của bộ dữ liệu, phương pháp nào hỏng trước, hỏng theo cách nào, và có đoán trước được từ cơ chế của nó không. Mỗi phương pháp có một giả định ngầm riêng: Autoencoder giả định lỗi khó tái tạo hơn vùng bình thường; PaDiM giả định ảnh đã căn chỉnh và tại mỗi vị trí chỉ có một kiểu bình thường; PatchCore giả định mọi patch bình thường đều đã có một patch gần nó trong bộ nhớ.

Từ đó nhóm đặt năm câu hỏi, ứng với năm thí nghiệm. Thí nghiệm 1 hỏi phương pháp nào suy giảm khi tập huấn luyện lẫn ảnh lỗi. Thí nghiệm 2 hỏi phương pháp nào mất khả năng



Hình 3.7. Sheet_metal, ba vết xước mảnh, ảnh chụp ở điều kiện quá sáng

phát hiện nhanh hơn khi chỉ có 1 đến 16 ảnh tốt. Thí nghiệm 3 hỏi mô hình học trên một loại sản phẩm dùng được cho loại khác tới đâu, và cần bao nhiêu ảnh của loại mới để thích nghi. Thí nghiệm 4 hỏi phương pháp nào nhằm thay đổi điều kiện chụp với lỗi thật nhiều hơn. Thí nghiệm 5 hỏi các thất bại cụ thể có đoán trước được từ cơ chế hay không.

Các thí nghiệm dùng grid, screw, transistor của MVTec AD, hai loại của MVTec AD 2 và bốn loại pcb của VisA, với đúng cấu hình ở Bảng 3.2 và ngưỡng ở Mục 3.1.4. Thí nghiệm 1, 2 và 5 so với một mốc huấn luyện trên toàn bộ `train/good` (Bảng 3.11), khác với mốc 9 loại ở Mục 3.4 vốn tách 15% ảnh để đặt ngưỡng, nên các số liệu có chênh lệch nhỏ. Chênh lệch lớn nhất là ở Autoencoder (ví dụ transistor 0,753 so với 0,786), cũng là phương pháp dao động nhiều nhất giữa các lần chạy (Mục 3.5.2). Thí nghiệm 1 lặp 5 seed, Thí nghiệm 2 lặp 3 seed, các thí nghiệm còn lại chạy một lần. Tất cả chạy trên Google Colab; một phần của Thí nghiệm 4 chạy trên CPU khi hết hạn mức GPU.

Bảng 3.11. Image-AUROC của mốc huấn luyện trên toàn bộ `train/good`

Loại	Autoencoder	PaDiM	PatchCore
grid	0,825	0,934	0,989
screw	0,787	0,722	0,955
transistor	0,753	0,968	0,999

3.5.2. Thí nghiệm 1: dữ liệu huấn luyện lẫn ảnh lỗi

Trong nhà máy, tập ảnh tốt thường được thu tự động và có thể lẫn một ít sản phẩm lỗi chưa được phát hiện. Mỗi phương pháp có một cách hỏng dự kiến. Autoencoder có thể học luôn cách tái tạo lỗi, nên sai số trên lỗi thật giảm. Với PaDiM, trung bình dịch về phía lỗi và hiệp phương sai phình ra, nên khoảng cách Mahalanobis của lỗi nhỏ lại. PatchCore ghi patch lỗi vào bộ nhớ, nên lỗi cùng kiểu ở tập test tìm được láng giềng rất gần và có điểm thấp.

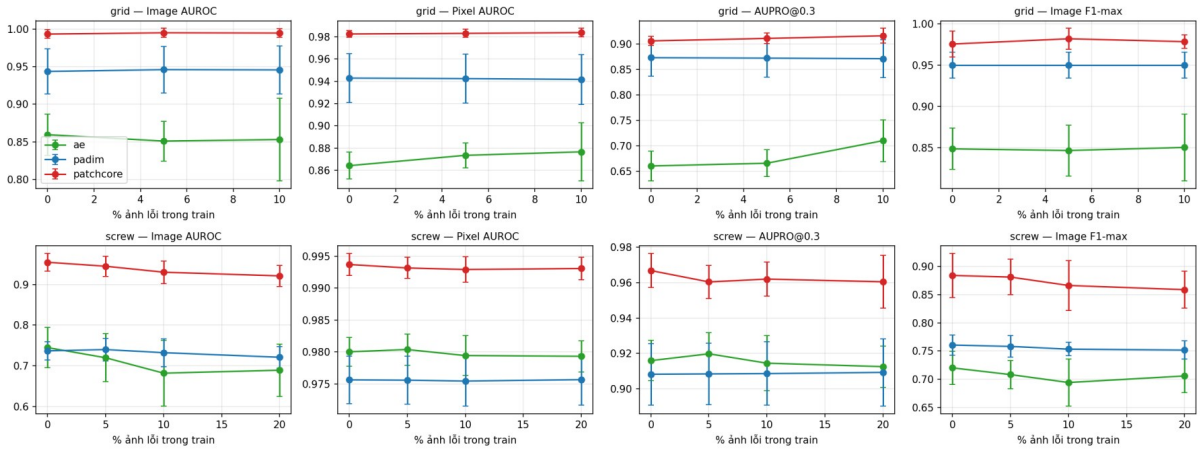
Nhóm trộn 0%, 5%, 10% và 20% ảnh lỗi vào tập huấn luyện của screw và grid; ảnh lỗi lấy từ tập test rồi bị loại khỏi tập đánh giá, mỗi mức lặp 5 seed. Grid chỉ có 57 ảnh lỗi nên dừng ở 10% (29 ảnh); số ảnh lỗi trộn vào screw ở ba mức là 17, 36 và 80. Để các mức so sánh được với nhau, toàn bộ kho ảnh lỗi dành cho mức cao nhất bị bỏ khỏi tập đánh giá ở mọi mức, kể cả mức 0%. Vì vậy mức 0% ở Bảng 3.12 là trung bình 5 seed trên một tập test nhỏ hơn, và lệch nhẹ so với mốc chạy một lần trên toàn bộ tập test ở Bảng 3.11 (ví dụ PaDiM trên screw 0,737 so với 0,722).

Bảng 3.12. Image-AUROC theo tỷ lệ ảnh lỗi trong tập huấn luyện (trung bình 5 seed)

Loại	Phương pháp	0%	5%	10%	20%	Chênh lệch
screw	Autoencoder	0,745	0,720	0,682	0,689	$-0{,}056$
	PaDiM	0,737	0,740	0,732	0,721	$-0{,}016$
	PatchCore	0,955	0,945	0,930	0,921	$-0{,}034$
grid	Autoencoder	0,860	0,851	0,853	–	≈ 0
	PaDiM	0,944	0,946	0,946	–	≈ 0
	PatchCore	0,993	0,995	0,995	–	≈ 0

Không phương pháp nào sụp đổ khi tập huấn luyện lẫn tới 20% ảnh lỗi (Bảng 3.12, Hình 3.8). Mức suy giảm lớn nhất chỉ 3 đến 6 điểm image-AUROC và chỉ xuất hiện trên screw; trên grid cả ba gần như không đổi. Lỗi trên grid là vùng nhỏ trên một bề mặt lặp lại, nên vài ảnh lỗi chỉ thêm một lượng rất nhỏ patch bất thường vào hàng chục nghìn patch tốt, không đủ để kéo lệch phân phối Gauss hay chiếm bộ nhớ.

Hình dạng đường suy giảm khớp với cơ chế. PatchCore giảm gần tuyến tính và nhất quán, cả 5 seed đều giảm trên screw: mỗi ảnh lỗi thêm vào là thêm một số patch lỗi vào bộ nhớ, nên ảnh hưởng cộng dồn theo số ảnh. PaDiM phẳng tới 10% và chỉ giảm nhẹ ở 20%, vì phân phối Gauss là thống kê trung bình trên hàng trăm ảnh nên vài ảnh lỗi bị pha loãng. Autoencoder giảm ở 5–10% rồi đi ngang, nhưng 2 trong 5 seed lại tăng image-AUROC, nên không kết luận được



Hình 3.8. Image-AUROC và AUPRO theo tỷ lệ ảnh lỗi trong tập huấn luyện trên screw và grid (trung bình 5 seed, thanh lỗi là độ lệch chuẩn)

đây là hiệu ứng thật hay dao động của quá trình huấn luyện.

Khả năng định vị gần như không đổi: trên screw, AUPRO thay đổi $-0{,}006$ với PatchCore, gần 0 với PaDiM và $-0{,}003$ với Autoencoder, còn pixel-AUROC thay đổi khoảng $-0{,}001$. Dữ liệu bẩn làm hỏng khả năng xếp hạng ảnh, vốn dựa vào giá trị lớn nhất của bản đồ và rất nhạy với vài patch, nhưng không làm hỏng hình dạng bản đồ. Riêng AUPRO của Autoencoder trên grid còn tăng từ 0,660 lên 0,710 ở mức 10%; nhóm chưa có thí nghiệm để giải thích hiện tượng này.

Độ lệch chuẩn image-AUROC giữa 5 seed là 0,064 (screw) và 0,036 (grid) với Autoencoder, 0,028 và 0,031 với PaDiM, 0,025 và 0,006 với PatchCore. Trên screw, Autoencoder dao động gấp 2 đến 3 lần hai phương pháp còn lại; trên grid nó chỉ ngang PaDiM nhưng gấp 6 lần PatchCore. Lý do là ngoài việc chọn ảnh lỗi nào được trộn vào, mạng còn được huấn luyện từ khởi tạo ngẫu nhiên. Trên screw, độ dao động này còn lớn hơn chính mức suy giảm do dữ liệu bẩn (0,056); với Autoencoder, seed ảnh hưởng tới kết quả nhiều hơn việc làm sạch dữ liệu.

Như vậy cả ba phương pháp đều khá bền với 5–20% dữ liệu bẩn. PatchCore suy giảm đều theo số ảnh lỗi nhưng vẫn tốt nhất về giá trị tuyệt đối: 0,921 ở mức 20% vẫn cao hơn PaDiM trên dữ liệu sạch (0,737).

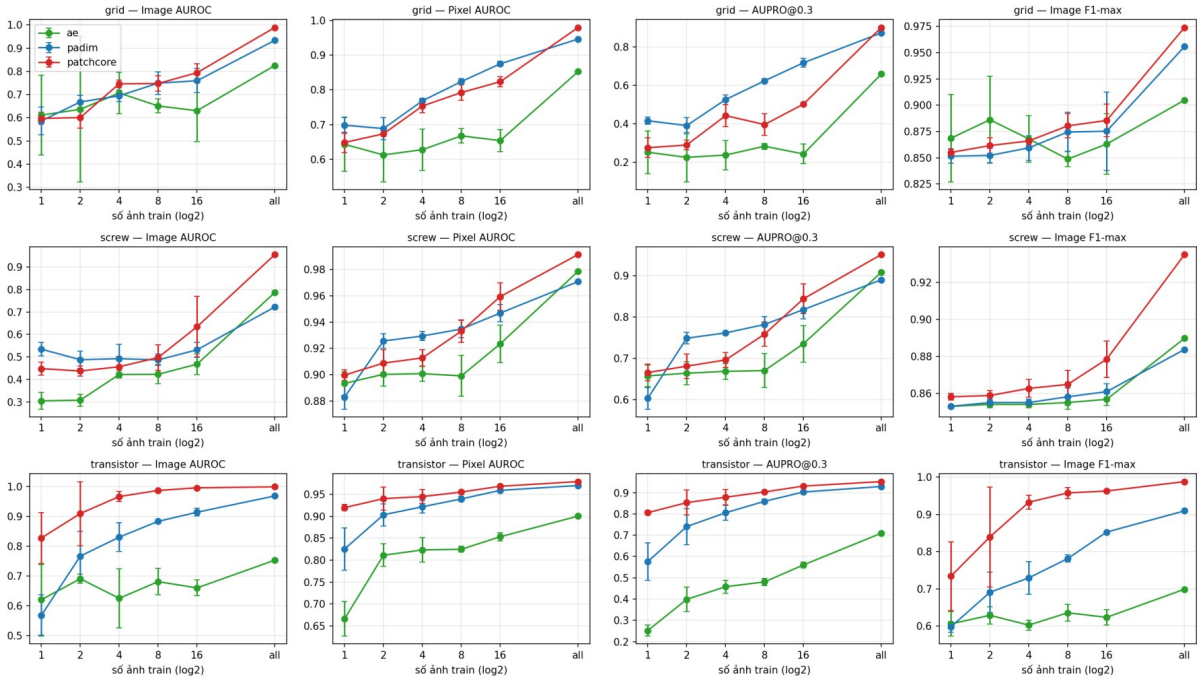
3.5.3. Thí nghiệm 2: chỉ có vài ảnh tốt

Khi một dây chuyền mới chỉ có vài ảnh sản phẩm tốt, ba cách học phản ứng khác nhau. PaDiM phải ước lượng trung bình và ma trận hiệp phương sai 100×100 tại mỗi vị trí; với ít ảnh, ma trận gần suy biến và chỉ còn nhờ thành phần $0{,}01\,I$, nên khoảng cách Mahalanobis gần như thành khoảng cách Euclid tới trung bình. PatchCore không cần ước lượng thống kê và mỗi ảnh vẫn cho hàng nghìn patch, nhưng bộ nhớ chỉ bao phủ những biến thể đã thấy. Autoencoder cần đủ dữ liệu để học cấu trúc chung; với vài ảnh, nó dễ học thuộc hoặc dựng ra ảnh mờ.

Với mỗi loại, nhóm chọn ngẫu nhiên $k \in \{1, 2, 4, 8, 16\}$ ảnh tốt từ tập huấn luyện (3 seed), xây dựng mô hình rồi đánh giá trên toàn bộ tập test, và so với mốc ở Bảng 3.11.

Bảng 3.13. Image-AUROC và AUPRO theo số ảnh huấn luyện k (trung bình 3 seed; "đủ" là mốc dùng toàn bộ tập huấn luyện)

		Image-AUROC			AUPRO		
Loại	Phương pháp	đủ	$k = 16$	$k = 1$	đủ	$k = 16$	$k = 1$
grid	Autoencoder	0,825	0,630	0,611	0,658	0,244	0,253
	PaDiM	0,934	0,759	0,586	0,872	0,717	0,416
	PatchCore	0,989	0,794	0,597	0,900	0,502	0,276
screw	Autoencoder	0,787	0,468	0,305	0,908	0,736	0,658
	PaDiM	0,722	0,532	0,535	0,890	0,818	0,604
	PatchCore	0,955	0,634	0,448	0,951	0,844	0,666
transistor	Autoencoder	0,753	0,660	0,621	0,709	0,561	0,252
	PaDiM	0,968	0,914	0,568	0,929	0,903	0,576
	PatchCore	0,999	0,995	0,827	0,952	0,931	0,806



Hình 3.9. Image-AUROC và AUPRO theo số ảnh huấn luyện k cho grid, screw, transistor (trung bình 3 seed; đường ngang là mốc dùng toàn bộ tập huấn luyện)

Phương pháp nào mất khả năng phát hiện nhanh hơn phụ thuộc mạnh vào loại sản phẩm (Bảng 3.13, Hình 3.9). Với transistor, vật thể cố định và ảnh được căn chỉnh tốt, PatchCore gần như không mất gì: còn 0,995 với 16 ảnh, đạt 0,966 với 4 ảnh và vẫn 0,827 với 1 ảnh. PaDiM giữ được tới 16 ảnh (0,914) nhưng rơi về 0,568 khi chỉ có 1 ảnh, đúng dự đoán vì một ảnh không đủ để ước lượng phân phối. Với grid, cả ba giảm về khoảng 0,6 khi chỉ có 1 ảnh, dù một ảnh grid đã chứa rất nhiều patch lặp lại; khả năng là lưới trong MVTec AD thay đổi góc và tỷ lệ giữa các ảnh, nên vài ảnh không bao phủ hết các biến thể. Với screw, cả ba ở gần hoặc dưới mức ngẫu nhiên cho tới 8 ảnh. Autoencoder với 1 ảnh chỉ còn 0,305, tức xếp hạng ngược: ảnh tốt bị chấm

điểm cao hơn ảnh lỗi. Một mô hình dưới 0,5 không chỉ yếu mà đang phản ứng với một tín hiệu khác với lỗi; Mục 3.5.3.1 tìm tín hiệu đó.

Phát hiện và định vị không suy giảm cùng nhịp, và thứ tự phụ thuộc vào loại. Trên screw, AUPRO giảm chậm hơn image-AUROC ở cả ba phương pháp: với 16 ảnh, PatchCore chỉ còn image-AUROC 0,634 nhưng AUPRO vẫn 0,844. Mô hình vẫn thấy vùng lỗi sáng lên trong ảnh lỗi, nhưng ảnh tốt cũng có những vùng sáng khác do góc xoay chưa gặp, nên giá trị lớn nhất của ảnh không còn phân biệt được. Trên grid thì ngược lại: với 16 ảnh, AUPRO của PatchCore giảm từ 0,900 xuống 0,502 và của Autoencoder từ 0,658 xuống 0,244, gấp đôi mức giảm image-AUROC, vì vài ảnh không bao phủ hết biến thể của lưới nên bản đồ sáng cả ở vùng tốt quanh vết lỗi. PaDiM giữ AUPRO trên grid tốt hơn PatchCore rõ rệt (0,717 so với 0,502): cách học theo vị trí có lợi khi bề mặt đồng nhất. Trên transistor, cả hai chỉ số của PatchCore và PaDiM gần như không đổi tới 16 ảnh, còn Autoencoder mất khả năng định vị nhanh hơn khả năng phát hiện.

Thứ hạng của ba phương pháp không nhất quán giữa các loại. Điều quyết định không phải bản thân số ảnh, mà là k ảnh đó có bao phủ được biến thiên tự nhiên của ảnh tốt (góc, vị trí, ánh sáng) hay không. Với transistor, vật thể cố định và ảnh gần như giống hệt nhau, PatchCore chỉ cần 4 ảnh; grid tuy không xoay nhưng lưới thay đổi giữa các ảnh, nên 16 ảnh vẫn chưa đủ (0,794); với screw xoay tự do, cả ba phương pháp vẫn dưới 0,64 ở 16 ảnh, tức không dùng được trong khoảng số ảnh đã thử.

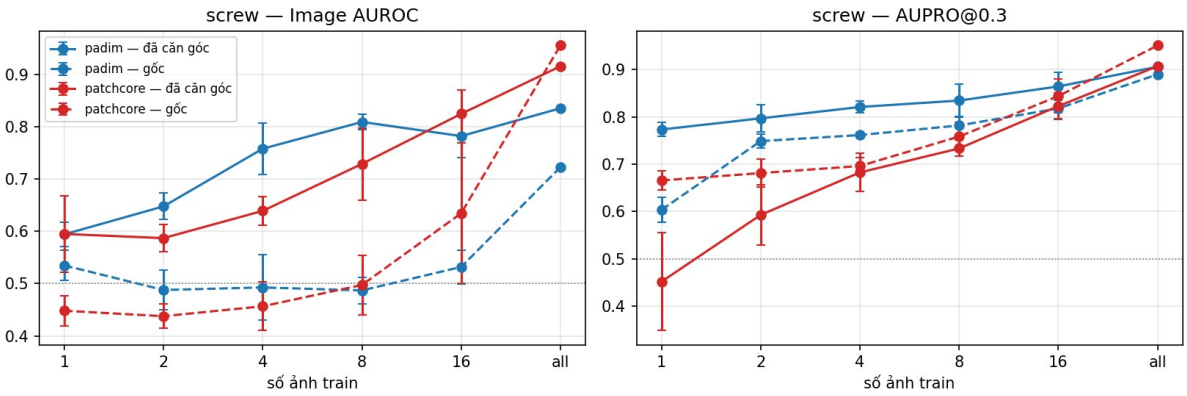
3.5.3.1. Vì sao screw thất bại khi có ít ảnh

Image-AUROC dưới 0,5 trên screw gợi ý mô hình đang chấm điểm theo một yếu tố không liên quan tới lỗi. Nhóm kiểm tra hai ứng viên: góc xoay của con vít và độ sáng của ảnh.

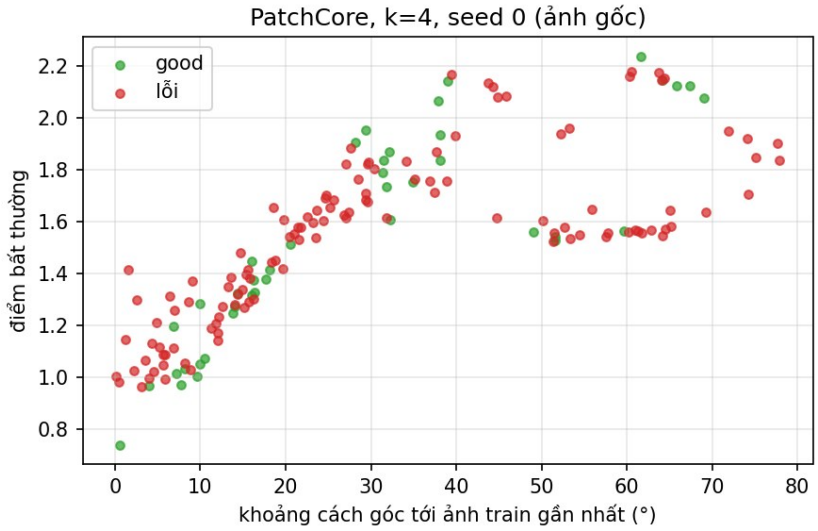
Góc của mỗi con vít được ước lượng từ trục chính của vùng tối là thân vít (Hình 3.10). Với mỗi ảnh test, nhóm tính khoảng cách góc tới ảnh huấn luyện gần nhất, rồi đo tương quan Spearman giữa khoảng cách này và điểm bất thường (Hình 3.11). Tương quan là 0,82 đến 0,92 với PatchCore (4 và 8 ảnh) và 0,38 đến 0,60 với PaDiM: với PatchCore, góc xoay giải thích gần như toàn bộ thứ hạng điểm. Trong khi đó, khoảng cách góc trung bình của ảnh tốt và ảnh lỗi gần như bằng nhau (35,1° so với 33,4° với 4 ảnh), nên điểm bị chi phối bởi một yếu tố không phân biệt tốt với lỗi, và image-AUROC xoay quanh 0,5.

Để kiểm chứng, nhóm xoay mọi ảnh, cả huấn luyện và test, về cùng một hướng trước khi xây dựng mô hình (Bảng 3.14, Hình 3.12). Căn chỉnh tăng image-AUROC 19 đến 32 điểm khi có ít ảnh, và tăng 11 điểm cho PaDiM khi dùng toàn bộ tập huấn luyện, xác nhận góc xoay là nguyên nhân chính. Riêng PatchCore với toàn bộ tập huấn luyện giảm nhẹ từ 0,955 xuống 0,915, chủ yếu do kiểu lỗi thread_side giảm từ 0,917 xuống 0,795: khi đủ ảnh, bộ nhớ đã tự bao phủ mọi góc, còn phép xoay với nội suy lại làm mờ chi tiết ren nhỏ.

Khi kiểm tra độ sáng, nhóm phát hiện một độ lệch có hệ thống trong dữ liệu (Bảng 3.15): ảnh test tốt của screw tối hơn rõ rệt so với cả ảnh huấn luyện lẫn ảnh lỗi. Chỉ dùng độ sáng trung bình làm điểm bất thường đã đạt AUROC bằng 1,0 trên 86 ảnh kiểm tra (41 ảnh tốt và 45 trong số 119 ảnh lỗi). Một mô hình nhạy với độ sáng vì vậy thấy ảnh test tốt "lạ" hơn ảnh lỗi và cho



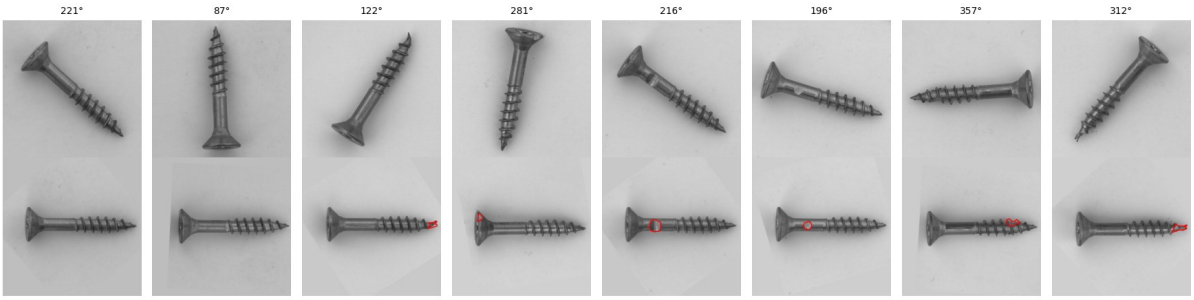
Hình 3.10. Ảnh screw tốt ở nhiều góc xoay khác nhau, kèm góc ước lượng



Hình 3.11. Điểm bất thường theo khoảng cách góc tới ảnh huấn luyện gần nhất

Bảng 3.14. Image-AUROC trên screw trước và sau khi căn chỉnh góc

Cấu hình	Trước căn chỉnh	Sau căn chỉnh
PaDiM, 4 ảnh	0,493	0,758
PaDiM, 8 ảnh	0,487	0,809
PatchCore, 8 ảnh	0,497	0,729
PatchCore, 16 ảnh	0,634	0,824
PaDiM, toàn bộ tập huấn luyện	0,722	0,835
PatchCore, toàn bộ tập huấn luyện	0,955	0,915



Hình 3.12. Image-AUROC trước và sau khi căn chỉnh góc trên screw

image-AUROC dưới 0,5; đây là lời giải thích cho Autoencoder với 1 ảnh chỉ đạt 0,305 trên screw. Ngược lại, một mô hình cũng có thể đạt điểm cao trên screw mà không thực sự thấy lỗi; hiện tượng này xuất hiện lại ở Thí nghiệm 3 và 4.

Bảng 3.15. Độ sáng trung bình (thang 0–255) của các tập ảnh screw

Tập ảnh	Toàn ảnh	Vùng nền
`train/good`	187,5	208,2
`test/good`	170,4	188,0
`test`, các kiểu lỗi	184,1–188,6	203,7–207,9

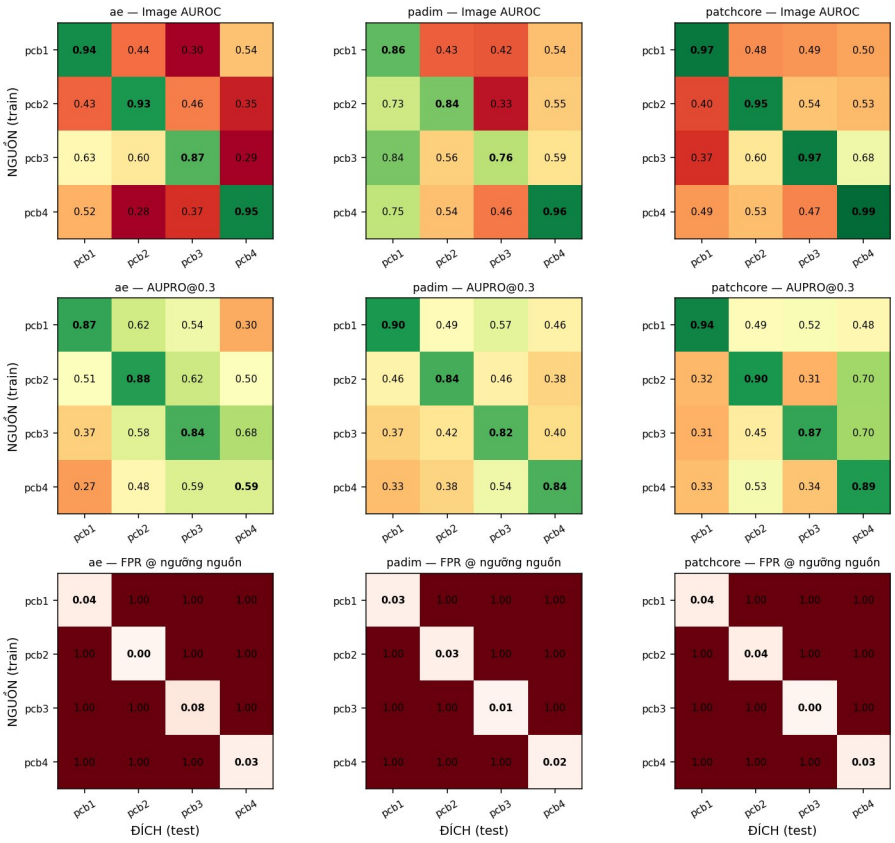
Thất bại trên screw khi có ít ảnh vì vậy không phải do thiếu ảnh nói chung, mà do thiếu độ bao phủ góc xoay (PatchCore, PaDiM) và do độ sáng lệch giữa tập huấn luyện và tập test tốt (Autoencoder). Căn chỉnh góc là một bước tiền xử lý rẻ và hiệu quả khi có ít ảnh. Bài học về cách làm là luôn kiểm tra các thống kê đơn giản như độ sáng hay góc trước khi giải thích kết quả của mô hình sâu.

3.5.4. Thí nghiệm 3: dùng mô hình cho loại sản phẩm khác

Một nhà máy có thể có nhiều mã sản phẩm gần giống nhau, ví dụ nhiều loại bảng mạch. Câu hỏi là có dùng được mô hình của sản phẩm A cho sản phẩm B không. Từ cơ chế, PaDiM giữ phân phối theo từng vị trí của sản phẩm nguồn nên bố cục khác là sai toàn bộ; PatchCore bám sát patch của nguồn nên patch của sản phẩm mới không có láng giềng gần; Autoencoder học cấu trúc chung như cạnh, bề mặt nên có thể linh hoạt hơn, nhưng không có cơ chế so khớp cục bộ.

Nhóm xây dựng mô hình trên một loại nguồn rồi đánh giá trên tập test của loại đích, thành ma trận 4×4 cho pcb1–pcb4 của VisA và 3×3 cho grid, screw, transistor của MVTec AD; đường chéo là trường hợp nguồn trùng đích. Để đo khả năng thích nghi, từ nguồn pcb1, nhóm dùng thêm 4 hoặc 16 ảnh tốt của loại đích theo hai cách: chỉ dùng ảnh đích, hoặc dùng ảnh nguồn cộng ảnh đích.

Ngoài đường chéo, cả ba phương pháp rơi về khoảng 0,43 đến 0,61, tương đương đoán ngẫu nhiên (Bảng 3.16, Hình 3.13 và 3.14). Tại ngưỡng lấy từ nguồn, mọi cặp ngoài đường chéo có FPR và TPR đều bằng 100%: mô hình coi toàn bộ sản phẩm mới là bất thường. Đây là hành vi



Hình 3.13. Ma trận image-AUROC nguồn và đích trên VisA pcb1–pcb4 cho ba phương pháp (hàng là nguồn, cột là đích)

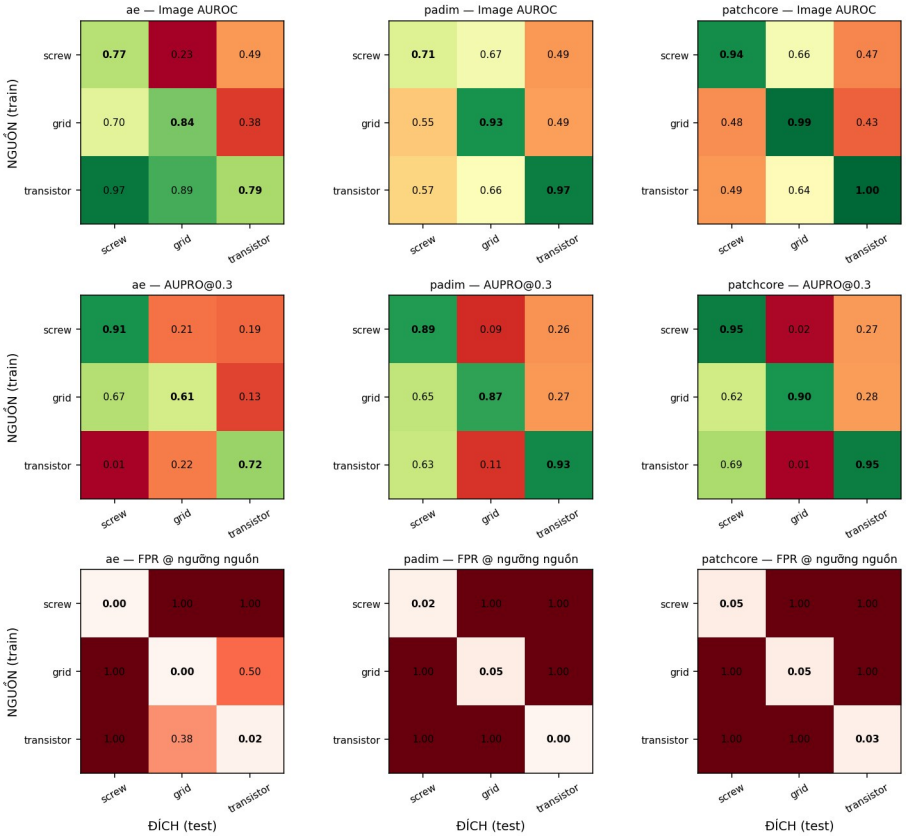
Bảng 3.16. Image-AUROC trung bình trên và ngoài đường chéo của ma trận nguồn và đích

Bộ dữ liệu	Vị trí	Autoencoder	PaDiM	PatchCore
VisA	Đường chéo	0,925	0,857	0,972
	Ngoài đường chéo	0,434	0,560	0,505
MVTec AD	Đường chéo	0,796	0,870	0,977
	Ngoài đường chéo	0,610	0,570	0,527

đúng của một bộ phát hiện bất thường, nhưng cũng có nghĩa là không dùng lại được mô hình. Sự "gần" về loại sản phẩm không giúp gì: dù cùng là bảng mạch, pcb1 sang pcb2 chỉ đạt 0,43–0,48. Với các phương pháp dựa trên đặc trưng cục bộ, "gần" phải là gần về bố cục điểm ảnh và kết cấu. Một vài cặp có kết quả cao bất thường, như PaDiM từ pcb3 sang pcb1 (0,837) và từ pcb2 sang pcb1 (0,733), nhưng không đối xứng, nên nhiều khả năng là trùng hợp về độ tương phản.

Autoencoder có giá trị ngoài đường chéo cao nhất trên MVTec AD (0,610), có vẻ khớp với dự đoán "linh hoạt hơn". Nhưng con số này bị kéo lên bởi một ô duy nhất: Autoencoder từ transistor sang screw đạt 0,975. Đây không phải khả năng tổng quát thật mà là lối tắt độ sáng ở Mục 3.5.3.1: ảnh screw lỗi sáng hơn ảnh screw tốt, và một Autoencoder học trên transistor dựng kém vùng nền sáng. Trên VisA, nơi không có lối tắt này, Autoencoder lại kém nhất (0,434).

Khả năng thích nghi khác nhau rõ rệt (Bảng 3.17, Hình 3.15). PatchCore thích nghi nhanh



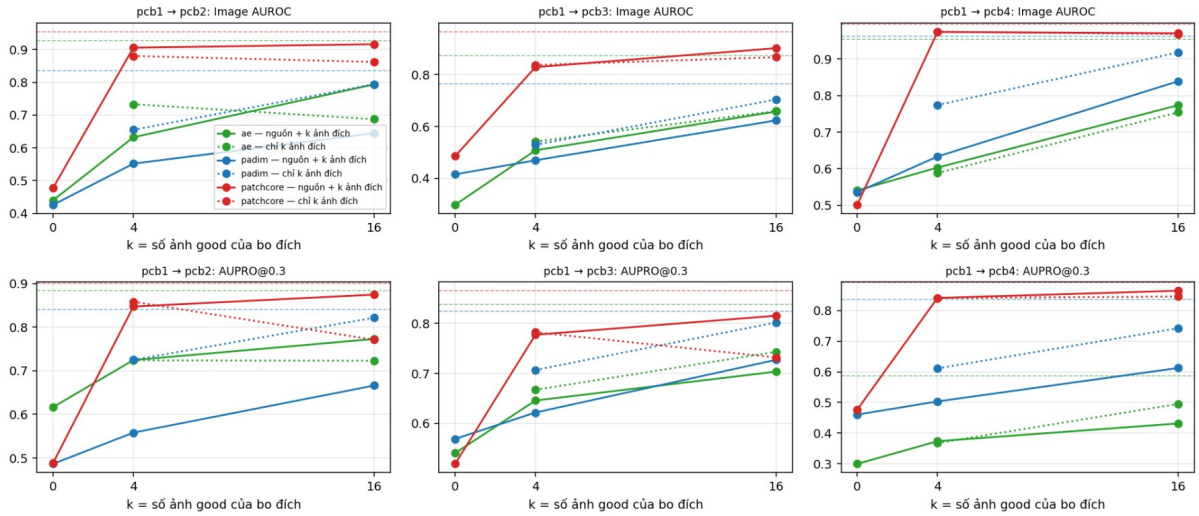
Hình 3.14. Ma trận image-AUROC nguồn và đích trên grid, screw, transistor

Bảng 3.17. Image-AUROC khi thích nghi từ nguồn pcb1 sang các loại đích

		Chỉ ảnh đích		Nguồn và ảnh đích		
Đích	Phương pháp	4 ảnh	16 ảnh	4 ảnh	16 ảnh	Đích đầy đủ
pcb2	PatchCore	0,88	0,86	0,91	0,92	0,96
pcb3	PatchCore	0,84	0,87	0,83	0,90	0,97
pcb4	PatchCore	0,98	0,97	0,97	0,97	0,99
pcb2	PaDiM	0,66	0,79	0,55	0,65	–
pcb3	PaDiM	0,53	0,70	0,47	0,62	–
pcb4	PaDiM	0,77	0,92	0,63	0,84	–
pcb2	Autoencoder	0,73	0,69	0,63	0,79	–
pcb3	Autoencoder	0,54	0,66	0,51	0,66	–
pcb4	Autoencoder	0,59	0,75	0,60	0,77	–

nhất: chỉ 4 ảnh đích đã đưa image-AUROC từ khoảng 0,5 lên 0,84–0,98, cách mô hình xây dựng đầy đủ trên loại đích 1 đến 13 điểm. Giữ lại bộ nhớ của nguồn không gây hại, vì patch nguồn chỉ là láng giềng thừa, và còn giúp nhẹ với pcb2, pcb3 khi có 16 ảnh. PaDiM bị hại khi trộn nguồn: dùng cả nguồn thấp hơn chỉ dùng ảnh đích ở mọi ô, ví dụ pcb4 với 16 ảnh là 0,84 so với 0,92, vì phân phối theo vị trí phải trộn hai bố cục khác nhau. Autoencoder nằm giữa và không hưởng lợi rõ từ nguồn. Khi đặt lại ngưỡng bằng ảnh tốt của loại đích, FPR quay về 0–5%.

Không phương pháp nào tổng quát được sang loại sản phẩm khác, kể cả loại "gần" nhau.



Hình 3.15. Image-AUROC khi thích nghi từ pcb1 sang pcb2, pcb3, pcb4 với 4 và 16 ảnh đích: chỉ ảnh đích so với nguồn cộng ảnh đích

Khi đổi sản phẩm, bắt buộc phải đặt lại ngưỡng; nếu dùng PatchCore thì chỉ cần thêm vài ảnh của sản phẩm mới vào bộ nhớ. Một kết quả cao bất thường như Autoencoder từ transistor sang screw cần được nghi ngờ và kiểm tra lối tắt trong dữ liệu trước khi kết luận.

3.5.5. Thí nghiệm 4: thay đổi điều kiện chụp

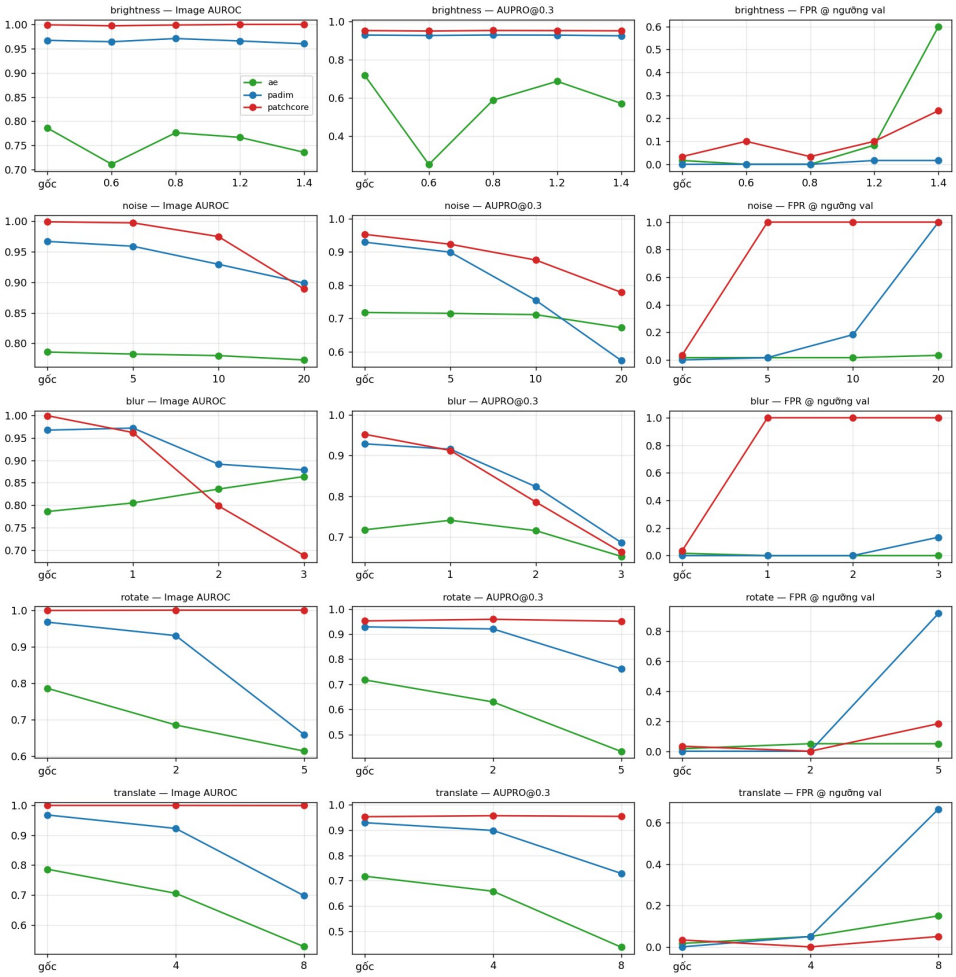
Sau khi triển khai, đèn có thể yếu đi, camera mất nét, cảm biến nhiễu hơn hoặc sản phẩm bị lệch vị trí, trong khi mô hình không được huấn luyện lại và ngưỡng đã chốt. Nhóm đặt ba giả thuyết từ cơ chế: H1, PatchCore nhạy với ánh sáng vì so khớp tuyệt đối với patch đã ghi nhớ; H2, PaDiM nhạy với nhiễu vì nhiễu đẩy đặc trưng ra ngoài phân phối Gauss hẹp tại từng vị trí; H3, Autoencoder nhạy với dịch và xoay vì lớp kết nối đầy đủ ở cổ chai gắn với vị trí tuyệt đối. Tiêu chí chính là FPR trên ảnh tốt tại ngưỡng cố định và mức thay đổi AUPRO, vì AUROC không phụ thuộc ngưỡng.

Có hai loại thay đổi. Thay đổi tạo nhân tạo áp lên tập test của grid, screw, transistor sau khi mô hình và ngưỡng đã được xây dựng trên ảnh gốc: làm mờ (σ = 1, 2, 3), đổi độ sáng (nhân 0,6 đến 1,4), thêm nhiễu Gauss (σ = 5 đến 20), xoay (tới 20°, riêng transistor tới 5°) và dịch (tới 32 điểm ảnh, riêng transistor tới 8 điểm ảnh). Xoay và dịch được thực hiện ở độ phân giải gốc; vùng biên bị đệm (32 điểm ảnh) được loại khỏi đánh giá, cùng những ảnh có lỗi bị đẩy ra ngoài khung. Thay đổi thật lấy từ tập `test_public` của MVTec AD 2, chụp ở các điều kiện bình thường, quá sáng, quá tối và bốn mức dịch ánh sáng.

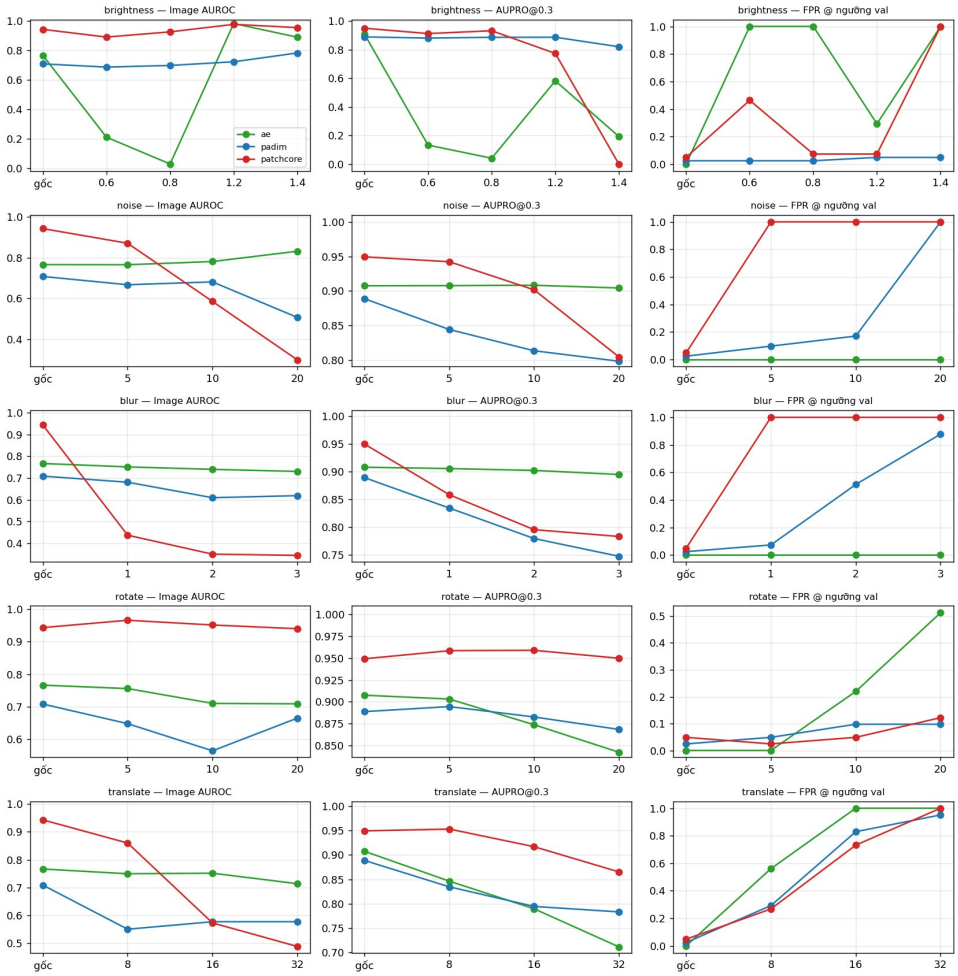
Bảng 3.18 và Hình 3.16, 3.17 cho kết quả ở mức thay đổi mạnh nhất. Dòng dịch trên screw và grid có thể còn bị ảnh hưởng bởi vùng biên nên không dùng để kết luận. Giả thuyết H1 bị bác bỏ với ánh sáng tạo nhân tạo: giảm độ sáng xuống 0,6 lần hầu như không đổi AUPRO của PatchCore (từ $-0{,}04$ đến 0), vì đặc trưng từ backbone ImageNet khá bất biến với thay đổi độ sáng toàn ảnh; dù vậy FPR vẫn tăng (46% trên screw), và với ánh sáng thật thì giả thuyết đúng một

Bảng 3.18. Thay đổi AUPRO và FPR tại ngưỡng cố định ở mức thay đổi mạnh nhất (thứ tự trong mỗi ô: Autoencoder / PaDiM / PatchCore; FPR gốc của cả ba khoảng 0–5%)

Thay đổi	Loại	Thay đổi AUPRO	FPR
Làm mờ $\sigma = 3$	grid	$-0{,}03$ / $-0{,}25$ / $-0{,}74$	0% / 5% / 100%
	screw	$-0{,}01$ / $-0{,}14$ / $-0{,}17$	0% / 88% / 100%
	transistor	$-0{,}07$ / $-0{,}24$ / $-0{,}29$	0% / 13% / 100%
Độ sáng $\times 0{,}6$	grid	$-0{,}10$ / $+0{,}01$ / $-0{,}02$	0% / 0% / 24%
	screw	$-0{,}77$ / $-0{,}01$ / $-0{,}04$	100% / 2% / 46%
	transistor	$-0{,}47$ / 0 / 0	0% / 0% / 10%
Nhiễu $\sigma = 20$	grid	0 / $-0{,}46$ / $-0{,}48$	0% / 100% / 100%
	screw	0 / $-0{,}09$ / $-0{,}14$	0% / 100% / 100%
	transistor	$-0{,}05$ / $-0{,}36$ / $-0{,}17$	3% / 100% / 100%
Xoay 20° (transistor 5°)	grid	$+0{,}18$ / $+0{,}06$ / $+0{,}05$	0% / 0% / 38%
	screw	$-0{,}07$ / $-0{,}02$ / 0	51% / 10% / 12%
	transistor	$-0{,}29$ / $-0{,}17$ / 0	5% / 92% / 18%
Dịch 32 điểm ảnh (transistor 8)	grid	$+0{,}18$ / $+0{,}03$ / $-0{,}02$	0% / 0% / 100%
	screw	$-0{,}20$ / $-0{,}11$ / $-0{,}08$	100% / 95% / 100%
	transistor	$-0{,}28$ / $-0{,}20$ / 0	15% / 67% / 5%



Hình 3.16. Image-AUROC, AUPRO và FPR theo cường độ từng loại thay đổi trên transistor



Hình 3.17. Image-AUROC, AUPRO và FPR theo cường độ từng loại thay đổi trên screw

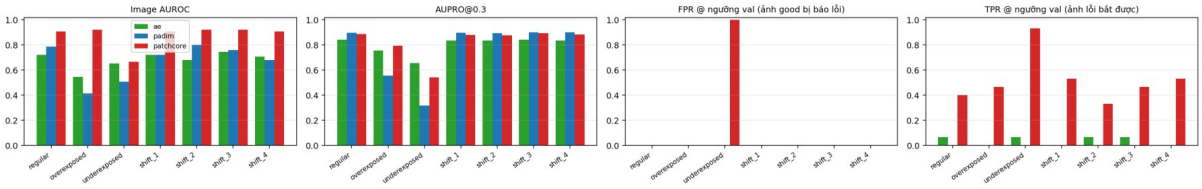
phần. Giả thuyết H2 đúng nhưng không riêng PaDiM: nhiễu $\sigma = 20$ đẩy FPR của PaDiM lên 100% ở cả ba loại, và PatchCore bị nặng tương đương, vì hai phương pháp dùng chung backbone nên cùng nhạy với nhiễu tần số cao. Giả thuyết H3 đúng trên transistor: xoay 5° làm AUPRO của Autoencoder giảm 0,29, dịch 8 điểm ảnh giảm 0,28, trong khi PatchCore không đổi. PaDiM cũng rất nhạy (FPR 92% khi xoay 5°, 67% khi dịch 8 điểm ảnh), đúng với cơ chế phân phối theo vị trí dù giả thuyết ban đầu không nêu.

Ba phát hiện nằm ngoài giả thuyết. Thứ nhất, Autoencoder sụp đổ với độ sáng: chỉ giảm độ sáng còn 0,8 lần trên screw đã làm image-AUROC giảm 0,74 và FPR thành 100%, vì mạng tối ưu MSE theo cường độ điểm ảnh nên mọi thay đổi cường độ đều thành sai số tái tạo; đây cũng là cơ chế của lối tắt độ sáng ở Mục 3.5.3.1. Thứ hai, PatchCore rất nhạy với làm mờ: chỉ $\sigma = 1$ trên screw đã làm image-AUROC giảm 0,51 và FPR thành 100%, còn trên grid AUPRO giảm 0,74, vì PatchCore dựa vào chi tiết kết cấu tần số cao, đúng thứ mà làm mờ xóa đi; ngược lại, Autoencoder gần như không bị ảnh hưởng bởi làm mờ và nhiễu vì bản thân nó đã dựng ra ảnh mờ. Thứ ba, AUROC che giấu thất bại: trên transistor, nhiễu $\sigma = 5$ làm image-AUROC của PatchCore gần như không đổi nhưng FPR tăng từ 3% lên 100%. Mô hình vẫn xếp hạng đúng nhưng mọi điểm bị đẩy vượt ngưỡng; trên dây chuyền, mọi sản phẩm sẽ bị loại.

Hai mức hỏng không đi cùng nhau. Nhiễu và làm mờ chủ yếu gây báo nhầm hàng loạt, với bản đồ sáng đều trên toàn ảnh. Độ sáng với Autoencoder và xoay, dịch với PaDiM gây bản đồ sai chỗ: AUPRO giảm mạnh vì vùng sáng nằm ở biên vật thể thay vì ở vùng lỗi.

Bảng 3.19. Kết quả trên vial theo điều kiện chụp (thứ tự trong mỗi ô: Autoencoder / PaDiM / PatchCore)

Điều kiện	Image-AUROC	AUPRO
Bình thường	0,72 / 0,787 / 0,907	0,841 / 0,898 / 0,887
Quá sáng	0,547 / 0,413 / 0,920	0,755 / 0,555 / 0,793
Quá tối	0,653 / 0,507 / 0,667	0,657 / 0,318 / 0,542
Dịch ánh sáng 1–4	0,68–0,75 / 0,68–0,80 / 0,907–0,92	–

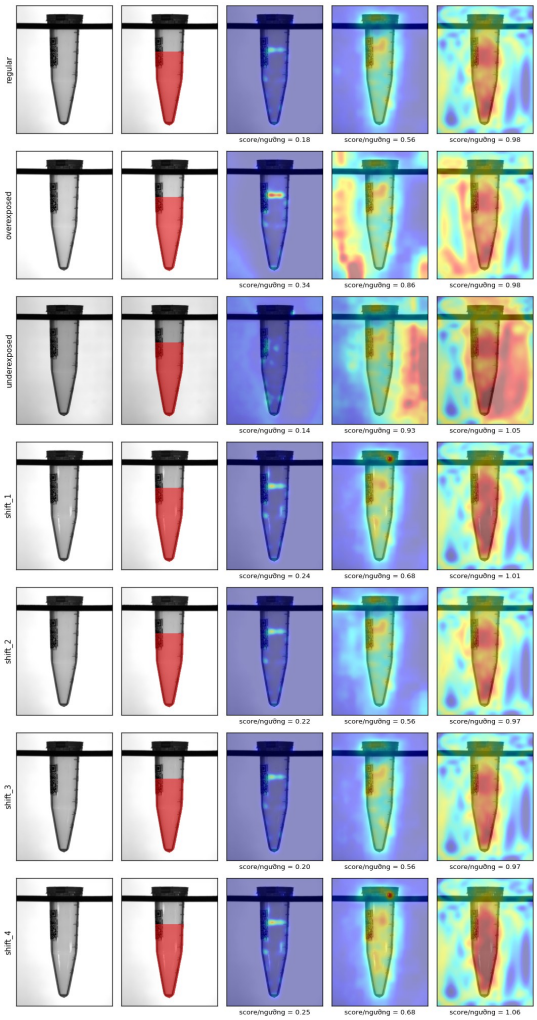


Hình 3.18. Image-AUROC và AUPRO của ba phương pháp trên vial theo từng điều kiện chụp

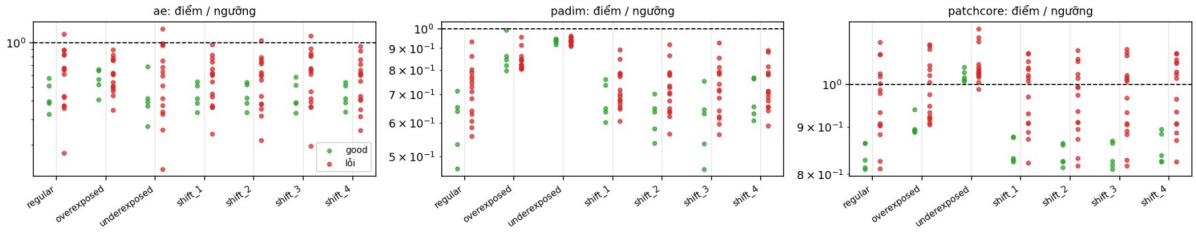
Với ánh sáng thật trên vial (Bảng 3.19, Hình 3.18 đến 3.20), PaDiM sụp đổ: image-AUROC 0,413 khi quá sáng và 0,507 khi quá tối, tức bằng hoặc dưới mức ngẫu nhiên, còn AUPRO giảm từ 0,898 xuống 0,318. Khác với ánh sáng tạo nhân tạo vốn nhân đều cả ảnh, ánh sáng thật thay đổi phản xạ và độ bão hòa cục bộ trên thủy tinh, nên làm lệch phân phối tại những vị trí cụ thể. PatchCore bền với ảnh quá sáng và các mức dịch ánh sáng (0,907–0,92) nhưng yếu khi ảnh quá tối, với image-AUROC 0,667 và FPR 100% tại ngưỡng cố định, khớp với Bảng 3.10. Đây là phần đúng của H1: PatchCore bền với thay đổi độ sáng tuyến tính nhưng không bền khi thiếu sáng làm mất chi tiết kết cấu, tương tự làm mờ. Autoencoder giảm vừa phải và đều, và giữ AUPRO tốt nhất khi ảnh quá tối (0,657).

Sheet_metal cho thấy một kiểu thất bại khác (Hình 3.21). Image-AUROC của PaDiM và PatchCore khá ổn định qua các điều kiện (0,70–0,85), của Autoencoder là 0,50–0,68, nhưng khả năng định vị rất kém ở mọi điều kiện: AUPRO khoảng 0,39 với PaDiM, 0,45 với PatchCore và 0,25 với Autoencoder. Ảnh sheet_metal rất dài với lỗi nhỏ và mảnh, nên vấn đề chính không phải thay đổi điều kiện chụp mà là đặc trưng quá thô so với kích thước lỗi. Vial hỏng theo điều kiện, tức ánh sáng thay đổi thì báo nhầm hàng loạt; sheet_metal hỏng ở mọi điều kiện, tức bỏ sót lỗi nhỏ và bản đồ không trùng lỗi. Hai loại cần hai cách khắc phục khác nhau: chuẩn hóa ánh sáng hoặc tăng cường dữ liệu cho vial, cắt ảnh thành nhiều mảnh hoặc dùng đặc trưng độ phân giải cao cho sheet_metal.

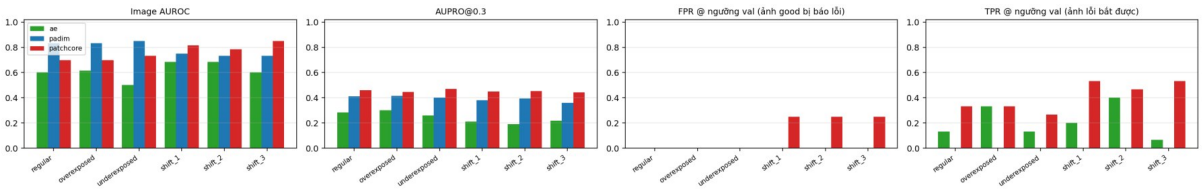
Không phương pháp nào bền với mọi loại thay đổi (Bảng 3.20); lựa chọn nên dựa vào loại biến động hay gặp nhất ở dây chuyền. Quan trọng hơn, cần theo dõi FPR trên ảnh tốt theo thời gian, vì AUROC có thể không đổi trong khi hệ thống đã báo lỗi mọi sản phẩm.



Hình 3.19. Cùng một cảnh vial ở điều kiện bình thường, quá sáng và quá tối, cùng bản đồ bất thường của ba phương pháp



Hình 3.20. Phân phối điểm bất thường của ảnh tốt và ảnh lỗi trên vial theo điều kiện ánh sáng, so với ngưỡng cố định



Hình 3.21. Image-AUROC và AUPRO trên sheet_metal theo từng điều kiện chụp

Bảng 3.20. Độ nhạy của ba phương pháp với thay đổi điều kiện chụp

Phương pháp	Nhạy nhất với	Bền với	Giải thích
Autoencoder	Độ sáng; dịch, xoay (transistor)	Làm mờ, nhiễu	MSE trên cường độ điểm ảnh; cổ chai gắn vị trí
PaDiM	Nhiễu; ánh sáng thật; dịch, xoay	Độ sáng tạo nhân tạo	Phân phối Gauss hẹp, theo từng vị trí
PatchCore	Làm mờ, nhiễu, ảnh quá tối	Độ sáng tạo nhân tạo; dịch, xoay (theo AUPRO; FPR vẫn tăng)	Dựa vào chi tiết kết cấu; so khớp không theo vị trí

3.5.6. Thí nghiệm 5: phân tích theo kiểu lỗi và các ca thất bại

Các con số trung bình che giấu kiểu thất bại. Từ cơ chế, nhóm dự đoán Autoencoder thất bại khi tái tạo quá tốt (dựng lại cả lỗi nên bỏ sót) hoặc quá kém (sai số cao ở vùng chi tiết hay vùng sáng bình thường nên báo nhầm); PaDiM thất bại khi giả định phân phối Gauss theo vị trí bị vi phạm; PatchCore thất bại khi đặc trưng quá thô so với lỗi, hoặc khi một patch tốt hiếm gặp không có láng giềng trong bộ nhớ. Bảng 3.21 và 3.22 cho image-AUROC của từng kiểu lỗi, tính trên ảnh lỗi kiểu đó so với toàn bộ ảnh tốt.

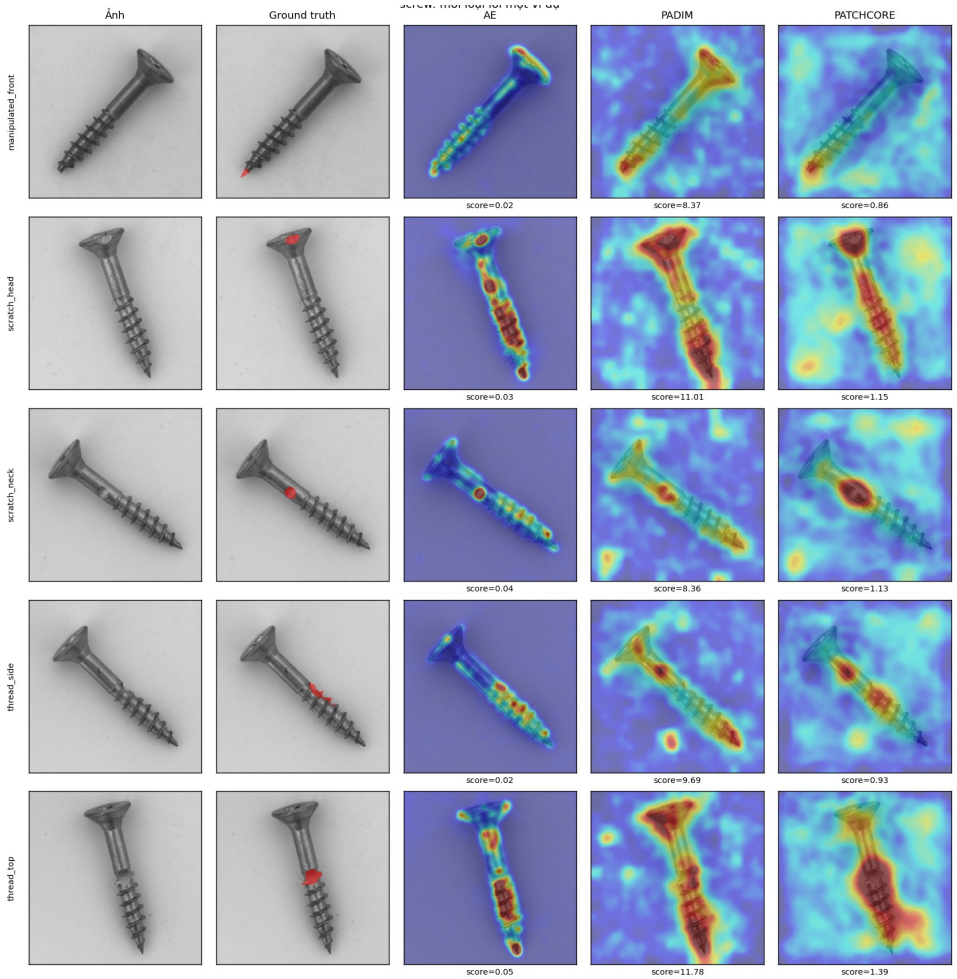
Bảng 3.21. Image-AUROC theo từng kiểu lỗi của screw

Nhóm lỗi	Kiểu lỗi	Autoencoder	PaDiM	PatchCore
Lỗi ren	thread_side	0,643	0,631	0,917
	thread_top	0,854	0,687	0,995
Lỗi đầu	scratch_head	0,787	0,737	0,947
Biến dạng	manipulated_front	0,841	0,772	0,928
Lỗi thân, cổ	scratch_neck	0,806	0,778	0,987

Lỗi ren bên thân (thread_side) khó nhất với cả ba (Bảng 3.21, Hình 3.22). Ren vít là cấu trúc lặp lại tần số cao, nên một đoạn ren hỏng vẫn trông giống ren ở mức đặc trưng thô. Autoencoder (0,643) dựng ren thành một dải mờ, nên sai số ở ren tốt và ren hỏng như nhau; đây là kiểu "tái tạo quá kém ở mọi nơi". PaDiM (0,631) thất bại vì ren nằm ở vị trí khác nhau tùy góc xoay, và PaDiM yếu đều ở mọi nhóm lỗi của screw (0,63–0,78), tức thất bại do giả định căn chỉnh chứ không phải do kiểu lỗi. PatchCore đạt từ 0,917 trở lên ở mọi nhóm, yếu nhất cũng ở thread_side, khớp với dự đoán đặc trưng quá thô với lỗi nhỏ lặp lại.

Trên transistor (Bảng 3.22, Hình 3.23), lỗi misplaced, tức thiếu hoặc lệch cả linh kiện, là kiểu duy nhất Autoencoder làm tốt (0,978), vì lỗi lớn và thay đổi cấu trúc toàn cục. Lỗi damaged_case, vết xước trên vỏ đen, gần như vô hình với Autoencoder (0,542): vết xước tối trên nền tối cho sai số MSE rất nhỏ, đúng kiểu "tái tạo quá tốt".

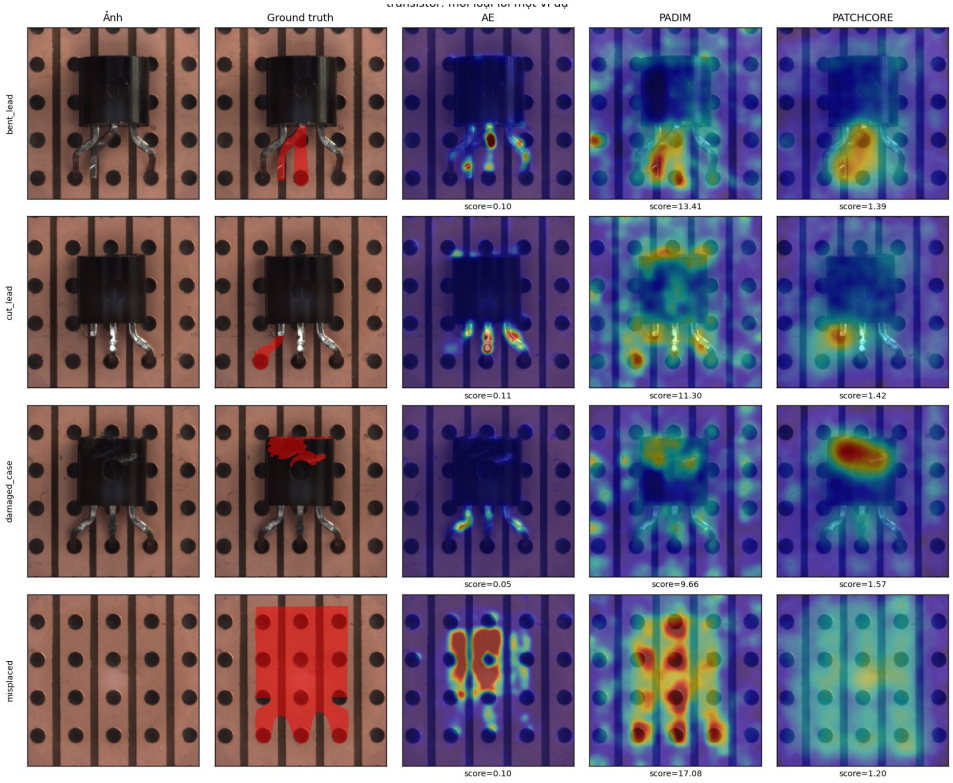
Mỗi phương pháp để lại một dấu hiệu nhận biết trên bản đồ. Các ảnh screw tốt bị PaDiM chấm cao nhất đều là vít nằm ở góc ít gặp, và bản đồ không sáng tại một điểm mà phủ dọc cả thân



Hình 3.22. Ví dụ từng kiểu lỗi của screw: ảnh gốc, mặt nạ lỗi và bản đồ của Autoencoder, PaDiM, PatchCore

Bảng 3.22. Image-AUROC theo từng kiểu lỗi của transistor và grid

Loại	Kiểu lỗi	Autoencoder	PaDiM	PatchCore
transistor	bent_lead	0,802	1,000	0,998
	cut_lead	0,692	0,978	1,000
	damaged_case	0,542	0,895	1,000
	misplaced	0,978	1,000	0,997
grid	bent	0,877	0,972	1,000
	broken	0,758	0,996	1,000
	glue	0,706	0,870	0,961
	metal_contamination	0,922	0,996	0,996
	thread	0,866	0,827	0,987



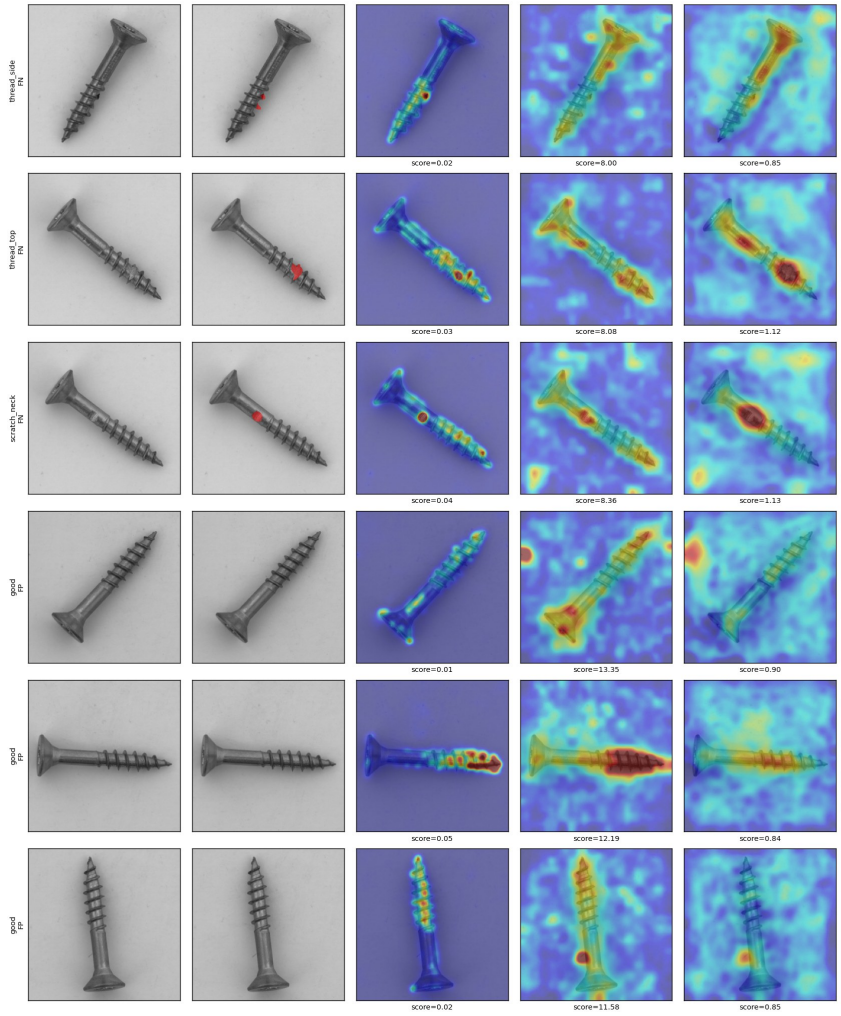
Hình 3.23. Ví dụ từng kiểu lỗi của transistor với bản đồ của ba phương pháp

vít (Hình 3.24): mọi vị trí mà thân vít đi qua đều chỉ được học là nền trắng. Với Autoencoder trên grid, những lỗi nhỏ chỉ cho điểm khoảng 0,03, trong khi một ảnh tốt có lưới sáng hơn lại đạt 0,11 (Hình 3.25); trên transistor, Autoencoder làm sáng các chân kim loại bóng ngay cả trên ảnh tốt. Cả hai trường hợp là cùng một cơ chế: sai số tái tạo tỷ lệ với độ sáng và độ chi tiết của vùng ảnh, không tỷ lệ với mức bất thường. Với transistor thiếu linh kiện, PatchCore vẫn xếp ảnh là bất thường (0,997) nhưng bản đồ yếu và lan rộng, vì vùng thiếu là nền trơn và patch nền trơn có láng giềng trong bộ nhớ (nền ở các vị trí khác). PaDiM và Autoencoder gắn với vị trí nên định vị vùng thiếu rõ hơn. Đây là mặt trái của việc PatchCore không dùng thông tin vị trí, cũng chính là lý do nó bền với dịch và xoay ở Thí nghiệm 4.

Các giải thích trên dựa trên bản đồ và AUROC theo kiểu lỗi. Để khẳng định chắc chắn, còn cần ba kiểm tra trực tiếp mà đề tài chưa thực hiện: hiển thị ảnh dựng lại của Autoencoder để phân biệt "tái tạo quá tốt" với "tái tạo quá kém"; vẽ phân phối khoảng cách Mahalanobis của patch tốt tại những vị trí PaDiM báo nhầm, để biết patch đó nằm ở đuôi phân phối hay phân phối tại vị trí đó có nhiều đỉnh; và truy ngược patch gần nhất trong bộ nhớ của PatchCore cho mỗi patch bị bỏ sót hoặc báo nhầm, để xác nhận vùng thiếu tìm được láng giềng là nền trơn.

3.5.7. Thảo luận tổng hợp

Bảng 3.23 tổng hợp các thí nghiệm. Thứ hạng trên dữ liệu chuẩn, PatchCore rồi PaDiM rồi Autoencoder, không còn đúng khi điều kiện thay đổi, nhưng cách mỗi phương pháp thất bại phần lớn đoán trước được từ cơ chế. Bốn quan sát xuyên suốt các thí nghiệm.



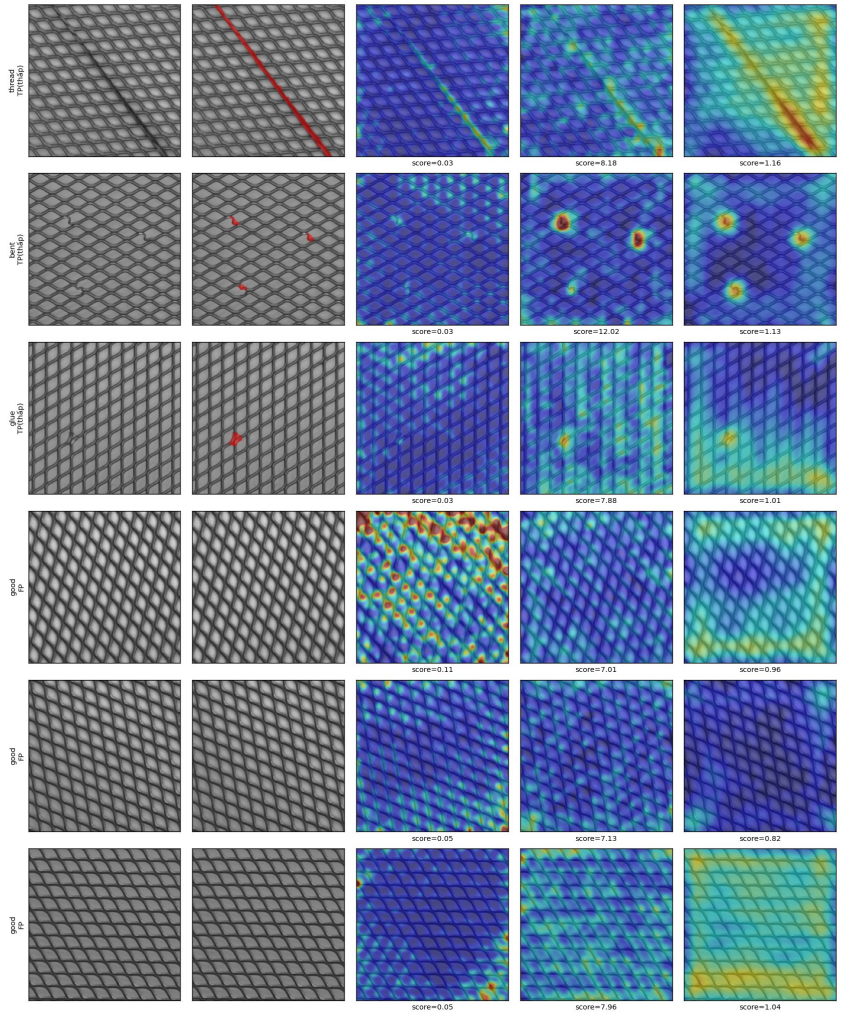
Hình 3.24. Những ảnh screw tốt có điểm PaDiM cao nhất (báo nhầm): bản đồ sáng dọc cả thân vít

Thông tin vị trí là con dao hai lưỡi. PaDiM và Autoencoder gắn với vị trí nên định vị tốt lỗi thiếu linh kiện, nhưng thất bại khi vật thể xoay hay lệch. PatchCore bỏ qua vị trí nên bền với dịch và xoay, nhưng không biết một vùng nền trơn lẽ ra phải có linh kiện.

Phát hiện và định vị hỏng theo những cách khác nhau. Dữ liệu bẩn làm hỏng phát hiện nhiều hơn định vị, vì điểm ảnh là giá trị lớn nhất của bản đồ và rất nhạy với vài patch sai; với ít ảnh, điều này đúng trên screw nhưng ngược lại trên grid, nơi định vị hỏng trước. Thay đổi vị trí cũng làm hỏng định vị trước. Báo cáo một chỉ số duy nhất sẽ bỏ lỡ một trong hai.

AUROC không đủ để quyết định triển khai. Image-AUROC đo khả năng xếp hạng, còn FPR tại ngưỡng cố định đo điều nhà máy thực sự gặp. Các trường hợp AUROC giữ nguyên mà FPR lên 100% ở Thí nghiệm 3 và 4 cho thấy việc đặt lại ngưỡng quan trọng không kém việc chọn mô hình.

Lối tắt trong dữ liệu có thật và ảnh hưởng tới kết luận. Độ lệch độ sáng giữa ảnh test tốt của screw và các tập khác đủ để chỉ riêng độ sáng trung bình đạt AUROC 1,0 trên tập kiểm tra. Nó giải thích ít nhất ba kết quả bất thường: Autoencoder với ít ảnh dưới 0,5, Autoencoder từ



Hình 3.25. Các ca thất bại của Autoencoder trên grid: bỏ sót lỗi nhỏ và báo nhầm ở vùng sáng

transistor sang screw đạt 0,975, và độ nhạy độ sáng cực đoan của Autoencoder trên screw.

3.6. Chi phí tính toán

Autoencoder nhanh nhất vì chỉ chạy một mạng nhỏ. PaDiM và PatchCore đều phải chạy Wide-ResNet-50-2 để trích đặc trưng; riêng bước này mất khoảng 6–8 ms mỗi ảnh. PatchCore chậm nhất với MVTec AD vì coreset 10% có 18–28 nghìn vector, mỗi patch kiểm thử phải so với tất cả; với VisA, coreset 1% chỉ khoảng 9,8 nghìn vector nên PatchCore còn nhanh hơn trên ảnh lớn hơn. Ngay cả trường hợp chậm nhất, sheet_metal với PatchCore, vẫn dưới 0,1 giây mỗi ảnh trên GPU máy tính xách tay.

Dung lượng thì ngược lại: PaDiM lớn nhất vì lưu một ma trận 100×100 cho mỗi vị trí, nên tăng theo kích thước ảnh, tới 631 MB với sheet_metal. Coreset của PatchCore chỉ 4–54 MB. Hai con số này chưa tính backbone dùng chung (Wide-ResNet-50-2, khoảng 132 MB). Trên Colab, ở 6 loại có ghi lại thời gian, cả ba phương pháp của một loại, gồm xây dựng mô hình, đặt ngưỡng và đánh giá, mất từ 84 giây (vial) đến 215 giây (sheet_metal).

Bảng 3.23. Điểm mạnh và điểm yếu của ba phương pháp theo từng tình huống

Tình huống	Autoencoder	PaDiM	PatchCore	Mục
Dữ liệu chuẩn, đủ ảnh	Trung bình	Khá	Tốt nhất	3.4
Dữ liệu huấn luyện lẫn lỗi	Dao động mạnh	Bền nhất	Giảm đều, vẫn cao nhất	3.5.2
Ít ảnh, vật thể cố định (transistor)	Kém	Khá từ 16 ảnh	Tốt từ 4 ảnh	3.5.3
Ít ảnh, vật thể xoay tự do	Thất bại	Thất bại	Thất bại, cần căn chỉnh	3.5.3.1
Chuyển sang sản phẩm khác	Thất bại	Thất bại, bị hại khi trộn nguồn	Thất bại, thích nghi nhanh	3.5.4
Độ sáng thay đổi	Rất nhạy	Bền (nhân tạo), nhạy (thật)	Bền, trừ ảnh quá tối	3.5.5
Làm mờ, nhiễu	Bền	Nhạy	Nhạy	3.5.5
Dịch, xoay nhỏ	Nhạy	Nhạy	Bền theo AUPRO, FPR tăng	3.5.5
Lỗi tối trên nền tối, lỗi ren nhỏ	Bỏ sót	Trung bình	Tốt	3.5.6
Vết xước mảnh trên ảnh dài (sheet_metal)	Kém	Kém	Kém	3.5.5
Định vị lỗi thiếu linh kiện	Tốt	Tốt	Bản đồ lan rộng	3.5.6

Bảng 3.24. Thời gian suy luận một ảnh trên RTX 3050 Laptop và dung lượng mô hình mỗi loại

	Autoencoder	PaDiM	PatchCore
Suy luận, ảnh 256×256 (ms/ảnh)	3,9–4,2	13–16	32–40
Suy luận, ảnh 256×320, VisA (ms/ảnh)	4,3–4,9	16–17	25–26
Suy luận, sheet_metal 256×1024 (ms/ảnh)	10,5	37	54
Dung lượng mô hình mỗi loại (MB)	45	118–631	4–54

3.7. So sánh với kết quả đã công bố

PatchCore với coreset 10% được báo cáo đạt image-AUROC 0,990 và pixel-AUROC 0,981 trung bình trên 15 loại MVTec AD [3]; trên ba loại MVTec AD của đề tài, nhóm đạt 0,977 và 0,984. Ba loại này gồm screw, loại khó nhất của MVTec AD với hầu hết phương pháp, nên trung bình thấp hơn là hợp lý. Ngoài ra, cách làm của nhóm khác bài gốc ở hai điểm: không cắt vùng giữa ảnh và không dùng bước hiệu chỉnh trọng số khi tính điểm ảnh.

PaDiM được báo cáo đạt pixel-AUROC 0,975 với Wide-ResNet-50-2 trên MVTec AD [2]; nhóm đạt 0,962 trên ba loại. Kết quả ở Bảng 3.7 cũng khớp với những lần chạy cùng cấu hình trong các thí nghiệm chuyên sâu, chạy riêng trước đó trên cùng loại GPU: đường chéo của ma trận nguồn và đích ở Thí nghiệm 3 (pcb1–pcb4) và mốc không thay đổi ở Thí nghiệm 4 (screw, transistor) chênh image-AUROC tối đa 0,001 với cả ba phương pháp, cho thấy quy trình lặp lại được. Khi đổi sang GPU khác, Autoencoder dao động hơn vì được huấn luyện từ khởi tạo ngẫu

nhiên (Mục 3.9).

3.8. Ứng dụng minh họa

Nhóm xây dựng một giao diện web bằng Gradio để dùng thử các mô hình đã lưu. Người dùng chọn loại sản phẩm, phương pháp và tải ảnh lên; giao diện trả về bản đồ bất thường chồng lên ảnh (cùng thang màu như các hình ở trên), điểm của ảnh, kết luận lỗi hay không lỗi theo ngưỡng đã lưu, kèm image-AUROC, tỷ lệ báo nhầm và tỷ lệ bắt lỗi của mô hình đó trên tập test. Ứng dụng chỉ suy luận, không huấn luyện lại, và chạy được hoàn toàn trên máy cá nhân. Điểm tính lại trong ứng dụng sai khác điểm lúc đánh giá dưới 0,1%.

3.9. Hạn chế

Kết quả trên 9 loại và các Thí nghiệm 3, 4, 5 chỉ chạy một lần; Thí nghiệm 1 lặp 5 seed và Thí nghiệm 2 lặp 3 seed, nên các khác biệt dưới khoảng 0,02 AUROC không nên coi là có ý nghĩa. PaDiM và PatchCore gần như tất định, còn Autoencoder thì dao động: cùng mã nguồn, grid đạt image-AUROC 0,836 trên T4 và 0,804 trên RTX 3050. Autoencoder cũng chỉ được thử một cấu hình (sai số bình phương, cổ chai 256, 1500 bước); hàm mất mát SSIM và các kích thước cổ chai khác chưa được thử; các biến thể mạnh hơn có thể thay đổi kết luận về độ nhạy với độ sáng và lỗi nhỏ. Autoencoder còn nhận mọi ảnh ở kích thước 256×256, nên ảnh không vuông bị ép tỷ lệ trước khi vào mạng: sheet_metal (256×1024) bị nén 4 lần theo chiều ngang, làm vết xước vốn đã mảnh càng mảnh hơn. Đây có thể là một phần lý do Autoencoder chỉ đạt 0,601 ở loại này, và so sánh với PaDiM, PatchCore trên sheet_metal vì vậy chưa hoàn toàn công bằng. Với PaDiM và PatchCore, ảnh VisA được đưa về cùng 256×320, nên pcb3 (tỷ lệ gốc khoảng 1,63) cũng bị ép khoảng 23% theo chiều ngang.

Kết quả trên 9 loại chưa đại diện cho toàn bộ ba bộ dữ liệu. Tập test của MVTec AD 2 theo từng điều kiện chụp chỉ có 4–5 ảnh tốt, nên FPR theo điều kiện có sai số lớn. Grid chỉ có 57 ảnh lỗi nên Thí nghiệm 1 trên grid dừng ở 10%. Kết quả dịch ảnh trên screw và grid ở Thí nghiệm 4 có thể còn bị ảnh hưởng bởi vùng biên nên không dùng để kết luận, và ba kiểm tra cơ chế ở cuối Mục 3.5.6 chưa được thực hiện, nên một số giải thích vẫn là giả thuyết được bản đồ bất thường ủng hộ chứ chưa được chứng minh trực tiếp.

Ngưỡng được đặt trên ảnh chụp ở điều kiện bình thường, nên khi ánh sáng thay đổi mạnh như ảnh quá tối của vial, ngưỡng không còn đúng. Ngưỡng $\bar{s} + 3\sigma_s$ cũng chặt nên Autoencoder và PaDiM bỏ sót phần lớn ảnh lỗi; ngưỡng khác chưa được so sánh. Cuối cùng, thời gian suy luận được đo trên máy cá nhân chứ không phải trên phần cứng triển khai, và thời gian xây dựng mô hình trên Colab chỉ ghi được tổng theo loại, không tách theo phương pháp.

KẾT LUẬN

Đề tài đã cài đặt và đánh giá ba phương pháp phát hiện và định vị bất thường không giám sát trên 9 loại sản phẩm của MVTec AD, MVTec AD 2 và VisA theo cùng một giao thức. Autoencoder, PaDiM và PatchCore đều do nhóm tự cài đặt; PaDiM và PatchCore dùng chung backbone Wide-ResNet-50-2 nên chênh lệch giữa chúng chỉ do cơ chế. Mỗi mô hình có ngưỡng đặt trên ảnh tốt tách riêng và được đưa vào một ứng dụng minh họa chạy trên máy cá nhân.

Trên dữ liệu chuẩn, PatchCore tốt nhất ở cả ba bộ dữ liệu (image-AUROC trung bình 0,937, pixel-AUROC 0,969, AUPRO 0,848), và là phương pháp duy nhất dùng được tại ngưỡng: bắt được 75% ảnh lỗi với 6% báo nhầm. PaDiM thấp nhất ở screw, vial và pcb1–pcb3; khoảng cách với PatchCore lớn nhất ở screw, loại duy nhất xoay tự do, đúng như dự đoán từ việc nó gắn thống kê với vị trí. Autoencoder thấp nhất trên MVTec AD nhưng tốt hơn PaDiM ở ba trong bốn loại bảng mạch của VisA, nơi lỗi làm đổi màu cả một vùng.

Năm thí nghiệm chuyên sâu cho thấy thứ hạng đó không còn đúng khi điều kiện thay đổi, nhưng cách mỗi phương pháp thất bại phần lớn đoán trước được từ cơ chế của nó. Với tập huấn luyện lẫn 5–20% ảnh lỗi, cả ba đều bền; PatchCore giảm đều, PaDiM bền nhất, Autoencoder dao động theo seed nhiều hơn chính mức suy giảm. Khi chỉ có vài ảnh tốt, PatchCore chỉ cần 4 ảnh với vật thể cố định như transistor, nhưng grid cần hơn 16 ảnh; còn với screw, nguyên nhân thất bại là góc xoay và độ lệch độ sáng trong dữ liệu chứ không phải số ảnh. Không phương pháp nào dùng được cho loại sản phẩm khác, nhưng PatchCore thích nghi được với 4 ảnh của loại mới, còn PaDiM bị hại khi trộn dữ liệu nguồn. Khi điều kiện chụp thay đổi, Autoencoder thua độ sáng, PaDiM thua nhiễu và vị trí, PatchCore thua làm mờ và ảnh quá tối; nhiều trường hợp AUROC gần như không đổi trong khi tỷ lệ báo nhầm lên 100%. Mỗi kiểu thất bại cũng để lại một dấu hiệu nhận biết được trên bản đồ bất thường.

Từ các kết quả này, nhóm khuyến nghị dùng PatchCore làm phương pháp mặc định nhưng phải kiểm soát độ nét của camera và nhiễu cảm biến; căn chỉnh vật thể khi có ít dữ liệu; luôn đặt lại ngưỡng khi đổi sản phẩm hoặc điều kiện chụp, và theo dõi tỷ lệ báo nhầm trên ảnh tốt theo thời gian thay vì chỉ nhìn AUROC; và kiểm tra các thống kê đơn giản của dữ liệu như độ sáng, góc xoay trước khi tin vào một kết quả quá tốt hoặc quá tệ.

Các hướng phát triển xuất phát trực tiếp từ hạn chế ở Chương 3. Autoencoder cần được chạy với nhiều seed và thử hàm mất mát SSIM [19] để biết phần nào của kết quả là do cấu hình. Ba kiểm tra cơ chế ở Mục 3.5.6 cần được thực hiện để khẳng định các giải thích dựa trên bản đồ bất thường. Các biện pháp khắc phục đã gợi ý cần được thử trực tiếp: căn chỉnh góc tự động cho vật thể xoay, chuẩn hóa độ sáng hoặc tăng cường dữ liệu cho Autoencoder và PaDiM, cắt ảnh lớn như sheet_metal thành nhiều mảnh, và đặt lại ngưỡng tự động khi phát hiện điều kiện chụp thay đổi. Cuối cùng, thực nghiệm có thể mở rộng ra toàn bộ các loại của ba bộ dữ liệu và sang các bộ dữ liệu gần với điều kiện sản xuất thật hơn, như Real-IAD với nhiều góc chụp cho mỗi sản phẩm.

TÀI LIỆU THAM KHẢO

[1] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD – a comprehensive real-world dataset for unsupervised anomaly detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9592–9600. DOI: 10.1109/CVPR.2019.00982 Accessed: Oct. 21, 2026. [Online]. Available: https://www.mvtec.com/fileadmin/Redaktion/mvtec.com/company/research/datasets/mvtec_ad.pdf

[2] T. Defard, A. Setkov, A. Loesch, and R. Audigier, "PaDiM: A patch distribution modeling framework for anomaly detection and localization," in *Pattern Recognition. ICPR International Workshops and Challenges*, ser. Lecture Notes in Computer Science, vol. 12664, Springer, 2021, pp. 475–489. DOI: 10.1007/978-3-030-68799-1_35 arXiv: 2011.08785. Accessed: Oct. 21, 2026. [Online]. Available: https://arxiv.org/abs/2011.08785

[3] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, "Towards total recall in industrial anomaly detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 14 318–14 328. arXiv: 2106.08265. Accessed: Oct. 21, 2026. [Online]. Available: https://arxiv.org/abs/2106.08265

[4] L. Heckler-Kram, J.-H. Neudeck, U. Scheler, R. König, and C. Steger, "The MVTec AD 2 dataset: Advanced scenarios for unsupervised anomaly detection," *International Journal of Computer Vision*, 2026. DOI: 10.1007/s11263-026-02743-0

[5] Y. Zou, J. Jeong, L. Pemula, D. Zhang, and O. Dabeer, "SPot-the-Difference self-supervised pre-training for anomaly detection and segmentation," in *Computer Vision – ECCV 2022*, ser. Lecture Notes in Computer Science, vol. 13690, Springer, 2022, pp. 392–408. DOI: 10.1007/978-3-031-20056-4_23

[6] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola. "Generalization," Dive into Deep Learning, Accessed: Oct. 21, 2026. [Online]. Available: https://d2l.ai/chapter_linear-regression/generalization.html

[7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016. Accessed: Oct. 21, 2026. [Online]. Available: https://www.deeplearningbook.org/

[8] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.

[9] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.

[10] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. DOI: 10.1109/5.726791

[11] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255. DOI: 10.1109/CVPR.2009.5206848

[12] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, 2012, pp. 1097–1105.

[13] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015, pp. 448–456. [Online]. Available: https://arxiv.org/abs/1502.03167

[14] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90

[16] S. Zagoruyko and N. Komodakis, "Wide residual networks," in *Proceedings of the British Machine Vision Conference (BMVC)*, 2016. [Online]. Available: https://arxiv.org/abs/1605.07146

[17] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola. "Multilayer perceptrons," Dive into Deep Learning, Accessed: Oct. 21, 2026. [Online]. Available: https://d2l.ai/chapter_multilayer-perceptrons/mlp.html

[18] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proceedings of the 27th International Conference on Machine Learning (ICML)*, 2010, pp. 807–814.

[19] P. Bergmann, S. Löwe, M. Fauser, D. Sattlegger, and C. Steger, "Improving unsupervised defect segmentation by applying structural similarity to autoencoders," in *Proceedings of the 14th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISAPP)*, 2019. [Online]. Available: https://arxiv.org/abs/1807.02011

[20] PyTorch. "Optimizing model parameters," PyTorch Tutorials, Accessed: Oct. 21, 2026. [Online]. Available: https://docs.pytorch.org/tutorials/beginner/basics/optimization_tutorial.html

[21] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010, pp. 249–256.

[22] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1026–1034.

[23] P. Micikevicius et al., "Mixed precision training," in *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: https://arxiv.org/abs/1710.03740

[24] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

[25] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in *Proceedings of the 25th International Conference on Machine Learning (ICML)*, 2008, pp. 1096–1103.

[26] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *International Conference on Learning Representations (ICLR)*, 2014. [Online]. Available: https://arxiv.org/abs/1312.6114

[27] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014. [Online]. Available: https://arxiv.org/abs/1411.1792

[28] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: https://arxiv.org/abs/1409.1556

[29] P. C. Mahalanobis, "On the generalised distance in statistics," *Proceedings of the National Institute of Sciences of India*, vol. 2, no. 1, pp. 49–55, 1936.

[30] W. B. Johnson and J. Lindenstrauss, "Extensions of Lipschitz mappings into a Hilbert space," *Contemporary Mathematics*, vol. 26, pp. 189–206, 1984.

[31] C. Wang et al., "Real-IAD: A real-world multi-view dataset for benchmarking versatile industrial anomaly detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 22 883–22 892.

[32] V. Zavrtanik, M. Kristan, and D. Skočaj, "DRAEM – a discriminatively trained reconstruction embedding for surface anomaly detection," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 8330–8339.

[33] D. Gudovskiy, S. Ishizaka, and K. Kozuka, "CFLOW-AD: Real-time unsupervised anomaly detection with localization via conditional normalizing flows," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2022, pp. 98–107.

[34] N. Cohen and Y. Hoshen, "Sub-image anomaly detection with deep pyramid correspondences," *arXiv preprint*, 2020. arXiv: 2005.02357. [Online]. Available: https://arxiv.org/abs/2005.02357

[35] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "Uninformed students: Student-teacher anomaly detection with discriminative latent embeddings," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 4183–4192.

[36] G. Wang, S. Han, E. Ding, and D. Huang, "Student-teacher feature pyramid matching for anomaly detection," in *British Machine Vision Conference (BMVC)*, 2021. [Online]. Available: https://arxiv.org/abs/2103.04257

[37] C.-L. Li, K. Sohn, J. Yoon, and T. Pfister, "CutPaste: Self-supervised learning for anomaly detection and localization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 9664–9674.

[38] O. Sener and S. Savarese, "Active learning for convolutional neural networks: A core-set approach," in *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: https://arxiv.org/abs/1708.00489

[39] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006. DOI: 10.1016/j.patrec.2005.10.010

[40] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[41] Scikit-learn. "Scikit-learn: Machine learning in Python," Accessed: Oct. 21, 2026. [Online]. Available: https://scikit-learn.org/stable/

[42] NumPy. "NumPy," Accessed: Oct. 21, 2026. [Online]. Available: https://numpy.org/

[43] Matplotlib. "Matplotlib: Visualization with Python," Accessed: Oct. 21, 2026. [Online]. Available: https://matplotlib.org/

[44] Nhóm 13, *Mã nguồn và tài liệu dự án MVTec anomaly detection*, 2026. Accessed: Oct. 21, 2026. [Online]. Available: https://github.com/AnDpTri/mvtec-anomaly-detection

PHỤ LỤC. MÃ NGUỒN VÀ CÁCH TÁI LẬP

Kho mã nguồn

Toàn bộ mã nguồn và tài liệu nằm trong kho mã nguồn của nhóm [44]:

https://github.com/AnDpTri/mvtec-anomaly-detection

Kết quả trong Chương 3 được tạo từ commit `d0216d1`.

Thư mục, tệp	Nội dung
`src/tn/data.py`	Đọc ảnh, mặt nạ và nhãn của MVTec AD, MVTec AD 2, VisA
`src/tn/splits.py`	Tách ảnh tốt thành phần fit và phần validation
`src/tn/methods.py`	Autoencoder, PaDiM, PatchCore: xây dựng mô hình và tính bản đồ bất thường
`src/tn/metrics.py`	AUROC, AUPRO, ngưỡng và các chỉ số tại ngưỡng
`tools/build_tn_models.py`	Xây dựng, đánh giá và lưu mô hình cho 9 loại
`tools/make_colab_notebook.py`	Sinh notebook Colab có nhúng sẵn mã nguồn
`tools/tn_report_figures.py`	Đo thời gian suy luận, vẽ hình minh họa của Chương 3
`src/demo/`	Ứng dụng minh họa Gradio

Dữ liệu

MVTec AD (grid, screw, transistor) tải từ bản sao trên Hugging Face bằng lệnh

```
python tools/download_mvtec.py <thư mục>/MVTecAD grid screw transistor
```

MVTec AD 2 (vial, sheet_metal) tải từ trang của MVTec, VisA tải từ kho của Amazon; đường tải cụ thể nằm trong `src/tn/data.py`. Thư mục dữ liệu cần có `MVTecAD/`, `MVTecAD2/` và `VisA/`.

Chạy lại thực nghiệm

Trên Google Colab: sinh notebook bằng `python tools/make_colab_notebook.py`, tải tệp `goi_colab/build_tn_models_colab.ipynb` lên Colab, chọn GPU T4 rồi chạy toàn bộ. Notebook tự tải dữ liệu, xây dựng mô hình cho 9 loại và ghi kết quả lên Google Drive.

Trên máy có GPU, từ thư mục gốc của kho mã nguồn, sau khi cài các thư viện trong `requirements.txt`:

```
python tools/build_tn_models.py --data <thư mục dữ liệu>
python -m src.demo.app
```

Lệnh đầu lưu mô hình, ngưỡng, chỉ số và điểm của từng ảnh test, mỗi loại một thư mục con trong `results/checkpoints_tn`, và bỏ qua các loại đã có kết quả, nên có thể chạy tiếp sau khi bị gián đoạn. Lệnh thứ hai mở ứng dụng minh họa tại `http://127.0.0.1:7864`.